ACESS - ADAPTIVE COGNITIVE AND EDUCATIONAL SKILL SUPPORT

MUHAMMAD ALIFF BIN AFFANDI

UNIVERSITI TEKNIKAL MALAYSIA MELAKA

ACESS - ADAPTIVE COGNITIVE AND EDUCATIONAL SKILL SUPPORT

MUHAMMAD ALIFF BIN AFFANDI

This report is submitted in partial fulfillment of the requirements for the Bachelor of Computer Science (Software Development) with Honours.

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY
UNIVERSITI TEKNIKAL MALAYSIA MELAKA

2026

DECLARATION

I hereby declare that this project entitled
ACESS - ADAPTIVE COGNITIVE AND EDUCATIONAL SKILL SUPPORT
is written by me and is my own effort and that no part has been plagiarized without citations.

STUDENT : _____ Date : _____
(MUHAMMAD ALIFF BIN AFFANDI)

I hereby declare that I have read this project report and found this project report is sufficient in term of the scope and quality for the award of Bachelor of Computer Science (Software Development) with Honours.

SUPERVISOR : _____ Date : _____
(TS. DR. ZURAINI BINTI OTHMAN)

DEDICATION

To my beloved parents, whose endless sacrifices and unwavering faith have been my foundation through every challenge. To Ts. Dr. Zuraini Binti Othman, whose mentorship and invaluable guidance shaped not only this research but also my growth as a researcher and software developer. To all neurodivergent learners who navigate a world not yet built for them, may ACESS serve as a small step toward a future where educational platforms adapt to every mind rather than demanding that every mind adapt to them.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor, Ts. Dr. Zuraini Binti Othman, for her continuous guidance, patience, and insightful feedback throughout the development of this project. Her expertise in software engineering and dedication to accessibility has been instrumental in shaping the ACESS platform into a functional and meaningful system.

My heartfelt thanks go to my family and friends, who offered unwavering emotional and moral support during the challenging moments of this journey. Their encouragement was a constant source of motivation during late nights and difficult debugging sessions.

ABSTRACT

Traditional e-learning platforms are commonly designed using a one-size-fits-all approach, creating accessibility challenges for neurodivergent learners, particularly those with Dyslexia, Attention Deficit Hyperactivity Disorder (ADHD), and Autism Spectrum Disorder (ASD). Existing learning management systems often provide limited support for these diverse cognitive needs, resulting in increased cognitive load and reduced learning effectiveness. This project presents the design and development of the Adaptive Cognitive and Educational Skill Support (ACESS) platform, a web-based learning management system that incorporates adaptive accessibility features to support learners with different cognitive profiles. The project adopts the Agile Scrum methodology, involving iterative planning, analysis, design, and development activities across six sprints, and is implemented using Next.js, Supabase, PostgreSQL, and Tailwind CSS. Key features include configurable accessibility presets for Dyslexia, ADHD and Autism, a distraction-free reading interface, an adaptive content recommendation engine, an authoring-time accessibility compliance auditor, an AI-assisted lesson summary and question-answering assistant, embedded H5P interactive content, and educator analytics, achievements, leaderboard ranking, and certificate generation for monitoring learner progress. The platform targets WCAG 2.2 Level AA conformance, though this has not yet been independently verified through formal audit or assistive-technology testing. The developed system was evaluated through structured walkthroughs guided by task scenarios for each learner profile, complemented by an expert review conducted against WCAG 2.2 success criteria to assess usability, accessibility, and alignment with the needs of neurodivergent learners. The outcomes of this project demonstrate the feasibility of integrating accessibility and adaptive learning mechanisms into a single platform, providing a foundation for a more inclusive digital learning environment for neurodivergent learners.

ABSTRAK

Platform e-pembelajaran tradisional sering direka menggunakan pendekatan satu-saiz-untuk-semua, yang menimbulkan cabaran kebolehcapaian bagi pelajar neurodivergen, khususnya mereka yang mengalami Disleksia, Attention Deficit Hyperactivity Disorder (ADHD), dan Autism Spectrum Disorder (ASD). Sistem pengurusan pembelajaran sedia ada sering memberikan sokongan terhad terhadap keperluan kognitif yang pelbagai ini, sekali gus meningkatkan beban kognitif dan mengurangkan keberkesanan pembelajaran. Projek ini membentangkan reka bentuk dan pembangunan platform Adaptive Cognitive and Educational Skill Support (ACESS), sebuah sistem pengurusan pembelajaran berasaskan web yang menggabungkan ciri kebolehcapaian adaptif bagi menyokong pelajar dengan profil kognitif yang berbeza. Projek ini menggunakan metodologi Agile Scrum, melibatkan aktiviti perancangan, analisis, reka bentuk dan pembangunan secara berulang merentasi enam sprint, dan dibangunkan menggunakan Next.js, Supabase, PostgreSQL, dan Tailwind CSS. Ciri utama termasuk praset kebolehcapaian boleh konfigurasi bagi Disleksia, ADHD dan Autisme, antara muka bacaan bebas gangguan, enjin pengesyoran kandungan adaptif, pengaudit pematuhan kebolehcapaian semasa penulisan kandungan, pembantu ringkasan pelajaran dan soal jawab berasaskan AI, kandungan interaktif H5P terbenam, serta analitik pendidik, pencapaian, kedudukan papan pendahulu, dan penjanaan sijil bagi memantau kemajuan pelajar. Platform ini menyasarkan pematuhan WCAG 2.2 Peringkat AA, walaupun perkara ini belum disahkan secara bebas melalui audit formal atau pengujian teknologi bantuan. Sistem yang dibangunkan ini dinilai melalui huraian berstruktur berpandukan senario tugasan bagi setiap profil pelajar, disokong oleh penilaian pakar berdasarkan kriteria kejayaan WCAG 2.2 untuk menilai kebolehgunaan, kebolehcapaian, dan keselarasan dengan keperluan pelajar neurodivergen. Hasil projek ini menunjukkan kebolehlaksanaan menggabungkan mekanisme kebolehcapaian dan pembelajaran adaptif dalam satu platform, seterusnya menyediakan asas bagi persekitaran pembelajaran digital yang lebih inklusif untuk pelajar neurodivergen.

TABLE OF CONTENTS

PAGE

DECLARATION ii
DEDICATION iii
ACKNOWLEDGEMENT iv
ABSTRACT v
ABSTRAK vi
TABLE OF CONTENTS vii
LIST OF TABLES xii
LIST OF FIGURES xv
LIST OF ABBREVIATIONS xvii
CHAPTER 1: INTRODUCTION 1
1.1 Background of Study 1
1.2 Problem Statement 2
1.3 Objectives 3
1.4 Scope 3
1.4.1 Target Users 3
1.4.2 Core System Modules 4
1.5 Significance of Project 6
1.6 Expected Output 6
1.7 Conclusion 7
CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY . . 9
2.1 Introduction 9
2.2 Theoretical Frameworks for Cognitive Accessibility 9
2.2.1 Universal Design for Learning (UDL) 9
2.2.2 Web Content Accessibility Guidelines (WCAG) 2.2 10

2.3 Neurodivergent Profiles and Interface Interventions 12
2.3.1 Attention Deficit Hyperactivity Disorder (ADHD) 12
2.3.2 Dyslexia 12
2.3.3 Autism Spectrum Disorder (ASD) 13
2.4 Comparative Analysis of Existing LMS Platforms 14
2.4.1 Moodle 14
2.4.2 Canvas LMS 15
2.4.3 Blackboard Learn 16
2.4.4 Google Classroom 16
2.5 Comparison Matrix 16
2.5.1 Alternative Approaches and Justification 19
2.6 Project Methodology 20
2.7 Project Requirements 21
2.7.1 Software Requirements 21
2.7.2 Hardware Requirements 23
2.7.3 Other Requirements 24
2.8 Project Schedule and Milestones 24
2.9 Summary 25
CHAPTER 3: SYSTEM ANALYSIS 27
3.1 Introduction 27
3.2 Problem Analysis 27
3.2.1 Current System Scenario 27
3.2.2 Detailed Problem Breakdown 28
3.3 Requirement Analysis 29
3.3.1 Data Requirement 29
3.3.2 Functional Requirement 30
3.3.3 Non-Functional Requirement 38
3.3.4 Others Requirement 42
3.4 System Architecture 43
3.4.1 Context Diagram 43
3.4.2 System Architecture Diagram 45
3.4.3 Deployment Diagram 47

3.5 Module Design 48
3.6 Summary 49
CHAPTER 4: SYSTEM DESIGN 50
4.1 Introduction 50
4.2 High-Level Design 51
4.2.1 Behavioral Design (Activity Diagrams) 51
4.2.2 Interaction Design (Sequence Diagrams) 56
4.2.3 User Interface Design 61
4.2.3.1 Navigation Design 61
4.2.3.2 Input Design 63
4.2.3.3 Output Design 67
4.2.4 Conceptual and Logical Database Design 70
4.3 Accessibility Design 78
4.3.1 Design approach and standards baseline 78
4.3.2 Mapping of standards to implementation 79
4.3.3 The three learner profiles 81
4.3.4 Preset composition 83
4.3.5 Effect of the presets on the lesson interface 85
4.3.6 Authoring-side compliance auditing 89
4.3.7 Known accessibility gaps 91
4.4 Data Dictionary and Normalisation 92
4.4.1 Normalisation 92
4.4.2 Core Identity and Configuration 94
4.4.3 Curriculum Structure 98
4.4.4 Lesson Content and Interaction 105
4.4.5 Assessment 108
4.4.6 Learner Progress, Recognition and Recommendation 110
4.5 Detailed Design 114
4.5.1 Software Design 114
4.5.2 Physical Database Design 119
4.5.3 Systemic Implementation Details 124
4.6 Limitations of Design 125

4.7 Summary . . . 127
CHAPTER 5: IMPLEMENTATION . . . 129
5.1 Introduction . . . 129
5.2 Development Environment Setup . . . 130
5.2.1 Hardware Environment . . . 130
5.2.2 Software Environment . . . 130
5.2.3 Local backend environment . . . 132
5.3 Software Configuration Management . . . 132
5.3.1 Configuration environment setup . . . 132
5.3.2 Version control procedure . . . 134
5.4 Implementation Status . . . 136
5.4.1 Verification performed during implementation . . . 138
5.5 Conclusion . . . 138
CHAPTER 6: TESTING . . . 140
6.1 Introduction . . . 140
6.2 Test Plan . . . 141
6.2.1 Test Organization . . . 141
6.2.2 Test Environment . . . 141
6.2.3 Test Schedule . . . 142
6.3 Test Strategy . . . 142
6.3.1 Test classes . . . 142
6.4 Test Design . . . 143
6.4.1 Automated end-to-end suite . . . 143
6.4.2 Manual spot-check matrix . . . 145
6.4.3 WCAG 2.2 expert review . . . 147
6.4.4 User acceptance testing: personas and form . . . 149
6.5 Test Results and Analysis . . . 151
6.5.1 Automated end-to-end suite: results . . . 151
6.5.2 Manual spot-check: results . . . 153
6.5.3 WCAG 2.2 expert review: results . . . 153
6.5.4 User acceptance testing: results . . . 153
6.6 Conclusion . . . 153

CHAPTER 7: CONCLUSION 155
7.1 Introduction 155
7.2 Project Summarization 155
7.3 Project Contribution 156
7.4 Project Limitation 157
7.5 Future Works 158
7.6 Conclusion 159
References 160

LIST OF TABLES

PAGE

Table 2.1 LMS Accessibility Capability Comparison 17
Table 2.2 Project Schedule and Milestones 25
Table 3.1 Functional Requirements: Learner Module 32
Table 3.2 Functional Requirements: Educator Module 35
Table 3.3 Functional Requirements: Administrator Module 37
Table 4.1 Input Validation Rules 64
Table 4.2 System Outputs 68
Table 4.3 Accessibility Criteria Mapping 79
Table 4.4 Learner Profiles and ACESS Support 82
Table 4.5 Accessibility Preset Values 83
Table 4.6 Users Data Dictionary 95
Table 4.7 User Profiles Data Dictionary 95
Table 4.8 Referral Codes Data Dictionary 96
Table 4.9 Instructor Applications Data Dictionary 96
Table 4.10 Contact Messages Data Dictionary 97
Table 4.11 Notifications Data Dictionary 97
Table 4.12 Accessibility Templates Data Dictionary 97
Table 4.13 Adaptive Interactions Data Dictionary 98
Table 4.14 Courses Data Dictionary 98
Table 4.15 Course Chapters Data Dictionary 100
Table 4.16 Lessons Data Dictionary 100
Table 4.17 Course Accessibility Categories Data Dictionary 102
Table 4.18 Course Milestones Data Dictionary 103
Table 4.19 Course Achievements Data Dictionary 103

Table 4.20 Lesson Templates Data Dictionary 104
Table 4.21 Lesson Versions Data Dictionary 104
Table 4.22 Media Assets Data Dictionary 105
Table 4.23 Lesson Interactive Content Data Dictionary 105
Table 4.24 Video Questions Data Dictionary 106
Table 4.25 H5P Contents Data Dictionary 106
Table 4.26 Lesson Checkpoints Data Dictionary 107
Table 4.27 Lesson AI Summaries Data Dictionary 107
Table 4.28 Lesson Comments Data Dictionary 108
Table 4.29 Quizzes Data Dictionary 108
Table 4.30 Quiz Questions Data Dictionary 109
Table 4.31 Quiz Options Data Dictionary 109
Table 4.32 Quiz Attempts Data Dictionary 109
Table 4.33 Quiz Answers Data Dictionary 110
Table 4.34 Enrollments Data Dictionary 110
Table 4.35 Lesson Progress Data Dictionary 111
Table 4.36 Learner Checkpoints Data Dictionary 111
Table 4.37 H5P Responses Data Dictionary 112
Table 4.38 Recommendations Data Dictionary 112
Table 4.39 Course Favorites Data Dictionary 112
Table 4.40 User Achievements Data Dictionary 113
Table 4.41 Certificates Data Dictionary 113
Table 4.42 Tables Removed During Schema Consolidation 120
Table 5.1 Development Workstation Specification 130
Table 5.2 Software Environment and Versions 131
Table 5.3 Implementation Status of Each Module 136
Table 5.4 Executable Verification Scripts 138
Table 6.1 Test Organization 141
Table 6.2 Test Schedule 142
Table 6.3 Automated Suite Test Descriptions 144
Table 6.4 Manual Spot-Check Matrix 146
Table 6.5 WCAG 2.2 Expert Review Findings 147

Table 6.6 UAT Form Structure 149
Table 6.7 UAT Likert Statements (asked once per persona) 150
Table 6.8 UAT Persona Task Briefs 150
Table 6.9 Automated Suite Results 151

LIST OF FIGURES

PAGE

Figure 2.1 Gantt chart of ACESS development schedule (14 weeks) 25
Figure 3.1 Current System Problem Analysis Diagram 28
Figure 3.2 Use Case Diagram for ACESS Platform 31
Figure 3.3 Context Diagram 44
Figure 3.4 System Architecture Diagram 45
Figure 3.5 Deployment Diagram 47
Figure 4.1 Student Activity Diagram 52
Figure 4.2 Educator Activity Diagram 54
Figure 4.3 Administrator Activity Diagram 55
Figure 4.4 Course Enrollment Sequence Diagram 56
Figure 4.5 Lesson Learning Sequence Diagram 57
Figure 4.6 Quiz Attempt Sequence Diagram 58
Figure 4.7 Course Creation Sequence Diagram 60
Figure 4.8 Learner Dashboard (Default State) 62
Figure 4.9 Educator Course Workspace (Lessons Tab) 63
Figure 4.10 Accessibility Settings Panel (Reading Group) 66
Figure 4.11 Quiz Interface 67
Figure 4.12 Learner Progress Page 69
Figure 4.13 Administrator Analytics Dashboard 69
Figure 4.14 Accessibility Analytics 70
Figure 4.15 Identity, Access and Platform ERD 73
Figure 4.16 Curriculum and Content ERD 75
Figure 4.17 Assessment, Learning Record and Recognition ERD 76
Figure 4.18 Complete ACESS Entity Relationship Diagram 77

Figure 4.19 Preset Preview Dialog 85
Figure 4.20 Default vs. Dyslexia Preset 86
Figure 4.21 Lesson Under the ADHD Preset 87
Figure 4.22 Lesson Under the Autism Preset 88
Figure 4.23 Learner Dashboard Under the Dyslexia Preset 89
Figure 4.24 Lesson Accessibility Check in the Educator Editor 90

LIST OF ABBREVIATIONS

ACESS	-	Adaptive Cognitive and Educational Skill Support
ADHD	-	Attention Deficit Hyperactivity Disorder
API	-	Application Programming Interface
ASD	-	Autism Spectrum Disorder
BDA	-	British Dyslexia Association
COGA	-	Cognitive and Learning Disabilities Accessibility
CSS	-	Cascading Style Sheets
DDL	-	Data Definition Language
ERD	-	Entity Relationship Diagram
H5P	-	HTML5 Package (Interactive Content Format)
HTML	-	Hypertext Markup Language
JSON	-	JavaScript Object Notation
JWT	-	JSON Web Token
LMS	-	Learning Management System
OOAD	-	Object-Oriented Analysis and Design
PDF	-	Portable Document Format
PostgREST	-	REST API Server for PostgreSQL
PostgreSQL	-	Object-Relational Database Management System
PSM	-	Projek Sarjana Muda
RBAC	-	Role-Based Access Control
React	-	JavaScript Library for Building User Interfaces
RLS	-	Row-Level Security
RPC	-	Remote Procedure Call
SSR	-	Server-Side Rendering
Supabase	-	Open-Source Backend-as-a-Service
Tailwind	-	Utility-First CSS Framework
Tiptap	-	Headless Rich-Text Editor
TTS	-	Text-to-Speech
UDL	-	Universal Design for Learning
UI	-	User Interface
URL	-	Uniform Resource Locator
Vercel	-	Cloud Platform for Frontend Deployment
WAI	-	Web Accessibility Initiative
WCAG	-	Web Content Accessibility Guidelines

CHAPTER 1: INTRODUCTION

1.1 Background of Study

In the current educational landscape, instruction is delivered through a blend of traditional face-to-face classroom settings and online platforms (Prensky, 2001). While digital tools have become increasingly common to support learning, many institutions still rely heavily on traditional teaching methods. This hybrid approach presents unique challenges for neurodivergent individuals, specifically those with Dyslexia, Attention Deficit Hyperactivity Disorder (ADHD), and Autism Spectrum Disorder (ASD). Whether learning occurs in a physical classroom or online, the tools used to deliver content often lack the flexibility required to accommodate diverse cognitive profiles.

Existing Learning Management Systems (LMS) are typically designed with a "one-size-fits-all" approach. They present dense textual information, complex navigation structures, and rigid assessment formats. These standard interfaces can erect significant barriers for neurodivergent learners. For instance, learners with Dyslexia may struggle with standard typography and line spacing (Franceschini et al., 2013), while those with ADHD might find cluttered interfaces and unstructured learning paths overwhelming (Barkley, 1997).

Therefore, there is a critical need for a natively adaptive learning platform that can seamlessly support both face-to-face and remote learning environments. The Adaptive Cognitive and Educational Skill Support (ACESS) platform is proposed to bridge this gap. ACESS is engineered to provide an inclusive digital learning environment that dynamically alters its interface and content delivery based on the specific cognitive requirements of the user. By integrating features such as dynamic typography

adjustment, focus-driven layouts, and rule-based adaptive learning paths, ACESS aims to ensure that educational materials are accessible and equitable for all learners, regardless of the instructional setting.

1.2 Problem Statement

Despite the availability of various educational technologies, significant cognitive accessibility barriers persist. The core problem this project addresses is the systemic inadequacy of current educational platforms to serve learners with disabilities.

Specifically, the problems identified are:

1. Most existing e-learning platforms are not designed for learners with disabilities. They use formats that overwhelm cognitive and sensory processing, leading to high dropout rates. Research indicates that learners with disabilities face significantly higher attrition rates in online education. For instance, only 46% of neurodivergent students completed online courses compared to 74% of their neurotypical peers (Spencer and Podesta, 2022), while up to 35% of students with ADHD drop out of e-learning programs within the first semester due to poor accessibility design (Burl and O'Brien, 2024).

2. Learners with disabilities lack platforms that adapt to their individual learning pace and present content in accessible, easy-to-process formats.

3. Employers and support organisations have no reliable way to verify the skills gained by learners with disabilities due to the absence of proper progress tracking and accessible certification.

4. Educators and mentors do not have a centralised platform to monitor learner progress, identify those who need extra support, and manage accessible learning materials effectively.

1.3 Objectives

This project aims to achieve the following objectives:

1. To analyze the accessibility requirements, learning challenges, and content delivery needs of learners with dyslexia, ADHD, and Autism Spectrum Disorder (ASD) through literature review and comparative study of existing e-learning platforms and assistive technology standards.
2. To develop a fully functional accessible web-based e-learning platform that incorporates rule-based adaptive content delivery, built-in Text-to-Speech (TTS) support, adjustable accessibility display settings, structured lesson and quiz management, and role-based user management for learners, educators, and administrators.
3. To implement a certificate generation module and an educator analytics dashboard that provide verifiable skill completion records and enable data-driven monitoring of individual learner progress.
4. To evaluate the functional correctness, usability, and accessibility of the developed platform through structured walkthroughs and expert review.

1.4 Scope

The scope of the ACESS platform defines the boundaries of the system architecture, target users, and core functional modules. The system is designed to act as a supportive educational tool for both traditional and online learning environments.

1.4.1 Target Users

- **Learners:** The primary consumers of the educational content. The interface is specifically optimized for neurodivergent individuals, though it remains highly beneficial for neurotypical users.

- **Educators:** The course creators and administrators who require structured, intuitive tools to author accessible content, manage media assets, and monitor student progress.
- **Administrators:** System operators responsible for platform governance, user role management, and global accessibility configuration.

1.4.2 Core System Modules

The ACESS architecture is divided into the following core modules:

- **Authentication and Profile Module:** Manages secure registration, login, role-based access control for learners, educators, and administrators, email verification, password reset, and profile management.
- **Accessibility and Personalization Engine:** The central differentiator of ACESS. Stores per-user accessibility preferences defining cognitive profiles with granular controls over font, spacing, theme, focus mode, reduced motion, and text-to-speech. Settings are applied globally, requiring no manual configuration after onboarding.
- **Educator Authoring Module:** Provides an interface for creating structured courses with chapters and milestones, writing lessons in a rich-text editor, embedding video with time-coded questions, designing interactive activities, managing lesson version history, and configuring quiz, certificate, and achievement criteria.
- **Accessibility Compliance Module:** A deterministic auditor embedded in the authoring workflow. It scores every lesson and course against a fixed rule set derived from WCAG 2.2, W3C COGA guidance and the British Dyslexia Association Style Guide, selected according to the course's declared accessibility focus profile. It reports each unmet standard, names the source of the rule, and offers one-click fixes where a fix can be applied automatically. The score is guidance to the educator rather than a publication gate.

- **Adaptive Learning Module:** Governs course enrollment, lesson progression, dynamic UI alterations based on accessibility profiles, and personalized recommendations triggered by quiz performance, including a rule-based engine that suggests revision, standard, or advanced content.
- **Interactive Assessment Module:** Handles timed and untimed quiz delivery, multiple attempts, automated grading, answer review, and pass/fail threshold evaluation that triggers remedial recommendations when scores fall below configured thresholds.
- **Analytics and Telemetry Module:** Tracks engagement metrics such as time per lesson, completion rates, and scores, and provides educators with per-student dashboards with at-risk detection, plus platform-wide analytics for administrators.
- **Certificate and Achievement Module:** Generates personalized PDF certificates upon course completion and administers a badge system that awards visual milestones based on predefined criteria to enhance motivation.
- **Interactive Content Module:** Supports five activity types (flashcards, drag-and-drop, fill-in-the-blanks, memory game and timeline) with dedicated builder and viewer components, plus time-coded video questions, embedded H5P content, and an optional generative-AI lesson assistant that produces plain-language lesson summaries and answers learner questions about the lesson currently open.
- **Communication and Notification Module:** Manages threaded lesson discussions, a contact form for inquiries, and a database-triggered notification system that alerts users on enrollment, lesson progress, quiz attempts, lesson additions, and course publication.
- **Localisation Module:** Presents the interface in English or Bahasa Melayu, switchable at runtime from the learner's own settings and persisted to the user profile.

1.5 Significance of Project

The development of the ACESS platform represents a significant step towards educational equity. By shifting the burden of accessibility from the user to the system architecture, the platform ensures that neurodivergent learners can focus entirely on comprehending the educational material rather than struggling with the interface.

For learners, the project provides a calm, predictable, and highly customizable environment that mitigates anxiety and cognitive fatigue. For educators, it offers a streamlined toolset that enforces accessibility best practices during content creation, alongside deep analytical insights into student engagement. Ultimately, ACESS demonstrates how modern web architecture can be leveraged to create a truly inclusive educational ecosystem applicable in both physical classrooms and remote learning scenarios.

1.6 Expected Output

The ACESS platform is expected to produce the following deliverables upon completion:

- **Adaptive Web Application:** A fully functional, production-ready learning management system accessible via standard web browsers and capable of dynamically adjusting its interface based on user accessibility profiles.
- **Role-Based Dashboards:** Three distinct interfaces for Learners, Educators, and Administrators, each with tailored navigation, permissions, and tooling appropriate to their respective responsibilities.
- **Accessibility Customisation System:** Three cognitive-profile presets (Dyslexia, ADHD, Autism) together with individually adjustable controls for typeface, text size, line and word spacing, background tint, reading measure, layout and pacing, motion, read-aloud, and the executive-function supports. A preset previews its

changes before applying them, and every value it sets remains adjustable afterwards.

- **Adaptive Recommendation Engine:** A rule-based engine that automatically analyzes quiz performance and generates personalized lesson recommendations to address identified knowledge gaps.
- **Course Authoring Tools:** A rich-text lesson editor integrated with media upload, five interactive activity builders, in-video question authoring and content embedding, with version snapshots for recovery.
- **Authoring-Time Accessibility Auditor:** A deterministic checker that scores each lesson and course against a fixed, profile-aware rule set drawn from published standards, reports the specific measured value behind every unmet standard, and offers a route to the fix.
- **Quiz and Assessment System:** A complete assessment lifecycle including timed and untimed quizzes, automated grading, multiple attempt management, and answer review capabilities.
- **Analytics Dashboard:** Visual engagement metrics and progress tracking with per-student and per-course views for educators, plus platform-wide analytics for administrators.
- **Certificate and Achievement System:** Automated PDF certificate generation upon course completion and a badge-based achievement system to incentivize learner progress.
- **Notification System:** Database-triggered real-time notifications for enrollment, quiz results, lesson additions, and course publication across all user roles.

1.7 Conclusion

This chapter established the foundational context for the ACESS project by identifying the critical accessibility barriers present in current educational platforms,

defining the specific problems to be addressed, and outlining the objectives and scope of the proposed system. The significance of the project was discussed in terms of its potential impact on educational equity for neurodivergent learners, and the expected deliverables were enumerated. The next chapter presents a comprehensive literature review, examining existing theoretical frameworks, analyzing current Learning Management Systems, describing the project methodology, and detailing the software and hardware requirements necessary to realize the ACESS platform.

CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY

2.1 Introduction

The literature review provides the academic and technical foundation upon which the ACESS platform is conceptualized. This chapter synthesizes contemporary research regarding cognitive accessibility in digital education, evaluates established theoretical frameworks such as Universal Design for Learning (UDL) (Meyer et al., 2014), and rigorously analyzes existing Learning Management Systems (LMS). By comparing platforms like Moodle (MoodleHQ, 2023b), Canvas (Instructure, Inc., 2023), Blackboard (Anthology Inc., 2023), and Google Classroom (Google LLC, 2023) against the specific requirements of neurodivergent learners, this chapter identifies the critical systemic gaps that justify the development of a natively adaptive, rule-based LMS architecture.

2.2 Theoretical Frameworks for Cognitive Accessibility

The design of digital interfaces for educational purposes must be grounded in empirically validated frameworks. The two primary pillars supporting the ACESS architecture are the Universal Design for Learning (UDL) guidelines (Meyer et al., 2014) and the Web Content Accessibility Guidelines (WCAG) 2.2 (W3C Web Accessibility Initiative, 2023).

2.2.1 Universal Design for Learning (UDL)

Universal Design for Learning is a framework developed by CAST (Meyer et al., 2014) aimed at optimizing teaching and learning for all individuals based on scientific

insights into how humans learn. UDL is predicated on three primary principles:

- **Multiple Means of Engagement:** Recognizing that learners differ markedly in the ways they are motivated. For neurodivergent learners, particularly those with ADHD, motivation can be severely hampered by unstructured environments (Barkley, 1997). ACESS implements this by offering gamified achievements and structured, step-by-step progressive disclosure of course material to prevent cognitive overwhelm.
- **Multiple Means of Representation:** Learners vary in how they perceive and comprehend information. A Dyslexic learner may struggle with printed text but excel with audio. ACESS implements this principle via integrated Text-to-Speech (TTS) capabilities and dynamic CSS injection that allows the user to radically alter the typography (e.g., using Atkinson Hyperlegible (Braille Institute of America, 2021)) and contrast ratios.
- **Multiple Means of Action and Expression:** Learners differ in how they navigate a learning environment and express what they know. ACESS supports this by providing multiple quiz formats (scenario-based, multiple-choice) and allowing untimed assessments to reduce anxiety-induced performance degradation.

2.2.2 Web Content Accessibility Guidelines (WCAG) 2.2

While UDL provides the pedagogical framework, WCAG 2.2 (W3C Web Accessibility Initiative, 2023) provides the strict, measurable technical parameters for digital accessibility. ACESS *targets* Level AA, together with four Level AAA criteria that matter specifically for reading, attention and sensory differences: 1.4.8 Visual Presentation, 2.2.6 Timeouts, 2.3.3 Animation from Interactions, and 3.2.5 Change on Request. This is stated as a design target rather than a conformance claim: at the time of writing no independent audit (automated or with assistive technology) has been carried out against the platform, and the published accessibility statement at `/accessibility-statement` records that limitation openly. Chapter 4 maps each

targeted criterion to the specific ACESS feature that addresses it, the learner profile it serves, and the source file in which it is implemented.

Because the barriers this project addresses are cognitive rather than purely perceptual, WCAG alone is not sufficient. ACESS therefore also works from the W3C *Making Content Usable for People with Cognitive and Learning Disabilities* guidance (W3C Web Accessibility Initiative, 2021), which supplies design objectives (chunking, avoiding memory load, making expectations clear before a task begins) that WCAG 2.2 does not express as testable success criteria, and from the British Dyslexia Association Style Guide (British Dyslexia Association, 2023), which supplies the typographic and prose-level thresholds used by the platform's own lesson auditor. Key criteria directly addressed by ACESS include:

- **1.4.8 Visual Presentation (AAA):** Line length is bounded by a `ch`-based reading measure so that it stays within the recommended range as the learner changes font size, rather than growing with the viewport.
- **1.4.12 Text Spacing (AA):** Line height, word spacing and font size are learner-controlled and applied as CSS custom properties on the document root, so a change takes effect immediately across lesson content without a reload.
- **2.2.1 Timing Adjustable (A) and 2.2.6 Timeouts (AAA):** Quiz time limits configured by an educator are displayed but never enforced by automatic submission, and the session-timeout warning is announced thirty seconds before it takes effect.
- **2.3.3 Animation from Interactions (AAA):** A single animation-level setting reduces or removes transition and motion effects across the whole interface.
- **3.2.5 Change on Request (AAA):** Applying an accessibility preset first shows the learner exactly which settings will change, and from what value to what value, before anything is altered. Text-to-speech never starts on its own.

2.3 Neurodivergent Profiles and Interface Interventions

Understanding the specific cognitive challenges faced by the target demographic is essential for architecting effective UI/UX interventions.

2.3.1 Attention Deficit Hyperactivity Disorder (ADHD)

ADHD is characterized by executive dysfunction (Zelazo et al., 2003), impacting working memory, task initiation, and sustained attention (Barkley, 1997). In digital environments, learners with ADHD are highly susceptible to ”saccadic fatigue,” the exhaustion caused by the eye constantly darting across a busy screen to parse disorganized information (Munoz et al., 2003). **ACESS Intervention:** A *distraction-free mode* that hides the primary navigation sidebar and notification chrome when a learner enters a lesson, combined with a bounded reading measure. The reading column is expressed in character units rather than pixels, so it stays within a comfortable range as the learner changes font size: 72 characters by default and 66 characters under the ADHD preset. Crucially, the measure is preserved *when* distraction-free mode is on rather than allowing the text to expand to the full viewport width, which was the behaviour before the constraint was corrected. The ADHD preset additionally raises a persistent “Now” bar carrying the single current task, an auto-save indicator and a task checklist, so that a learner who looks away can re-enter the lesson without reconstructing where they were. The technical implementation, including the activity diagram and the settings schema, is detailed in Chapter 4 (see Figure 4.1 and Table 4.5).

2.3.2 Dyslexia

Dyslexia involves difficulties with accurate or fluent word recognition and poor spelling and decoding abilities (Rello and Baeza-Yates, 2013). Research indicates that specific typographical adjustments can significantly improve reading velocity and comprehension (Franceschini et al., 2013). **ACESS Intervention:** The platform abandons reliance on browser-level zoom. Instead, a React context provider writes the

learner's stored preferences onto the document root as `data-*` attributes and CSS custom properties, which the stylesheet then consumes. Selecting the Dyslexia preset sets the typeface to Atkinson Hyperlegible (Braille Institute of America, 2021) (OpenDyslexic (Gonzalez, Abelardo, 2024) is also selectable), raises the base size to 19 px, sets the line-spacing multiplier to 1.7, sets word spacing to 0.16 em (the WCAG 1.4.12 floor), narrows the reading measure to 62 characters in line with the British Dyslexia Association's 50–70 character recommendation (British Dyslexia Association, 2023), applies a cream background to reduce glare, and enables both text-to-speech and a reading spotlight that dims the paragraphs the learner is not currently reading (Batanero et al., 2014). The preset implementation and the exact values it writes are documented in Chapter 4 (see Table 4.5 and Figure 4.23).

2.3.3 Autism Spectrum Disorder (ASD)

Learners on the Autism Spectrum often thrive in highly structured, predictable environments and can experience severe distress when confronted with unexpected auditory or visual stimuli (Lawson, 2011). **ACESS Intervention:** ACESS addresses predictability from two directions. On the learner side, the Autism preset fixes the reading column to a single width for every course, so that the page does not change shape between lessons, suppresses motion entirely, reduces colour saturation, and turns on a *visual schedule* that lists every phase of the lesson before the learner starts, together with step-by-step guidance that always states why a "Next" control is unavailable. Media never autoplays, and text-to-speech never starts on its own. On the authoring side, the compliance auditor described in Section 4.3.6 checks the educator's own draft for the barriers this profile is most sensitive to: figurative language, missing learning objectives, a missing plain-language summary, and structural inconsistency against the other lessons in the same course. The auditor advises. It does not rewrite the educator's content. These features are described in Chapter 4 (see Table 4.3 and Figure 4.24).

2.4 Comparative Analysis of Existing LMS Platforms

To justify the development of ACESS, an extensive evaluation of existing market-leading platforms was conducted. The platforms analyzed include Moodle (MoodleHQ, 2023b), Canvas LMS (Instructure, Inc., 2023), Blackboard Learn (Anthology Inc., 2023), and Google Classroom (Google LLC, 2023).

Before the individual platforms are discussed, one distinction must be made explicit, because it decides whether a comparison is fair. A capability can be present in three quite different ways:

- **Native:** shipped in the product's own interface and available to every learner without an administrator installing anything.
- **Plugin or add-on:** available, but only after an institution's administrator installs and configures a separate component, which the learner cannot do for themselves.
- **External:** not provided by the platform at all, and obtained from the operating system, the browser, or third-party assistive software.

The comparison in Table 2.1 is expressed in these terms rather than as a simple yes/no, because "Moodle cannot do this" would be false for almost any capability (Moodle's plugin ecosystem is very large) while "Moodle does this" would misrepresent what a learner actually finds on opening the product. What follows are the four platforms judged on that basis. It should be noted that all four are mature, well-resourced products with substantial accessibility programmes. The gap this project identifies is specifically in *learner-controlled cognitive adaptation*, not in accessibility generally.

2.4.1 Moodle

Moodle is an open-source learning platform with very wide global adoption and a modular architecture (MoodleHQ, 2023a). It maintains an active accessibility programme and documents conformance work publicly (MoodleHQ, 2024b), and its

editors ship an accessibility checker that flags problems such as missing image alternative text and low-contrast text while an educator writes (MoodleHQ, 2024a). Institution-wide auditing and remediation reporting are available through the widely used Brickfield Accessibility Toolkit (Brickfield Education Labs, 2024).

Limitations relative to this project's aims: the accessibility effort is directed primarily at authored-content correctness and at screen-reader and keyboard operability rather than at cognitive load. A learner cannot select a cognitive profile that simultaneously changes typography, reading measure, pacing and executive-function scaffolding. The closest equivalents are theme selection by an administrator and third-party blocks that must be installed first. Adaptive remediation is likewise conditional on an educator having authored the conditional activity rules in advance, and there is no rule-based engine that reads assessment performance and proposes revision content on its own.

2.4.2 Canvas LMS

Canvas by Instructure presents a more modern, card-based interface and publishes accessibility conformance documentation, including a Voluntary Product Accessibility Template (Instructure, Inc., 2024a; Instructure, Inc., 2023). Its Rich Content Editor includes an accessibility checker that an educator can run over a page before publishing (Instructure, Inc., 2024c), and Microsoft Immersive Reader, which provides read-aloud, line focus, and adjustable text spacing, is integrated into Canvas pages, although it must be enabled by the institution and is invoked per page rather than applied as a persistent learner profile (Instructure, Inc., 2024b).

Limitations relative to this project's aims: the cognitive supports that exist are delivered through an embedded third-party reader rather than as first-class platform settings, so they do not persist as a learner preference across the whole product, and they do not extend to assessment or to activity interfaces. Course progression remains linear unless an educator builds mastery paths by hand.

2.4.3 Blackboard Learn

Blackboard Learn is an enterprise platform whose accessibility strategy centres on Anthology Ally, which scores uploaded course materials for accessibility, gives instructors targeted remediation guidance, and generates alternative formats (including audio, ePub and tagged PDF) for learners to download (Anthology Inc., 2024a). Blackboard also provides a native Achievements tool for badges and certificates (Anthology Inc., 2024b).

Limitations relative to this project's aims: Ally operates on *files* rather than on the running interface. It will convert a lecture handout into an audio file, but it does not change the typography, spacing, reading measure or motion behaviour of the platform the learner is sitting in. The alternative-format model also places the learner outside the course, working on a downloaded artefact, rather than adapting the course itself.

2.4.4 Google Classroom

Google Classroom is a deliberately lightweight coordination layer over Google Workspace, and inherits the accessibility features of that ecosystem: screen-reader support, keyboard navigation, and the read-aloud and dictation tools built into Docs and Chrome (Google LLC, 2024; Google LLC, 2023).

Limitations relative to this project's aims: the supports live in the surrounding Workspace tools rather than in Classroom, and Classroom itself provides neither cognitive profiles, nor lesson-level engagement telemetry such as time-on-lesson, nor rule-based recommendation. Functionally it is an assignment distribution and collection system rather than a pedagogical support engine.

2.5 Comparison Matrix

Table 2.1 summarises the four platforms against ACESS using the native / plugin / external distinction introduced above. Every cell is a statement about how a capability is

delivered, not a claim that a platform is incapable of it.

Table 2.1: LMS Accessibility Capability Comparison

Capability	**Moodle**	**Canvas**	**Blackboard**	**Google Classroom**	**ACESS**
Read-aloud of lesson text	Plugin	Plugin: per page (Instructure, Inc., 2024b)	Plugin: audio alt-format (Anthology Inc., 2024a)	External: Workspace (Google LLC, 2024)	**Native**, learner setting
Learner-selectable dyslexia typeface and spacing	Plugin / theme (MoodleHQ, 2024b)	Plugin: per page (Instructure, Inc., 2024b)	External	External	**Native**, persistent
One profile changing typography, layout *and* pacing together	Not provided	Not provided	Not provided	Not provided	**Native**: three presets
Distraction-reduced view with bounded line length	Theme-dependent	Not provided	Not provided	Minimal UI by design	**Native**, measure-bounded
Automatic remedial recommendation from assessment	Educator-authored rules	Educator-authored paths	Educator-authored rules	Not provided	**Native**, rule-based
Authoring-time accessibility checker	Native, editor (MoodleHQ, 2024a)	Native, editor (Instructure, Inc., 2024c)	Native, file-level (Anthology Inc., 2024a)	Not provided	**Native**, profile-aware
Achievements / badges	Native	Native / add-on	Native (Anthology Inc., 2024b)	Not provided	**Native**

continued on the next page

Table 2.1 continued from the previous page

Capability	**Moodle**	**Canvas**	**Blackboard**	**Google Classroom**	**ACESS**
Lesson-level engagement telemetry	Native	Native	Native	Limited	**Native**
Reduced-motion control	External: OS	External: OS	External: OS	External: OS	**Native** setting
Published WCAG position	Documented (MoodleHQ, 2024b)	VPAT published (Instructure, Inc., 2024a)	Documented (Anthology Inc., 2024a)	Documented (Google LLC, 2024)	Targeted, not audited

Three points about Table 2.1 should be stated plainly, since a comparison table of this kind invites overreading.

First, the ACESS column is not a claim of superiority in accessibility overall. Moodle, Canvas and Blackboard all publish conformance documentation and maintain accessibility teams. ACESS does not yet have an independent audit at all, which is why its final row reads "targeted, not audited" rather than "compliant". On the evidence available, those three platforms have a stronger *verified* accessibility position than ACESS does.

Second, "Not provided" means the capability was not found in the vendor's own current documentation for the base product. It does not mean no third party offers it. The plugin ecosystems, Moodle's especially, are large, and an institution with budget and administrative capacity can close several of these gaps.

Third, the row that carries the actual argument of this project is the third one. Every one of these platforms can be made to enlarge text, and three of the four can be made to read text aloud. What none of them offers is a single learner-owned profile that changes typography, reading measure, content chunking, pacing and executive-function scaffolding *as one coherent decision*, without an administrator's involvement. That is the gap ACESS is built to occupy, and Section 4.3 documents what the platform actually

does in that space, along with what it does not yet do.

The ACESS entries in this table are evidenced by the current implementation: the screenshots in Chapter 4 (Figures 4.10 to 4.24), the preset value table (Table 4.5) generated from the platform's own preset definitions, and the WCAG mapping table (Table 4.3), which names the source file implementing each claim.

2.5.1 Alternative Approaches and Justification

Several alternative technical approaches were evaluated during the design of the ACESS platform before arriving at the selected architecture:

- **Monolithic Architecture vs. Decoupled Client-Server:** A monolithic Ruby on Rails or PHP Laravel application was considered for its simpler deployment model. However, this approach was rejected in favor of a decoupled Next.js frontend (Vercel Inc., 2023a) with Supabase backend-as-a-service (Supabase, 2023), which allows the accessibility engine to dynamically inject CSS variables on the client without full page reloads, preserving the fluid user experience critical for ADHD learners.

- **CSS-in-JS Libraries vs. Tailwind CSS:** Alternatives such as styled-components and Emotion were evaluated. While they offer dynamic styling, they introduce runtime overhead that degrades performance on low-end devices. Tailwind CSS v4 (Tailwind Labs, 2023) with Just-in-Time compilation and CSS custom properties was selected because it generates minimal production CSS and enables instantaneous theme switching without JavaScript re-computation.

- **Document Database vs. Relational Database:** MongoDB was considered for its schema flexibility. However, the relational integrity enforced by PostgreSQL with foreign key constraints and Row-Level Security (Supabase, 2024) was deemed essential for ensuring data consistency across enrollments, progress tracking, and quiz attempts, particularly given the complex relational model required by the adaptive recommendation engine.

- **Traditional State Management vs. React Context:** Redux was considered for global state management but was deemed overly verbose for the accessibility settings use case. React Context with the `AccessibilityProvider` pattern was chosen for its simplicity (Meta Platforms, Inc., 2023), native React integration, and minimal boilerplate when propagating CSS theme attributes across the component tree.

2.6 Project Methodology

The ACESS platform was developed using the **Agile Scrum** methodology (Schwaber, 1997), an iterative framework that emphasizes continuous delivery, regular stakeholder feedback, and adaptive planning. Scrum was selected over traditional sequential models such as Waterfall because designing for cognitive accessibility is an inherently empirical process. Assumptions about the effectiveness of specific interface interventions, such as a Dyslexia font or Focus Mode layout, must be validated through iterative user testing rather than deferred to the final stages of development.

The development lifecycle was organized into six sprints, each spanning two weeks:

- **Sprint 0: Requirements and Scaffolding.** Definition of the product backlog based on WCAG 2.2 guidelines and UDL principles. Establishment of the Next.js project structure, Supabase project initialization, and PostgreSQL schema design.
- **Sprints 1-2: Core Architecture.** Implementation of user authentication with Supabase Auth, database schema normalisation with Row-Level Security policies, and role-based route gating in the Next.js request interceptor (`src/proxy.ts`, since Next.js 16 renamed the former `middleware` entry point to `proxy`).
- **Sprints 3-4: Feature Development.** Development of the Educator authoring interface (Tiptap editor, course creation, quiz builder) and the Learner experience (course enrollment, lesson viewing, progress tracking).
- **Sprints 5-6: Advanced Features and Refinement.** Integration of the rule-based adaptive recommendation engine, accessibility personalization engine with CSS

variable injection, analytics dashboards, certificate generation, and notification triggers.

Each sprint concluded with a review and retrospective, ensuring continuous alignment with project objectives. A detailed breakdown of the Agile implementation is provided in Chapter 3.

2.7 Project Requirements

2.7.1 Software Requirements

The following software tools and technologies were utilized in the development and deployment of the ACESS platform:

- **Visual Studio Code:** Primary editor, with extensions for TypeScript, Tailwind CSS, and ESLint integration.
- **Node.js:** Runs the Next.js development server and build toolchain.
- **Git and GitHub:** Source control, with the repository hosted on GitHub for continuous deployment to Vercel.
- **Supabase CLI (Supabase Inc., 2025b):** Runs the entire Supabase stack locally in Docker containers (PostgreSQL, GoTrue authentication, PostgREST, Storage and Studio), applies database migrations, resets the database from scratch, and generates TypeScript types from the live schema.
- **Docker Desktop (Docker, Inc., 2025):** Container runtime hosting the local Supabase stack, allowing the full database, authentication and storage layers to be rebuilt deterministically on a development machine without touching the hosted project.
- **Vercel (Vercel Inc., 2023b):** Hosts the deployed Next.js application, with continuous deployment from the GitHub repository.

- **Next.js 16 (Vercel Inc., 2023a):** React framework with the App Router, providing server-side rendering, file-system routing and request-level route protection through its `proxy` entry point (Vercel Inc., 2024).
- **TypeScript (Microsoft, 2023):** Used throughout the codebase for compile-time type checking.
- **Tailwind CSS v4 (Tailwind Labs, 2023):** Styling framework, used for dynamic theme injection via CSS custom properties that drive accessibility customisation.
- **Supabase (PostgreSQL 17) (Supabase, 2023; PostgreSQL Global Development Group, 2025):** Managed PostgreSQL database with Row-Level Security, JWT-based authentication (Supabase Inc., 2025a), object storage, and an automatically generated REST interface over the schema via PostgREST (PostgREST Contributors, 2025).
- **Framer Motion (Framer, 2024):** Animation library for React used to implement smooth transitions while respecting the user's reduced-motion accessibility preference.
- **Tiptap (Brainhub, 2024):** Headless rich-text editor built on ProseMirror, integrated into the Educator authoring workflow for creating structured lesson content.
- **Recharts (Recharts Team, 2024):** Charting library for React used to render engagement analytics and progress visualisations on the educator and administrator dashboards.
- **jsPDF (Parallax Agency Ltd., 2025):** Client-side PDF generation library used to produce completion certificates and administrator reports, paired with a QR-code generator that encodes each certificate's public verification URL.
- **Google Gemini API (Google DeepMind, 2025):** Generative model service used for two optional, clearly labelled learner-facing features: a plain-language lesson summary and a lesson-scoped question-answering assistant. No assessment is graded by a model. Quiz scoring is deterministic and happens in the database.

- **Web Speech API (Mozilla Developer Network, 2025):** The browser's own speech-synthesis interface, used for the "Listen" text-to-speech control. Using the platform voice rather than a network service keeps the feature available offline-of-network and free of per-request cost.
- **DOMPurify (Heiderich, Mario and Cure53, 2025):** HTML sanitiser applied to rich-text lesson content before it is rendered, preventing stored cross-site scripting through the editor.
- **dnd kit (Bernier, Claudéric, 2025):** Drag-and-drop toolkit used by the interactive activity builders and viewers. Its keyboard sensor is what makes the drag-and-drop activity operable without a pointing device.
- **Nodemailer and Resend:** Electronic mail transports used for password-recovery and notification messages.
- **Figma (Figma, 2023):** UI/UX design tool used to prototype interface layouts and accessibility component mockups prior to implementation.

2.7.2 Hardware Requirements

The following hardware specifications were used during development and are recommended for optimal deployment:

- **Development Workstation:** A personal computer with a minimum of 8 GB RAM, a multi-core processor (Intel Core i5 or equivalent), and a solid-state drive with at least 256 GB of storage for running the development server, database migrations, and local testing.
- **Display:** A monitor with a minimum resolution of 1920x1080 pixels to adequately preview responsive layouts and accessibility adjustments across viewport sizes.
- **Network Connectivity:** A stable broadband internet connection with at least 10 Mbps download speed for accessing cloud services (Supabase, Vercel, GitHub).

- **Server Infrastructure:** Cloud-based infrastructure provided by Vercel (edge functions and static hosting) and Supabase (managed PostgreSQL and file storage), eliminating the need for dedicated physical server hardware.

2.7.3 Other Requirements

- **Supabase Account:** A cloud-hosted Supabase project for database management, authentication configuration, and storage bucket policies. A parallel local stack, started with the Supabase CLI under Docker, is used for schema development and for the verification scripts described in Chapter 5, so that destructive operations such as a full database reset are never carried out against the hosted project.
- **Google AI Studio API Key:** Credential for the Gemini API, required only for the optional lesson-summary and lesson-assistant features. The platform degrades gracefully when it is absent.
- **GitHub Account:** Repository hosting for source code version control and integration with Vercel for continuous deployment.
- **Vercel Account:** Deployment platform account linked to the GitHub repository for automated builds and hosting.
- **SMTP Email Service:** A Gmail SMTP configuration for sending password reset emails and account verification notifications via Nodemailer.
- **Web Browser:** A modern browser (Google Chrome, Mozilla Firefox, or Microsoft Edge) for testing and accessing the deployed application.

2.8 Project Schedule and Milestones

The ACESS project was executed across six Agile Scrum sprints, each spanning two weeks, for a total development duration of twelve weeks. Table 2.2 summarizes the milestone schedule.

Table 2.2: Project Schedule and Milestones

Sprint	Focus Area	Key Deliverables
0	Requirements and Scaffolding	Product backlog, Next.js project setup, Supabase initialization, database schema design
1	Core Authentication	User registration and login, role-based middleware, email verification flow
2	Database and Security	Row-Level Security policies, storage bucket configuration, data dictionary implementation
3	Educator Authoring	Course creation interface, Tiptap editor integration, quiz builder, media upload
4	Learner Experience	Course enrollment, lesson viewer, progress tracking, Focus Mode implementation
5	Adaptive and Analytics	Recommendation engine, accessibility presets, analytics dashboards, certificates, notifications

The Gantt chart in Figure 2.1 provides a visual representation of the project timeline, illustrating the scheduling and duration of each development phase across 14 weeks.

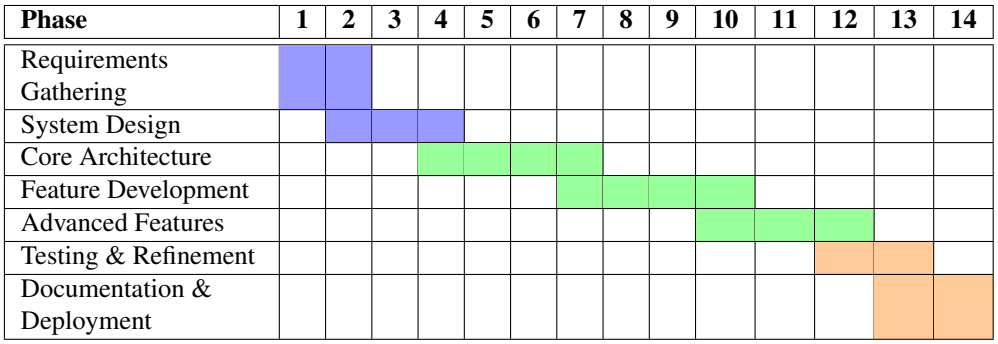


Figure 2.1: Gantt chart of ACESS development schedule (14 weeks)

2.9 Summary

The literature review demonstrates that while current LMS platforms provide robust administrative and content-hosting capabilities, they systematically fail to address

the specific cognitive requirements of neurodivergent learners. Reliance on external plugins, static content delivery models, and dense navigation hierarchies induce significant extraneous cognitive load (Sweller, 1988). The theoretical frameworks of UDL and WCAG 2.2 demand a more integrated approach. The comparative analysis definitively justifies the engineering of the ACESS platform: a system built from the database schema upward to be natively adaptive, utilizing dynamic CSS injection and rule-based server logic to provide an equitable, highly personalized learning environment. The project methodology, requirements, and schedule outlined in this chapter establish the roadmap for implementing the ACESS system in the subsequent phases.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Introduction

This chapter transitions the theoretical justifications established in the literature review into concrete software engineering specifications. It begins with a detailed problem analysis of existing educational platforms, followed by a comprehensive requirements analysis covering data, functional, non-functional, and other requirements. The chapter then presents the system architecture, module design, and deployment considerations necessary to execute a secure, highly accessible Learning Management System.

3.2 Problem Analysis

The ACESS platform is designed to address specific, systemic shortcomings in current educational technology. While the literature review in Chapter 2 established the general limitations of platforms such as Moodle, Canvas, Blackboard, and Google Classroom, this section analyzes the concrete operational problems that directly influenced the architectural decisions of ACESS.

3.2.1 Current System Scenario

Existing Learning Management Systems typically operate under a one-size-fits-all paradigm. Learners are presented with a fixed interface regardless of their cognitive profile, forcing neurodivergent users to seek third-party accommodations such as

browser extensions for text-to-speech, external tools for font customization, or manual workarounds for managing visual clutter. This places the burden of accessibility on the learner rather than the system.

From a data flow perspective, current platforms process learner interactions in a linear fashion: enrollment, content delivery, assessment, and completion. They lack the feedback loops necessary to detect when a learner is struggling and respond adaptively. For example, if a student performs poorly on a quiz about database normalization, the system continues to present the next topic as though no intervention is needed.

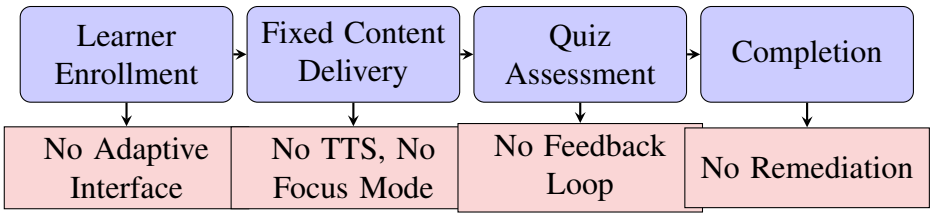


Figure 3.1: Current System Problem Analysis Diagram

3.2.2 Detailed Problem Breakdown

Figure 3.1 summarises that linear pipeline and the point at which each barrier arises. The problems identified in Chapter 1 are analysed here in greater technical depth:

1. **Rigid Typography and Layout:** Standard LMS platforms use fixed CSS stylesheets that do not respond to individual user needs. Learners with Dyslexia require specific typographic conditions, including increased letter spacing (tracking) of at least 0.12em, line height of 1.5 to 1.8, and sans-serif fonts such as Atkinson Hyperlegible (Braille Institute of America, 2021) designed to prevent letter confusion, such as differentiating ’b’ from ’d’ (Batanero et al., 2014). Without these adjustments, reading velocity decreases and cognitive fatigue accelerates, leading to early lesson abandonment.

2. **Cognitive Overload from Navigation:** Platforms like Moodle and Blackboard present deeply nested navigation menus (often three to four levels deep) combined

with dense dashboard widgets showing calendars, announcements, grades, and messages simultaneously. For learners with ADHD (Barkley, 1997), this multi-stimulus environment triggers rapid attention fragmentation, making it difficult to sustain focus on a single learning objective. The absence of a distraction-free mode means learners must manually close or minimize interface elements.

3. **Lack of Adaptive Remediation:** Current assessment systems grade quizzes and record scores but do not analyze performance patterns to generate corrective action. When a learner fails a prerequisite concept, the system does not automatically unlock remedial content or recommend revision. This linear progression model assumes uniform learner readiness, which is particularly detrimental for neurodivergent students who may require repeated exposure to foundational topics before advancing.

3.3 Requirement Analysis

3.3.1 Data Requirement

The ACESS platform requires a robust relational data model to manage the complex relationships between users, courses, lessons, assessments, and accessibility preferences. The system must store and process the following categories of data:

- **User Data:** Authentication credentials (email, password hash), profile information (name, avatar, role), and notification preferences. User data must be isolated by role with strict access controls.
- **Accessibility Data:** Per-user cognitive profile settings, including the active preset, font family and size, line-spacing multiplier, word spacing, background tint, theme, layout and structure mode, animation level, text-to-speech and caption toggles, and the executive-function supports (task checklist, visual schedule, step-by-step guidance, progress timeline, auto-save indicator). These settings must

persist across sessions and apply globally. They are stored as a single JSON document on the user's profile row rather than as a wide table of columns, because the setting vocabulary is expected to change more often than the rest of the schema and because every consumer reads the whole object at once. Section 4.4 discusses the normalisation trade-off this represents.

- **Curriculum Data:** Hierarchical course structures (courses, chapters, lessons) with rich-text HTML content, video assets, transcripts, and metadata such as difficulty level, estimated duration, and prerequisite relationships that drive the adaptive engine.
- **Assessment Data:** Quiz configurations (time limits, pass thresholds, max attempts), question banks with multiple-choice and scenario formats, answer options with correctness flags, and learner attempt records including scores and answer selections.
- **Telemetry Data:** Granular engagement metrics including lesson view status, time spent per lesson (in seconds), enrollment status (active, completed, dropped), and quiz attempt history with pass/fail outcomes.
- **Content and Achievement Data:** Interactive activity configurations (flashcards, drag-and-drop, memory games, timelines), certificate records with unique reference codes, and badge/achievement definitions and user progress toward them.
- **Notification and Communication Data:** System-generated notifications triggered by enrollment, quiz attempts, lesson additions, and course publications, along with lesson-level threaded comments and contact form submissions.

The detailed database schema and data dictionary for these entities are presented in Section 4.4.

3.3.2 Functional Requirement

Functional requirements define the explicit behaviors, processes, and services the system must execute. They are derived directly from the system's operational architecture

and are categorized by user role.

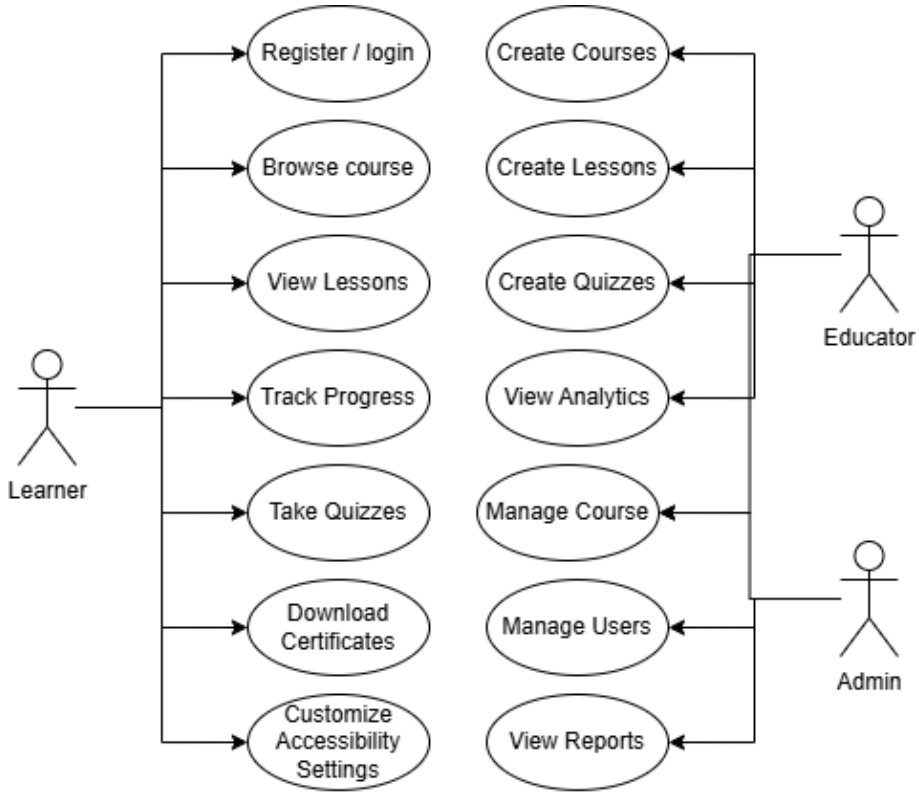


Figure 3.2: Use Case Diagram for ACESS Platform

Use Case Diagram Figure 3.2 visually maps the interactions between the primary actors (Student, Educator, Admin) and the system's functional boundaries, which are explicitly detailed in the subsequent tables.

Learner Functions Table 3.1 lists the functional requirements of the learner module.

Table 3.1: Functional Requirements: Learner Module

Req ID	Description	Actor	Inputs	Outputs
FR-STU-01	Registration and secure login.	Learner	Email, password	Supabase Auth session (JWT in an HTTP-only cookie), and a provisioning trigger creates the matching `users` and `user_profiles` rows with the role pinned to `learner`.
FR-STU-02	Accessibility preset and setting selection.	Learner	Preset choice, or any individual setting	Preview of what will change, then an UPDATE of `user_profiles.accessibility_prefs`, after which the document root's `data-*` attributes and CSS custom properties change immediately.
FR-STU-03	Course discovery and enrolment.	Learner	`course_id` selection	INSERT into `enrollments` under RLS, and a notification trigger informs the educator.
FR-STU-04	Lesson viewing with adaptive presentation.	Learner	Navigation to `/learner/lesson/[id]`	Renders sanitised HTML at the profile's reading measure, layout mode and typography, optionally hiding navigation chrome.
FR-STU-05	In-video questions.	Learner	Playback reaching a stored timestamp	Pauses the YouTube player and overlays the question from `video_questions`.

continued on the next page

Table 3.1 continued from the previous page

Req ID	Description	Actor	Inputs	Outputs
FR-STU-06	Quiz attempt.	Learner	Selected option identifiers	Calls a database function that grades server-side and writes `quiz_attempts` and `quiz_answers`. The client never receives the answer key before submission.
FR-STU-07	Automated progress telemetry.	Learner	Dwell time on the lesson route	UPDATEs `lesson_progress.time_spent_learning`, `view_count` and the viewed/completed flags.
FR-STU-08	Achievement earning.	Learner	Course progress reaching a threshold	A database function derives completion and inserts the earned `user_achievements` rows. The client cannot award a badge directly.
FR-STU-09	Certificate claim and download.	Learner	Course completion	A database function gates the claim, a reference code and public verification URL are issued, and the PDF is rendered client-side with jsPDF.
FR-STU-10	Text-to-speech playback.	Learner	Press of the Listen control	Speech synthesis of the lesson text at the learner's chosen rate. Never starts automatically.

continued on the next page

Table 3.1 continued from the previous page

Req ID	Description	Actor	Inputs	Outputs
FR-STU-11	Adaptive recommendations.	Learner	Quiz results, viewing history, favourites	Reads `recommendations` and displays revision, next-lesson, near-completion and cross-course suggestions with the reason for each.
FR-STU-12	Favourites and lesson discussion.	Learner	Save action, comment text	INSERT into `course_favorites`, and a threaded INSERT into `lesson_comments`.
FR-STU-13	AI lesson assistant.	Learner	Question about the open lesson	A model-generated answer scoped to that lesson's content, plus a cached plain-language summary in `lesson_ai_summaries`.
FR-STU-14	Interface language switch.	Learner	English or Bahasa Melayu	Interface strings re-render immediately, and the choice persists to `user_profiles.preferred_language`.

Educator Functions Table 3.2 lists the functional requirements of the educator module. Requirement FR-EDU-05 is the one with no counterpart in the platforms surveyed in Chapter 2, and its design is set out in Section 4.3.6.

Table 3.2: Functional Requirements: Educator Module

Req ID	Description	Actor	Inputs	Outputs
FR-EDU-01	Course creation and editing.	Educator	Title, metadata, difficulty, accessibility focus profile	INSERT/UPDATE on `courses`, restricted by RLS to rows the educator created.
FR-EDU-02	Lesson authoring.	Educator	Rich text, headings, images, tables, links	Sanitised HTML written to `lessons.content_html`, with a snapshot appended to `lesson_versions`.
FR-EDU-03	Quiz and activity creation.	Educator	Question text, options, correct-option flag, activity configuration	Populates `quiz_questions` and `quiz_options`, and activity payloads are stored as typed JSON in `lesson_interactive_content`.
FR-EDU-04	Asset management.	Educator	Image, document and media files	Uploads to the `course-assets` storage bucket and records the reference in `media_assets`. Instructional video is embedded from an external host rather than uploaded.

continued on the next page

Table 3.2 continued from the previous page

Req ID	Description	Actor	Inputs	Outputs
FR-EDU-05	Accessibility compliance check.	Educator	The lesson draft currently open, plus the course's focus profile	A score out of 100, a pass/fail list of the applicable standards, the source of each rule, the editor tab that fixes it, and a one-click fix where one exists. Re-evaluated on every keystroke against unsaved form state.
FR-EDU-06	Student monitoring and analytics.	Educator	Course or student selection	Recharts visualisations aggregating `lesson_progress` and `quiz_attempts`, with at-risk flagging.
FR-EDU-07	Achievement and milestone configuration.	Educator	Requirement type and threshold	Rows in `course_achievements` and `course_milestones`.
FR-EDU-08	Certificate issue and revocation.	Educator	Learner selection, revocation reason	INSERT or UPDATE on `certificates`. A revoked certificate keeps its row and reference code so that verification reports the revocation rather than a missing record.
FR-EDU-09	Preview as learner.	Educator	Course or lesson selection	Renders the learner-facing view without creating an enrolment or progress record.
FR-EDU-10	Lesson version history.	Educator	Version selection	Restores a previous `content_html` snapshot from `lesson_versions`.

Administrator Functions Table 3.3 lists the functional requirements of the administrator module.

Table 3.3: Functional Requirements: Administrator Module

Req ID	Description	Actor	Inputs	Outputs
FR-ADM-01	User role management.	Admin	Target user, new role	A service-role API route UPDATEs `users.role` *and* the corresponding authentication metadata, so that routing and database policy agree. A guard trigger blocks any attempt to change the same columns from a normal session.
FR-ADM-02	Course moderation.	Admin	Approve or reject	UPDATEs `courses.status` from `pending_review` to `published` or back to `draft`, and a notification trigger informs the educator.
FR-ADM-03	Educator application review.	Admin	Application decision, reviewer notes	UPDATEs `instructor_applications`. Approval promotes the applicant to `educator` in both the users table and the authentication metadata.

continued on the next page

Table 3.3 continued from the previous page

Req ID	Description	Actor	Inputs	Outputs
FR-ADM-04	System course creation.	Admin	Course metadata	Creates a course flagged `system_course = true` and `managed_by_admin = true`, which the learner catalogue presents as an official course.
FR-ADM-05	Platform analytics.	Admin	Date-range selection	Aggregates registrations, enrolments, completion, learning time, and accessibility adaptation and preset-adoption counts drawn from `adaptive_interactions`.
FR-ADM-06	Report generation.	Admin	Report type, date range	A rendered PDF report, generated client-side with jsPDF, covering platform, course, learner and content-coverage views.
FR-ADM-07	Certificate oversight.	Admin	Search or filter	Read access across all issued certificates, with revocation status.
FR-ADM-08	Enquiry handling.	Admin	Message status change	UPDATEs `contact_messages.status` across the unread, read and replied states.

3.3.3 Non-Functional Requirement

Non-functional requirements (NFRs) specify the critical quality attributes and systemic constraints that govern the operation of the ACESS platform.

Performance

- **Page Response Time:** Client-side route transitions should complete within **300 ms** so that navigation does not itself become a source of cognitive disruption.
- **Course Loading:** Initial course data fetches should resolve within **800 ms** on a broadband connection.
- **Setting Application Latency:** A change to any accessibility setting must be visible **without a page reload**. This is the strictest performance requirement in the system, because a learner adjusting spacing or size is engaged in a feedback loop and cannot evaluate the change if it does not appear immediately. It is met by writing CSS custom properties and `data-*` attributes on the document root rather than by re-rendering the component tree.
- **Video Loading:** Instructional video is embedded from an external host rather than served from platform storage. Playback start therefore depends on that host, and no delivery target is asserted for it here.
- **Concurrency:** The managed PostgreSQL instance uses connection pooling supplied by the platform. No concurrent-user figure is claimed, because no load test has been carried out on this project. Establishing one is identified in Chapter 4 as future work.

Security

- **Authentication:** All user sessions are secured with JSON Web Tokens managed by Supabase Auth (Supabase Inc., 2025a) and carried in HTTP-only cookies, with a 15-minute inactivity timeout that warns 30 seconds before it takes effect.
- **Authorisation (RBAC):** The request interceptor at `src/proxy.ts` gates every non-public route on the role claim in the session token. This layer is deliberately treated as a *routing* control, not as the security boundary: the authoritative check is made again inside the database by RLS policies that read `public.users.role`, and inside privileged API routes that re-verify the caller under the service

role. A forged token therefore obtains a user-interface shell and nothing behind it. Section 4.5.2 documents this two-layer arrangement and its rationale.

- **Database Protection:** Row-Level Security must be enabled on **every** table in the application schema (currently 36 of 36, carrying 88 policies), so that isolation holds at the storage layer regardless of which client issues the query. This is verified by an automated probe (`npm run verify:rls`) rather than asserted.
- **Privilege Guards:** Three database trigger functions prevent a learner from forging an outcome even with a valid session: one blocks self-promotion by rejecting a client attempt to change a privileged account column, another blocks declaring a course complete outside the normal completion process, and a third blocks writing a quiz score directly instead of through the grading function. They exempt only the service role and the platform's own trusted derivation logic.
- **Input Handling:** Rich-text lesson content is sanitised with DOMPurify (Heiderich, Mario and Cure53, 2025) before rendering, which is the platform's principal cross-site-scripting control. All database access goes through parameterised PostgREST or `pg` queries, so string-concatenated SQL does not occur. Field-level validation is performed in the form components and enforced a second time by `CHECK` constraints in the schema. A single shared schema-validation library is *not* currently used, and adopting one is listed as future work in Section 4.6.

Accessibility

- **WCAG Target:** The platform *targets* **WCAG 2.2 Level AA** (W3C Web Accessibility Initiative, 2023) together with four Level AAA criteria (1.4.8, 2.2.6, 2.3.3, 3.2.5). Body-text contrast must meet or exceed 4.5:1. Conformance is a target rather than an audited result, discussed further in Section 4.3.
- **Dyslexia Support:** The system must provide the Atkinson Hyperlegible (Braille Institute of America, 2021) and OpenDyslexic (Gonzalez, Abelardo, 2024) typefaces, a line-spacing multiplier adjustable from 1.0 to 3.0, and word spacing

reaching at least the WCAG 1.4.12 floor of 0.16 em at a setting the learner can actually reach on the slider.

- **Reading Measure:** Lesson text must be bounded by a measure expressed in character units, so that line length remains within the 50–80 character band as font size changes: 72 characters by default, 62 under the Dyslexia preset, 66 under the ADHD preset. The bound must hold when navigation chrome is hidden, not be released by it.
- **ADHD and Autism Support:** A single animation-level setting must reduce or remove motion across the interface. No media may autoplay, and text-to-speech must never begin without an explicit action.
- **Screen Reader Compatibility:** Custom components must expose the appropriate WAI-ARIA states, navigation must be built from real links rather than click handlers, and every interactive activity must be operable from the keyboard. It must be recorded, however, that no assistive-technology testing has yet been performed against this build. This requirement is therefore currently met by construction and code review only.

Usability

- **Consistent Navigation:** The primary navigational architecture must remain structurally identical across all modules to ensure predictability for Autistic learners.
- **Error Prevention:** Destructive actions (e.g., an Educator deleting a course) must require a confirmed multi-step verification modal to prevent accidental data loss.

Reliability

- **Data Integrity:** The database schema must strictly enforce foreign key constraints with cascade deletes where appropriate to prevent orphaned records, such as a lesson progress record existing without a valid enrollment.

- **Backup and Reproducibility:** The hosted Supabase project relies on the provider's managed backup facility. Independently of that, the entire schema must be reconstructible from version-controlled migrations, and the demonstration dataset from a deterministic seed script, so that a complete working database can be rebuilt from the repository alone. This is exercised routinely (`npm run db:rebuild`) and is what makes the figures in Chapter 4 reproducible.

Maintainability and Scalability

- **Modular Architecture:** The codebase must adhere to strict separation of concerns, decoupling the data-fetching logic (Server Components) from the interactive accessibility logic (Client Components).
- **Database Normalization:** The schema must adhere to the Third Normal Form (3NF) to eliminate data redundancy, ensuring the database scales efficiently as course and user volume increases.

3.3.4 Others Requirement

The software and hardware tools selected for the ACESS platform, as listed in Chapter 2, Section 2.4, are justified by the following architectural considerations:

- **Next.js 16 and React (Vercel Inc., 2023a):** Server-side rendering reduces the initial JavaScript payload, which matters for the rapid page loads that prevent cognitive drop-off for ADHD learners. The App Router's request interceptor (`src/proxy.ts`) performs role-based route gating before a protected page is rendered (Vercel Inc., 2024).
- **Supabase with PostgreSQL (Supabase, 2023):** Row-Level Security provides data isolation (Supabase, 2024) at the database level rather than relying solely on

application-layer guards, ensuring that cognitive telemetry and assessment data remain cryptographically isolated per user.

- **Tailwind CSS v4 (Tailwind Labs, 2023):** Just-in-Time compilation enables dynamic CSS variable injection, allowing instantaneous theme and typography switching without JavaScript re-evaluation, which is essential for real-time accessibility adjustments.
- **TypeScript (Microsoft, 2023):** Static typing across the full stack reduces runtime errors in critical paths such as quiz scoring and accessibility profile application, where a type mismatch could result in incorrect interface rendering.
- **Vercel (Vercel Inc., 2023b):** Serverless edge deployment provides global low-latency access to the platform, which is particularly important for learners whose cognitive focus may be disrupted by slow page transitions.

3.4 System Architecture

The ACESS platform employs a modern, decoupled client-server architecture, leveraging edge computing paradigms to maximize performance and security.

3.4.1 Context Diagram

The Context Diagram establishes the highest-level view of the system, illustrating the boundaries of the ACESS platform and its interactions with external entities.

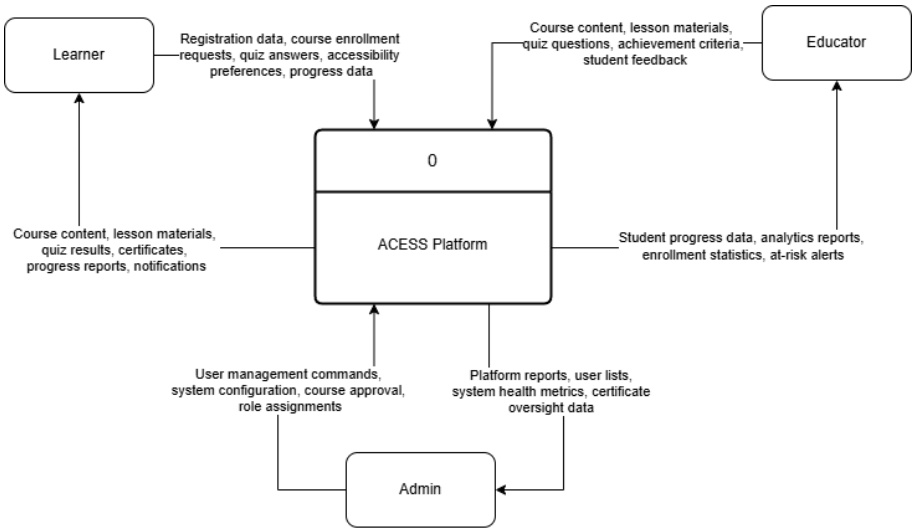


Figure 3.3: Context Diagram

As depicted in Figure 3.3, the ACESS system acts as the central processing hub. The primary external entities are the Learner (who consumes content and generates engagement telemetry), the Educator (who authors content and configures assessments), and the Administrator (who governs system integrity). Four external services sit on the system boundary:

- an SMTP mail provider for password-recovery and notification messages
- an external video host, from which instructional video is embedded rather than uploaded
- the Gemini generative-model service, used only for the optional lesson summary and lesson assistant
- the browser's own Web Speech synthesis engine (Mozilla Developer Network, 2025), which performs text-to-speech locally on the learner's device rather than through a network service

That last choice is deliberate: it keeps read-aloud free of per-request cost and keeps lesson text from leaving the device in order to be spoken.

3.4.2 System Architecture Diagram

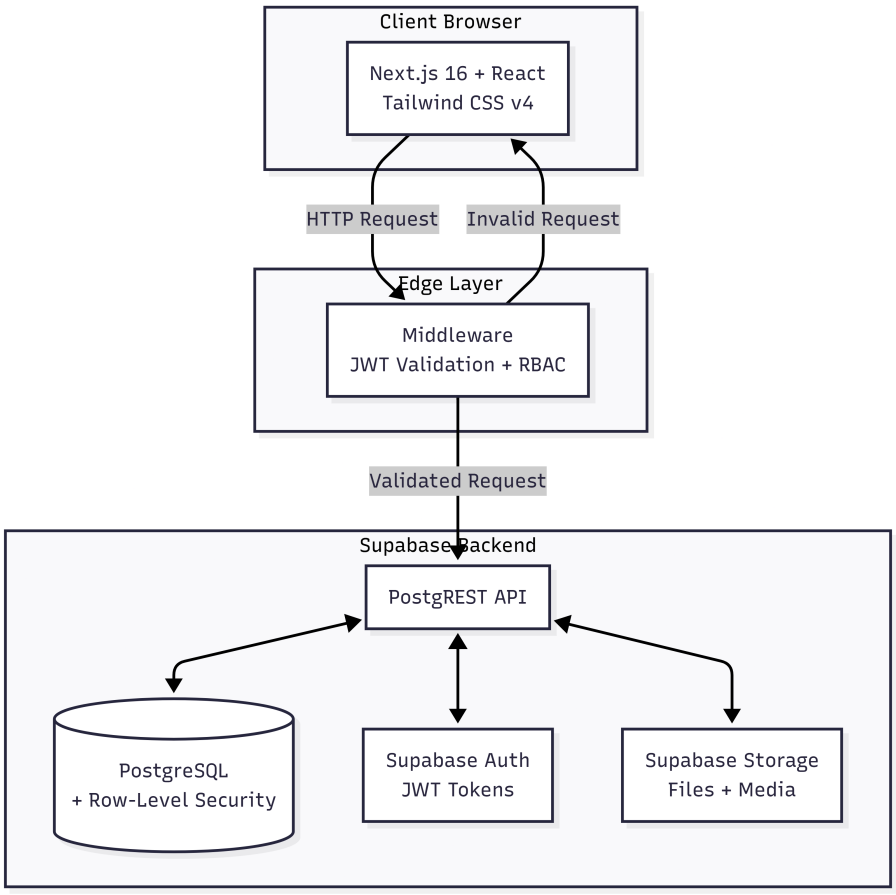


Figure 3.4: System Architecture Diagram

Figure 3.4 details the three-tier decoupled architecture:

- **Frontend Architecture (Next.js 16 and React):** The presentation layer is built on the Next.js App Router. It utilizes React Server Components to execute data fetching securely on the server, drastically reducing the JavaScript payload delivered to the client. This is crucial for rapid page loads, preventing cognitive drop-off for ADHD learners. Client components are restricted to highly interactive elements, such as the rich-text editor and interactive quiz interfaces. The UI is styled utilizing Tailwind CSS v4, which enables the dynamic injection of CSS variables for accessibility toggling.
- **Request Interception and Authentication Layer:** A Next.js request interceptor (`src/proxy.ts`) runs before every non-public route. It validates the session

cookie with Supabase Auth, rejects suspended or deleted accounts, and gates the `/learner`, `/educator` and `/admin` prefixes on the role claim, redirecting unauthorised attempts before the page is rendered. It reads the role from the session token rather than the database, deliberately, to avoid a database round trip on every request. The authoritative role check is repeated in the database and in privileged API routes, so this layer decides *which shell a user sees*, not *which rows they may touch*.

- **Backend and Database Architecture (Supabase):** Supabase provides a managed PostgreSQL 17 database exposed over HTTP by PostgREST (PostgREST Contributors, 2025). Security does not rest on the application layer: 88 Row-Level Security policies across all 36 tables are written into the schema itself, three guard triggers prevent privilege and outcome forgery, and the two operations that must not be client-decided (quiz grading and course completion) are implemented as `SECURITY DEFINER` database functions rather than as client logic.

- **Storage Architecture:** Three Supabase Storage buckets hold unstructured assets: `course-assets` for lesson media and course thumbnails, `certificates` for educator-uploaded custom certificates, and `avatars` for profile pictures. All three are public-read, because the application distributes public URLs that are rendered directly in the page. Write access is restricted by policy, and a learner may only write inside a folder named for their own user identifier. Instructional video is *not* stored in these buckets. It is embedded from an external host, and completion certificates are generated in the browser at download time rather than persisted as files.

3.4.3 Deployment Diagram

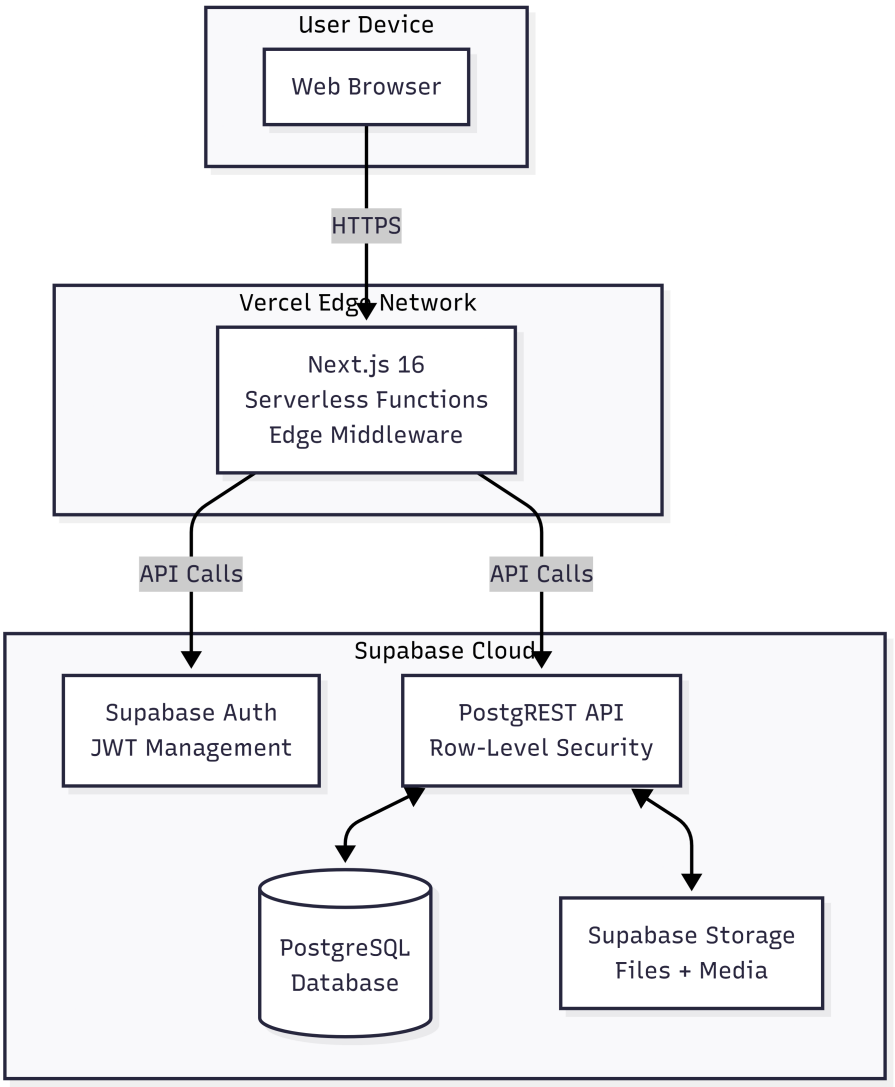


Figure 3.5: Deployment Diagram

The Deployment Diagram (Figure 3.5) illustrates the physical distribution of the system components. The Next.js application is deployed to Vercel's serverless platform (Vercel Inc., 2023b), which builds from the GitHub repository on each push to the release branch. The Supabase backend operates on managed cloud infrastructure, exposing the schema as a REST interface through PostgREST (PostgREST Contributors, 2025), which both the browser client (under the anonymous key, and therefore under RLS) and the server-side API routes consume. A small number of routes (account provisioning, role changes, recommendation generation and certificate issue) use the service-role key and therefore bypass RLS by design. These run only on the server and are the reason those

operations are implemented as API routes rather than as client calls.

A second, complete deployment of the same system runs locally under Docker for development, comprising the PostgreSQL database, the authentication service, PostgREST, the storage service and the Supabase Studio console. Because the schema is defined entirely by version-controlled migrations, this local stack is rebuilt from empty on demand, which is what makes the database evidence presented in Section 4.5.2 reproducible rather than a description of a single mutable instance.

3.5 Module Design

The system is logically partitioned into discrete, highly cohesive modules to facilitate maintainability.

- **Student Module:** Encapsulates all logic related to course discovery, enrollment, lesson consumption, and quiz execution.
- **Educator Module:** Manages the creation pipeline. Handles the rich-text editor state, quiz generation, and the uploading of media assets to storage.
- **Admin Module:** Provides a high-level oversight interface for user management, role reassignment, and global accessibility preset configurations.
- **Accessibility Module:** A globally available React context provider that loads the learner's stored preferences, resolves conflicts between them, and writes the result to the document root as `data-*` attributes and CSS custom properties, from which the stylesheet derives typography, palette, reading measure and motion behaviour.
- **Accessibility Compliance Module:** A pure, synchronous auditing engine shared by two surfaces: the lesson editor, which re-runs it against unsaved form state on every edit, and the course-level report, which runs the same rules against rows read from the database. Because it is deterministic and side-effect free, both surfaces necessarily agree.

- **Interactive Activity Module:** Governs five activity types together with in-video questions and embedded H5P content. Multiple-choice assessment is handled separately, and is graded inside the database rather than in this module.
- **Analytics Module:** Aggregates lesson progress, quiz attempts and accessibility adaptation events into visualisations, giving educators cohort-level insight and administrators platform-level insight, including which accessibility features are actually being used.

3.6 Summary

This chapter analyzed the engineering specifications required to build the ACESS platform. It began with a detailed problem analysis of existing educational systems, followed by a comprehensive requirements analysis covering data, functional, non-functional, and other requirements. The system architecture was presented through Context, Architecture, and Deployment diagrams, and the modular decomposition of the system was described. The subsequent chapter will translate these requirements into concrete database schemas and interface designs.

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

This chapter translates the requirements established in the previous chapter into concrete engineering design. It is organised in the order the writing guide prescribes. High-Level Design comes first, covering behavioural and interaction diagrams, user interface design, and the conceptual and logical database model. The accessibility design that is this project's substantive contribution follows, then the data dictionary, and finally Detailed Design, which covers software components and the physical database implementation.

Two things about this chapter differ from a conventional design chapter, and both are deliberate. First, the figures are evidence rather than illustration. The interface figures are screenshots of the running system taken against a database that can be rebuilt from the repository, and the Entity Relationship Diagrams are generated from that database's own catalogue by a script in the repository. A design chapter that documents an intended system rather than the built one is of no use in verifying either. The previous revision of this report had drifted in exactly that way, documenting six database tables that no longer exist and one accessibility settings table that was never read by any code path.

Second, Section 4.3 states what the accessibility implementation does *and* what it does not do. A design chapter for an accessibility project that lists only successes would be the same failure as an inert setting that looks like it works.

4.2 High-Level Design

4.2.1 Behavioral Design (Activity Diagrams)

Activity diagrams model the workflow of the system, illustrating the procedural logic executed by the different actors.

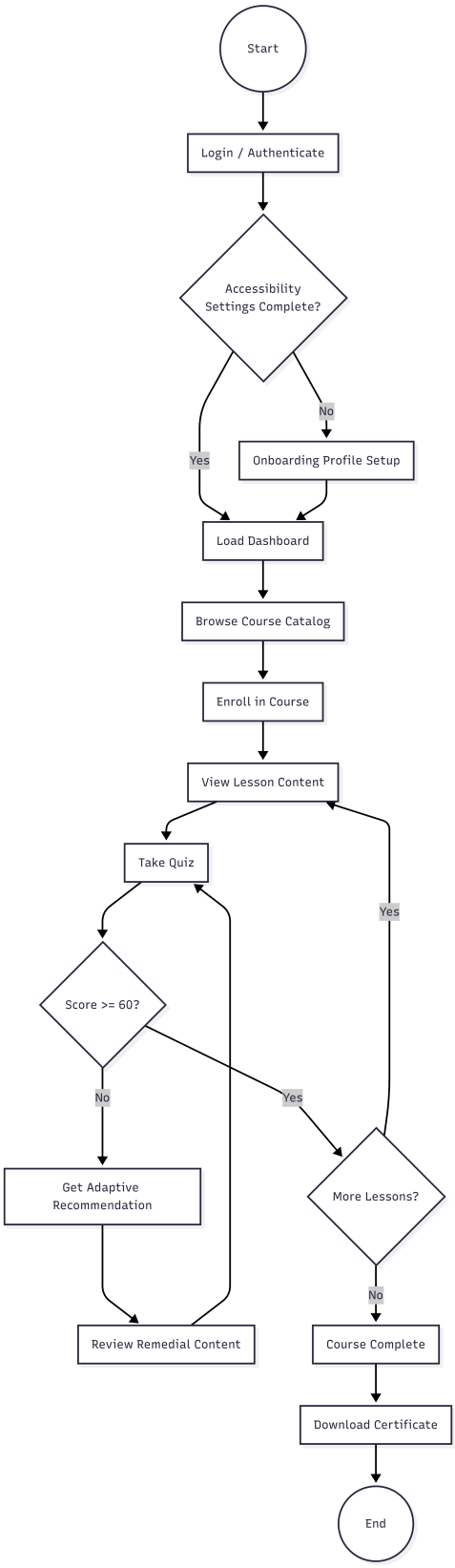


Figure 4.1: Student Activity Diagram

Student Activity Diagram As shown in Figure 4.1, the learner workflow has one structural feature that distinguishes it from a conventional course flow. On authentication the route interceptor resolves the role and admits the learner shell. The accessibility provider then loads the learner's stored accessibility preferences and writes the resolved settings onto the document root before the first lesson is rendered, so a learner never sees an unstyled pass followed by a correction. A learner with no stored preferences is offered onboarding, but is not blocked by it. An accessibility profile is an aid, not a gate.

The core loop is browsing the catalogue, enrolling in a course, and working through its lessons in order, with progress updated as each lesson is opened and completed. The branch of interest occurs after assessment. The attempt is graded by the database, not the client. If the resulting score falls below the quiz's pass threshold, the recommendation engine records a revision-tier recommendation naming the lesson and the score that triggered it. That recommendation is surfaced prominently, but it does not lock the learner out of the next lesson. Forcing remediation would convert a support into a punishment, which for a learner already struggling is the wrong intervention.

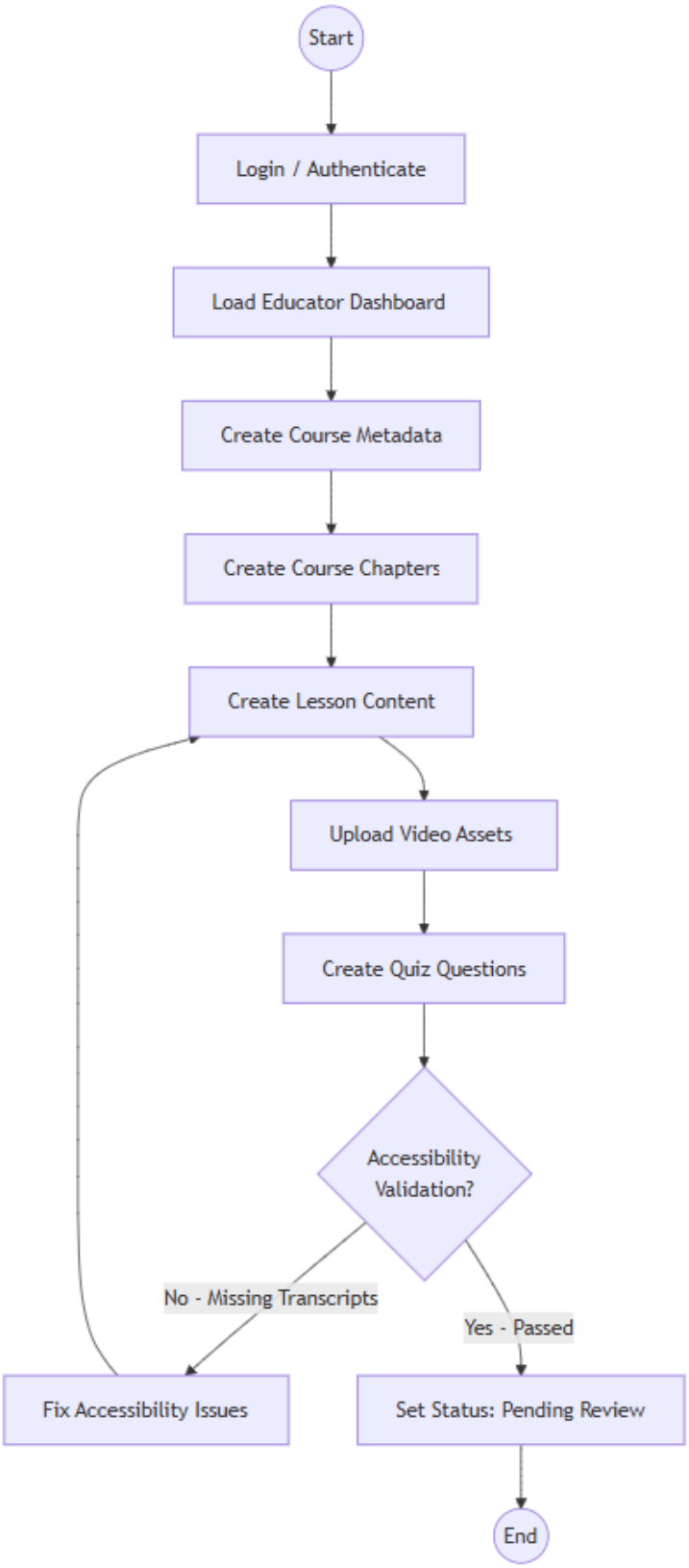


Figure 4.2: Educator Activity Diagram

Educator Activity Diagram Figure 4.2 maps the educator's authoring workflow. The process begins with course metadata and, optionally, chapter structure, and includes one decision with consequences throughout the rest of the flow: the course's *primary accessibility focus*. That choice selects which rule set the compliance auditor will apply to every lesson in the course, so an educator writing for dyslexic readers is checked on sentence length and readability while one writing for autistic learners is checked on figurative language and stated objectives.

The educator then enters a content loop: drafting lesson text, embedding media, building activities and writing quiz questions. The auditor re-evaluates the draft continuously against unsaved editor state, so the accessibility score in the editor header moves as the lesson is written rather than appearing at the end. When the educator submits the course, its status moves to pending review for administrator moderation. The accessibility check is *not* a gate in the publication path: a lesson with unmet standards can still be saved and published, and the panel says so explicitly. The reasoning is given in Section 4.3.6.

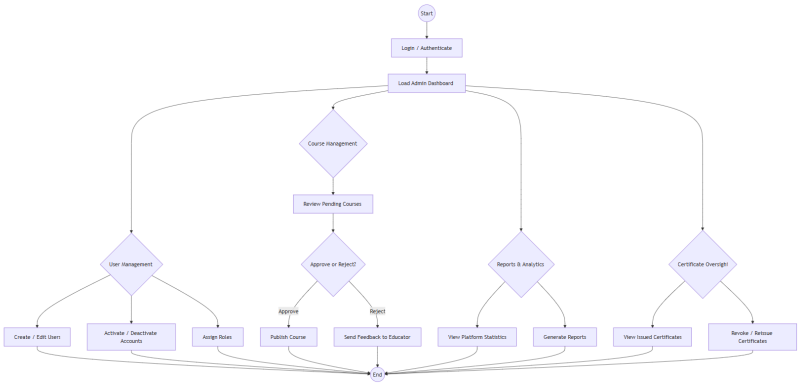


Figure 4.3: Administrator Activity Diagram

Administrator Activity Diagram The administrator workflow (Figure 4.3) is primarily reactive. The administrator monitors two queues: courses awaiting review and applications from users wishing to become educators. When a course enters pending review, the administrator evaluates it, including against the course-level accessibility report. Approval sets its status to published, which fires the trigger that notifies the educator. Approving an educator application promotes the applicant, updating both the

users table and the account's authentication metadata. Updating only one of the two previously left newly promoted educators unable to reach their own dashboard.

The remaining administrator activities are oversight rather than moderation: reviewing platform analytics, generating reports, managing user status, and handling enquiries submitted through the contact form.

4.2.2 Interaction Design (Sequence Diagrams)

Sequence diagrams detail how objects interact over time to execute specific use cases, mapping the calls between the client, the Next.js request interceptor, and the Supabase backend. One convention runs through all four: an operation drawn as reaching the database directly is one the client issues under its own session and Row-Level Security, while an operation drawn as passing through a server route is one that requires privileges the learner does not have. Which of the two a step uses is a security decision, not an implementation detail, so the diagrams distinguish them.

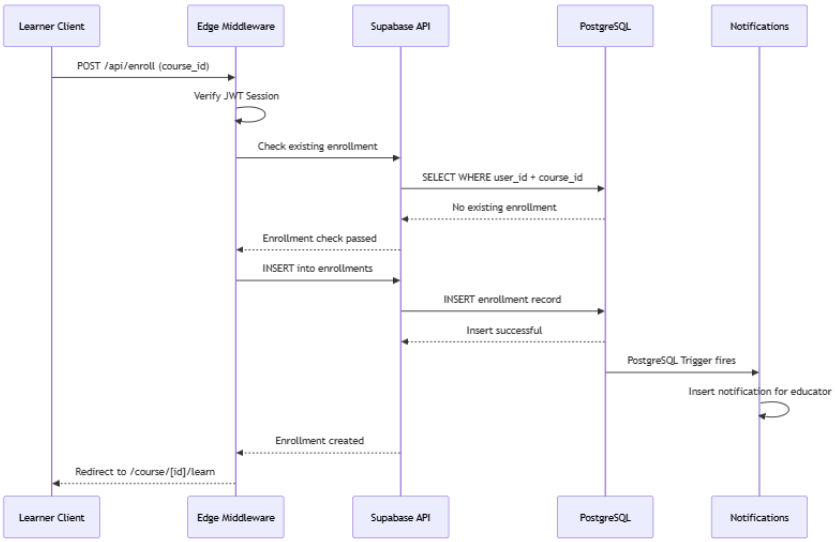


Figure 4.4: Course Enrollment Sequence Diagram

Sequence: Course Enrollment Figure 4.4 illustrates the enrolment transaction. It is a good illustration of how little application code sits between the learner and the database in this architecture, and how much of the correctness is carried by the schema instead.

1. The learner activates Enrol on a course page. The client calls Supabase directly under the anonymous key, since there is no bespoke enrolment endpoint, so the request carries the learner's own session and is evaluated under Row-Level Security.

2. The database's insert policy on the enrolments table admits the new record only if the learner id on it matches the caller's own identity, so a learner cannot enrol anybody else. No application check is required for this, and none can be bypassed.

3. A uniqueness constraint on the learner and course together prevents a duplicate enrolment. A repeat enrolment reactivates the existing record instead, preserving the learner's earlier progress.

4. A database trigger fires and creates a notification addressed to the course's educator.

5. The client navigates to the course page, which now renders the lesson roadmap rather than the enrolment call to action.

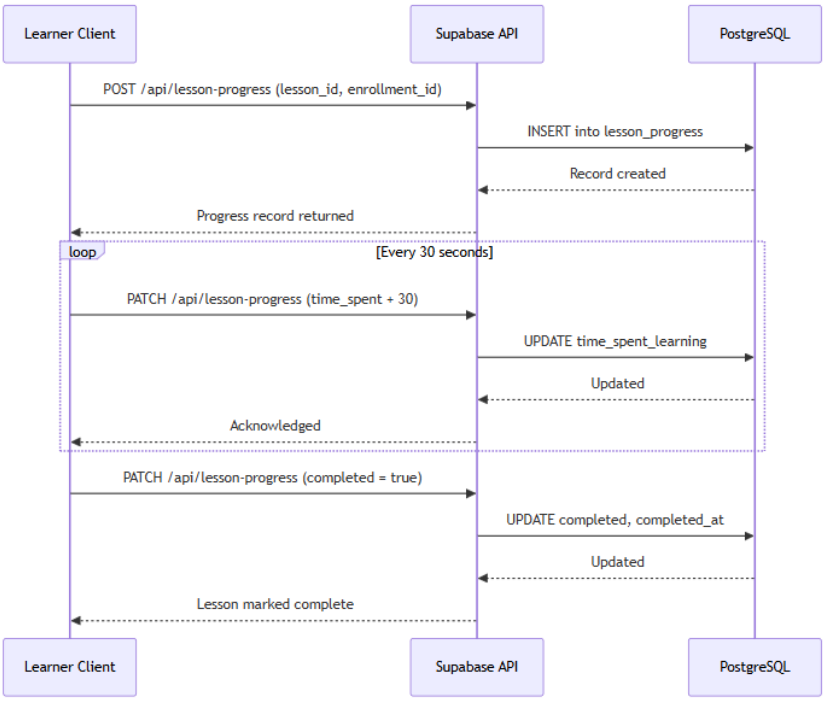


Figure 4.5: Lesson Learning Sequence Diagram

Sequence: Lesson Learning & Telemetry Figure 4.5 maps lesson telemetry. Opening a lesson creates or updates the learner's progress record for it, marking the lesson viewed,

incrementing the view count and stamping the time it was first opened. While the lesson remains open, elapsed time is accumulated periodically into that record rather than being written only when the page unloads, so that a browser closed abruptly still leaves an accurate record up to the last interval rather than losing the whole session.

Completion is recorded separately from viewing, and the two are constrained to remain consistent. A database check rejects any progress record marked complete but not viewed. This distinction matters to the recommendation engine, which keys what a learner should do next on completion rather than on viewing. Keying it on viewing instead would skip every lesson the learner had merely glanced at.

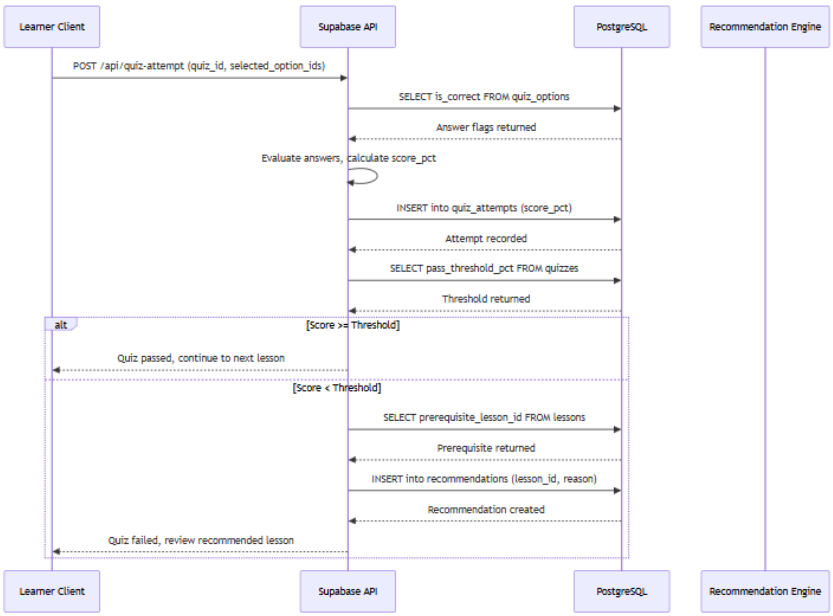


Figure 4.6: Quiz Attempt Sequence Diagram

Sequence: Quiz Attempt & Adaptive Recommendation Figure 4.6 details the most security-sensitive interaction in the system, and the one where the design most deliberately refuses to trust the client.

1. While answering, the client reads options from a restricted database view rather than the underlying options table. That view withholds which option is correct for any quiz the learner has not yet submitted, so the answer key is not present in the client at all. It is not merely hidden. It is absent.

2. On submission the client calls a database grading function with the quiz identifier and the selected options. It does not compute a score and does not write an attempt record itself. A guard trigger would reject the write if it tried.

3. The function verifies authentication, verifies an active enrolment in the owning course, and refuses the attempt if the quiz's attempt limit is exhausted.

4. It grades each answer against the stored correctness flag, writes the attempt and its answers to the database, and computes the pass decision against the quiz's pass threshold. It returns the score, the pass decision and the threshold used, so the interface can explain the result rather than merely report it.

5. A database trigger raises a notification for the educator.

6. The answer key becomes readable to that learner for that quiz only now, which is what allows the review screen to show which answers were wrong.

7. Separately, the recommendation engine records a revision-tier recommendation for the lesson, carrying the score that triggered it as its stated reason.

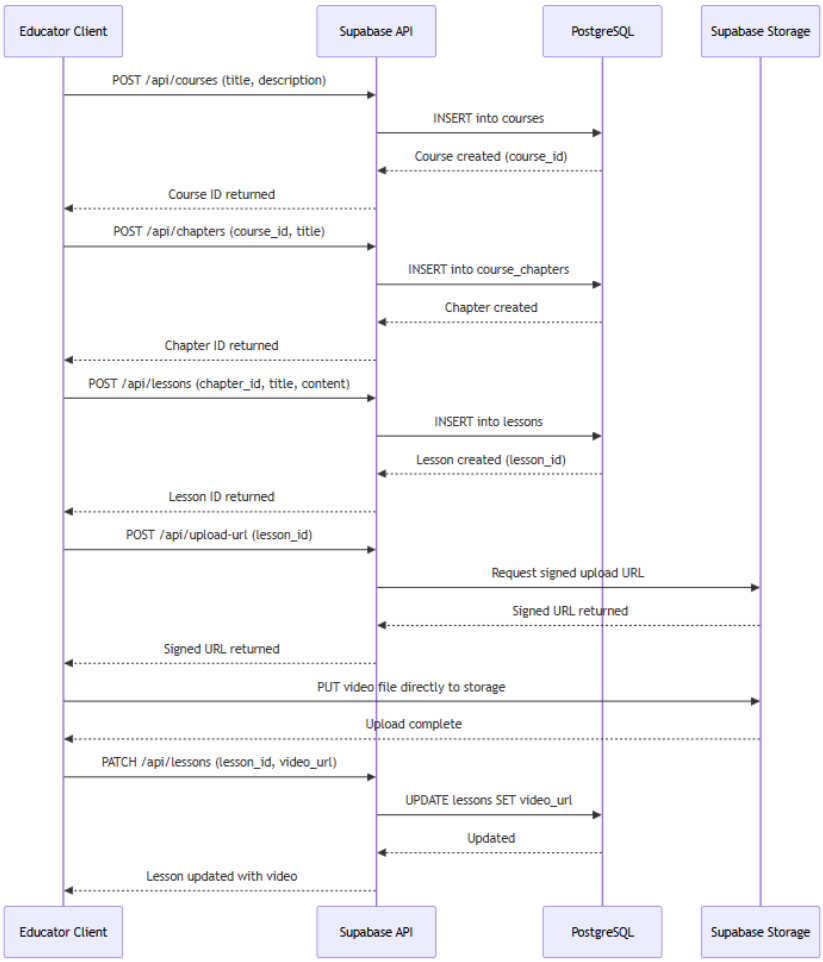


Figure 4.7: Course Creation Sequence Diagram

Sequence: Course Creation & Media Upload Figure 4.7 shows the authoring pipeline. Lesson text is composed in the rich-text editor, sanitised, and written to the lesson's content field, with a snapshot appended to the version history so that an edit is recoverable. Images and documents are uploaded from the client straight into cloud storage, bypassing the application server so that a large file never has to be buffered by it, and the returned public URL is recorded against the course's media assets.

Instructional video is handled differently, and the distinction matters for the storage and cost model. Video is *embedded from an external host* by URL rather than uploaded, so the lesson holds only a reference to a resource the platform does not store or serve. Throughout this pipeline the auditor re-runs on every change, so the accessibility consequence of an edit, such as adding a video without a transcript, is visible at the moment it is made rather than at publication.

4.2.3 User Interface Design

The interface architecture of ACESS is organised into three role-specific shells, each with its own navigation structure, input mechanisms and output presentation. Every figure in this section is a screenshot of the running system taken against the reproducible demonstration database described in Section 4.5.2, not a mockup.

4.2.3.1 Navigation Design

Navigation follows a role-based shell pattern. The route prefix determines the shell, and a server-side check refuses a shell the signed-in role is not entitled to before the page renders.

- **Public navigation.** Unauthenticated visitors reach the landing page, the course catalogue preview, login, signup, password recovery, the contact form, the educator-application form, and the published accessibility statement. Certificate verification (`/verify/[code]`) is also public, so that a third party can check a certificate without an account.
- **Learner shell.** A persistent sidebar carries Dashboard, My Courses, Progress, Achievements & Certificates, and Accessibility. A top bar holds the notification bell with an unread count and the profile menu. Two things about this sidebar are design decisions rather than defaults. First, every item is a real hyperlink rather than a click handler, so that a screen reader announces navigation as navigation and the browser's own affordances (middle-click, open in a new tab, copy link) work. Second, the sidebar's contents are *preset-dependent*: the Dyslexia preset widens it and enlarges its type, and the distraction-free presets remove it entirely inside a lesson.
- **Educator shell.** A sidebar carries Dashboard, Courses (All Courses, My Courses, Add Course), Educator Ranking, Analytics, Students Progress and Certificates. Inside a course, a workspace tab bar provides Overview, Lessons, Assets, Students, Certificates, Achievements and Settings.

- **Administrator shell.** A sidebar carries Dashboard, User Management, Course Management, Educators (educator applications), Feedback (contact messages), Certificates, Analytics and Reports.

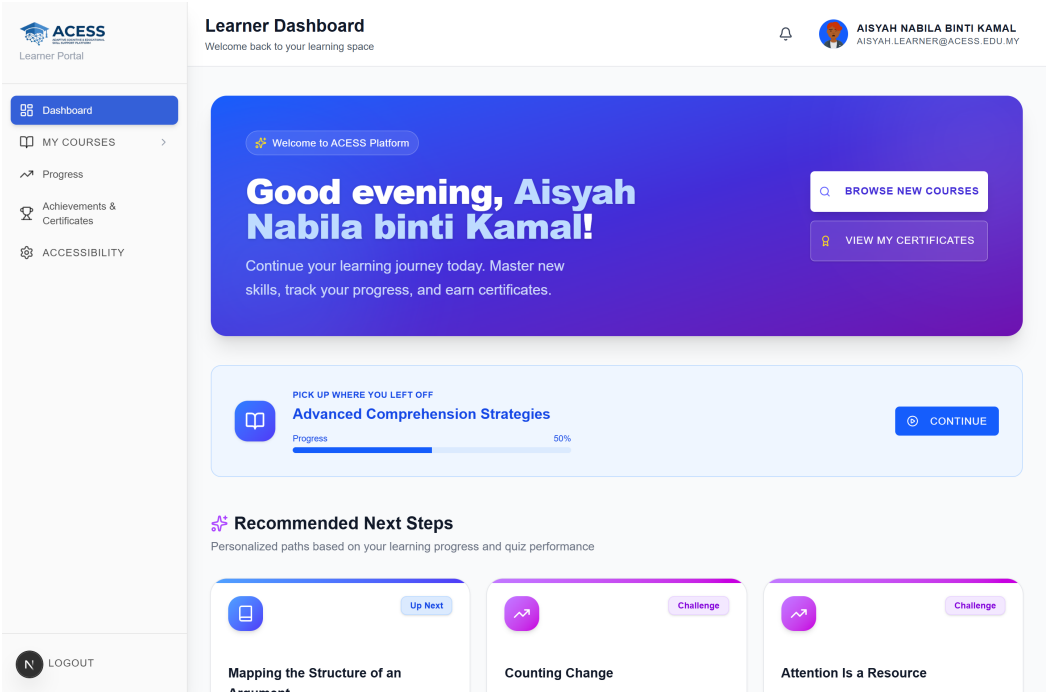


Figure 4.8: Learner Dashboard (Default State)

Figure 4.8 shows the learner dashboard as a learner with no accessibility preset applied encounters it, with its persistent sidebar, continue-learning card and recommendation region visible together. The corresponding educator workspace is shown in Figure 4.9, where the workspace tab bar and the ordered lesson curriculum are visible together with the per-lesson authoring controls.

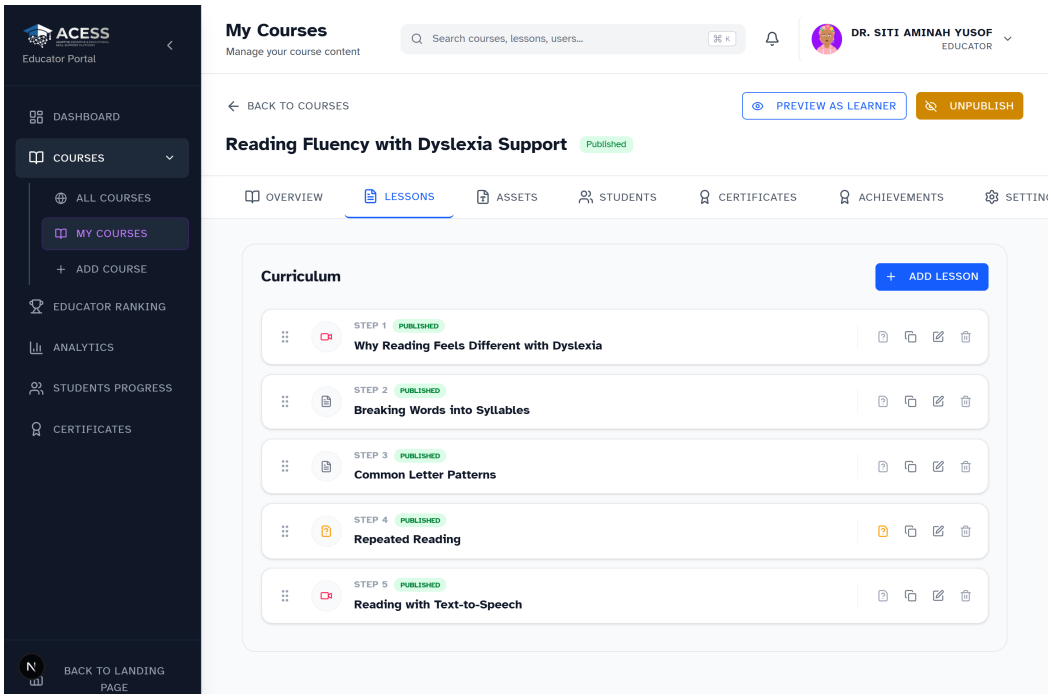


Figure 4.9: Educator Course Workspace (Lessons Tab)

4.2.3.2 Input Design

Input surfaces are grouped below by the role that uses them, matching the structure of Navigation Design above.

- **Public input.** An unauthenticated visitor supplies an email and password at sign-up, and a category, subject and message on the contact form. A visitor's requested role is never accepted from the sign-up request itself: self-registration is always pinned to the learner role by a database trigger.
- **Learner input.** A learner selects a course to enrol in, adjusts accessibility settings such as font size and line spacing from the accessibility panel (Figure 4.10), answers quiz questions (Figure 4.11), and uploads a profile avatar.
- **Educator input.** An educator supplies course metadata such as title, slug, difficulty and status, writes lesson content in the rich-text editor, sets the sequence order and layout of each lesson, and builds quiz questions and their options.
- **Administrator input.** An administrator sets a target user's role, decides whether a

submitted course is approved or rejected, and selects the date range for a platform report.

Table 4.1 then states the exact validation rule applied to each significant field above and the layer at which it is enforced. The distinction matters: a rule enforced only in the browser is a convenience, whereas a rule enforced by a database constraint or an RLS policy cannot be bypassed by a crafted request.

Table 4.1: Input Validation Rules

Screen	Field	Validation rule	Enforced at
Sign up	Email	Well-formed address, must be unique	Client, then Supabase Auth
Sign up	Password	Minimum length and complexity	Client, then Supabase Auth
Sign up	Role	Always `learner`, not accepted from the request	Database trigger
Course form	Title	Required, non-blank	Client + `NOT NULL`
Course form	Slug	Unique across courses	`UNIQUE` constraint
Course form	Difficulty	One of beginner / intermediate / advanced	`CHECK` constraint
Course form	Status	One of draft / pending_review / published / archived	`CHECK` constraint
Lesson editor	Rich text	Sanitised before render, disallowed markup stripped	Client sanitiser
Lesson editor	Sequence order	Integer greater than zero	`CHECK` constraint
Lesson editor	Lesson layout	One of standard / focus / two_column / wide / slideshow	`CHECK` constraint
Quiz builder	Pass threshold	Integer between 0 and 100	`CHECK` constraint
Quiz builder	Options	At least two options, exactly one correct	Client
Quiz attempt	Selected options	Graded server-side, answer key never sent early	Database function + view

continued on the next page

Table 4.1 continued from the previous page

Screen	**Field**	**Validation rule**	**Enforced at**
Accessibility panel	Font size	Slider bounded 12–24 px	Client
Accessibility panel	Line spacing	Slider bounded 1.0–3.0	Client
Enrolment	Course reference	Learner may only enrol themselves	RLS policy
Enrolment	Status	One of active / completed / dropped, completion not client-settable	`CHECK` + guard trigger
Contact form	Category	One of five permitted categories	`CHECK` constraint
Avatar upload	File path	Must sit inside the uploader's own folder	Storage policy

The single most important input surface in the system is the accessibility panel, shown in Figure 4.10. It presents four preset buttons above five tabbed groups (Reading, Focus, Sensory, Supports and Profile), and every control writes through to the learner's stored preferences and takes effect without a page reload.

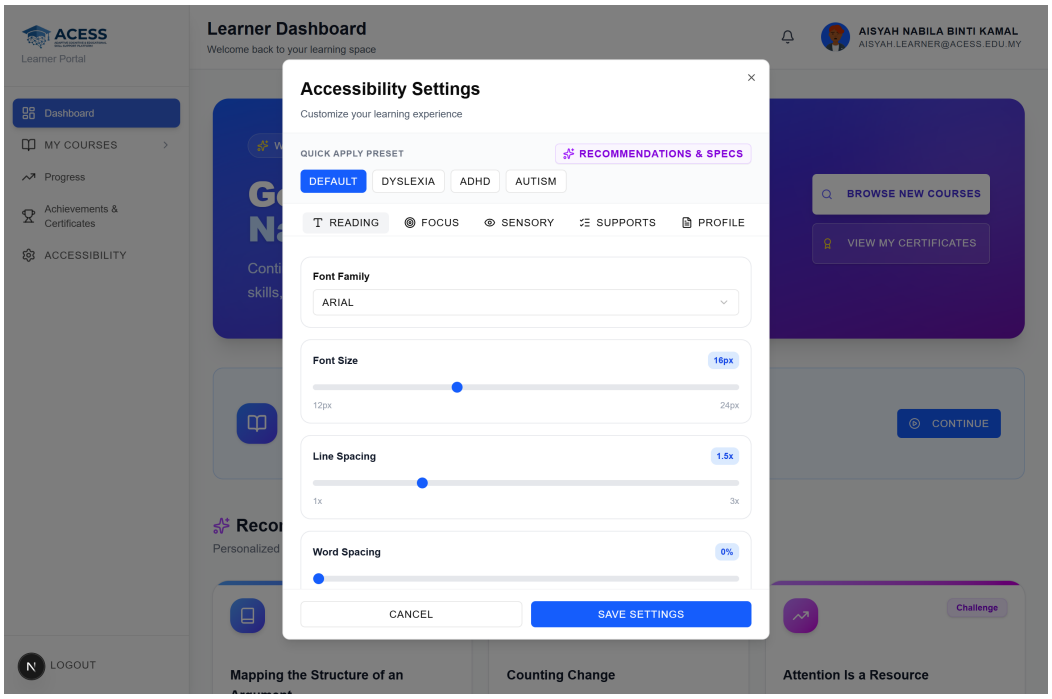


Figure 4.10: Accessibility Settings Panel (Reading Group)

The second significant input surface is the quiz, shown in Figure 4.11. Its design carries three deliberate accessibility decisions: questions are presented one per screen regardless of the lesson's own layout setting, an educator-configured time limit is displayed but never triggers an automatic submission, and the expectations for the attempt are stated before it begins rather than discovered during it.

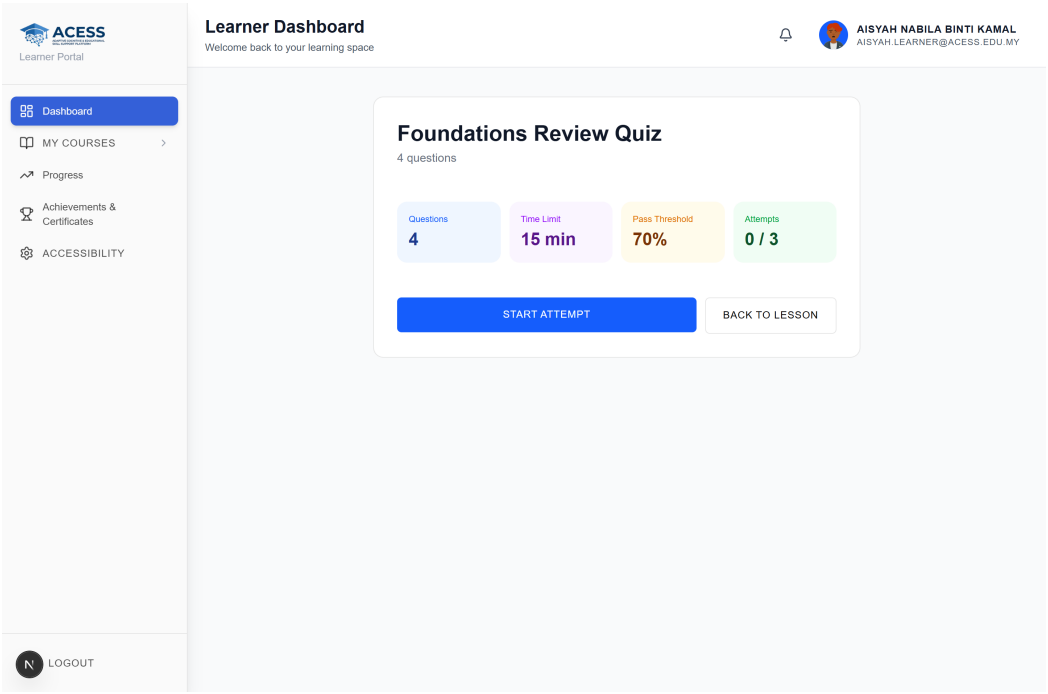


Figure 4.11: Quiz Interface

4.2.3.3 Output Design

Outputs fall into three classes: on-demand screen output, on-demand document output, and event-driven output. They are summarised below by the role each output serves.

- **Public output.** The certificate verification page reports a certificate's validity, issue date and revocation status to anyone holding its reference code, without requiring an account.
- **Learner output.** A learner sees their own progress page (Figure 4.12), receives event-driven notifications, and downloads a completion certificate once a course is finished.
- **Educator output.** An educator sees course analytics, the continuous accessibility compliance report described in Section 4.3.6, and notifications on enrolment, quiz attempts and course publication.
- **Administrator output.** An administrator sees platform-wide analytics

(Figure 4.13), including the accessibility-adaptation view unique to this platform (Figure 4.14), and generates a report as a PDF over a chosen date range.

Table 4.2 then classifies every output above by its trigger basis, its audience and its content in full.

Table 4.2: System Outputs

Output	Basis	Audience	Content
Learner progress page	Ad hoc	Learner	Course completion, lessons completed, study time, quiz history, visit history
Course analytics	Ad hoc	Educator	Enrolment and completion trends, per-lesson completion, quiz score distribution, at-risk flags
Platform analytics	Ad hoc	Admin	Registrations, enrolments, learning time, accessibility adaptation usage and preset adoption
Accessibility compliance report	Continuous	Educator	Per-lesson and per-course standards score, unmet rules, source of each rule
Administrator report (PDF)	Ad hoc	Admin	Platform, course, learner or content-coverage report over a chosen date range
Completion certificate (PDF)	Event	Learner	Learner name, course, completion date, reference code, verification QR code
Certificate verification page	Ad hoc	Public	Validity, issue date and revocation status for a reference code
Notifications	Event	All roles	Enrolment, lesson progress, quiz attempt, lesson added, course published
Achievement award	Event	Learner	Badge unlocked on meeting a course achievement's requirement

Figure 4.12 shows the learner-facing analytics output, with completion, study time and quiz telemetry for the signed-in learner, and Figure 4.13 the administrator equivalent, with platform-level summary tiles and the registration trend series. Both are rendered with the same charting library, so that a figure carries the same visual grammar wherever it appears.

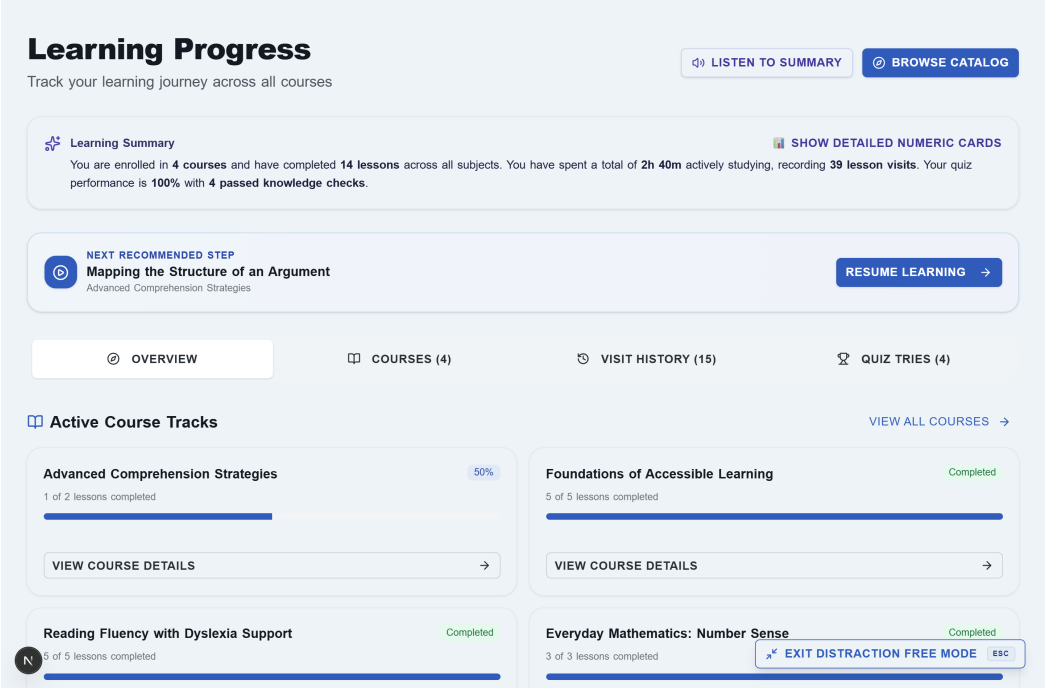


Figure 4.12: Learner Progress Page

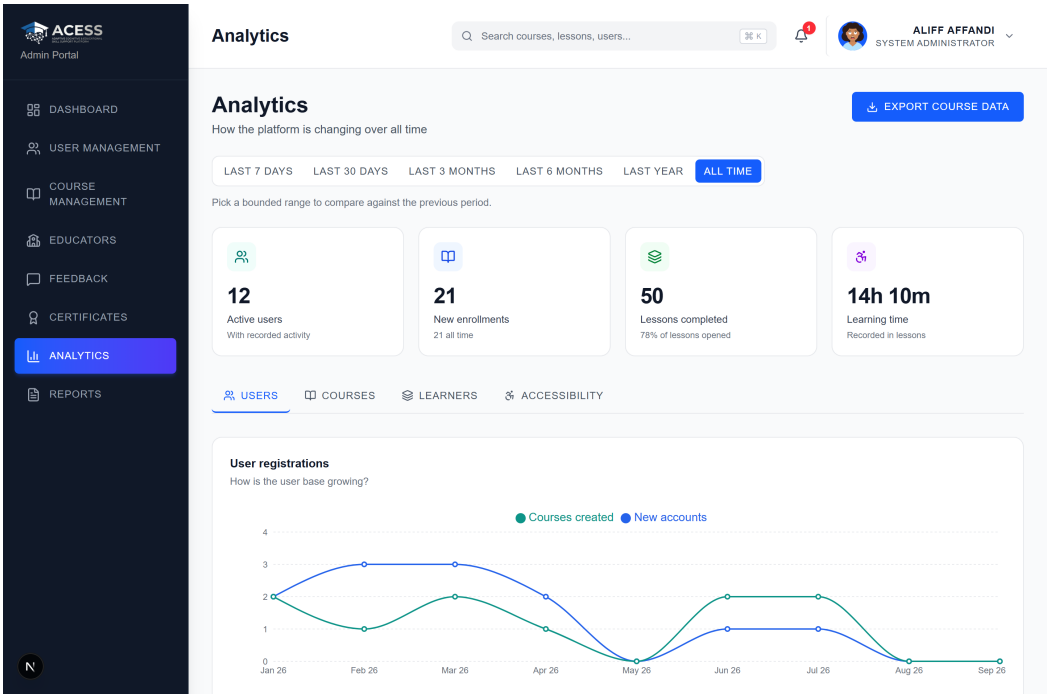


Figure 4.13: Administrator Analytics Dashboard

One output deserves separate mention because it is unusual in a learning platform. The administrator analytics view carries an Accessibility tab, shown in Figure 4.14, which reports which accessibility adaptations are actually being exercised and which presets learners actually choose, drawn from the platform's own adaptation-telemetry records.

Accessibility features are frequently built and then never measured. This view exists so that the question “is any of this being used?” has an answer.

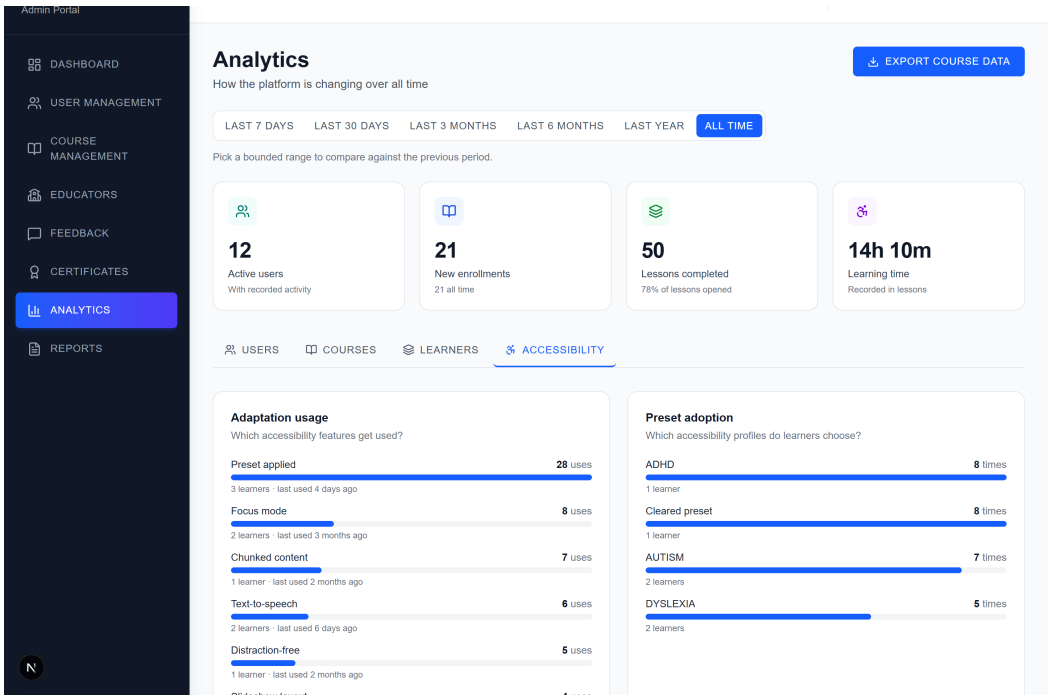


Figure 4.14: Accessibility Analytics

4.2.4 Conceptual and Logical Database Design

The ACESS data model is a normalised relational schema implemented on PostgreSQL 17. In its current form it comprises **36 base tables, one view, 370 columns, 57 foreign keys, 33 check constraints, 20 unique constraints, 115 indexes, 20 functions, 15 triggers and 88 Row-Level Security policies**. These figures are not estimates: they were obtained by querying the system catalogues of a database rebuilt from the project’s migrations, and the same query can be re-run to confirm them.

The Entity Relationship Diagrams in this section were likewise *generated* from that live schema by a script in the repository (`scripts/erd-gen.mjs`) rather than drawn by hand, so that a diagram in this report cannot silently fall out of step with the database it claims to describe. Because thirty-six entities cannot be rendered legibly on a single portrait page, the model is presented as three domain diagrams, with the complete schema shown afterwards in landscape.

Business rules governing the relationships The following rules were derived from the platform's operational requirements and are what the foreign keys, unique constraints and delete rules in the schema encode.

1. **One account, one identity record.** Every account has exactly one extended profile, and that profile is deleted automatically when the account is, because it holds nothing of value on its own.

2. **A course must have an owner, and that owner cannot vanish.** Deleting the educator who owns a course is blocked outright. This is the only such restriction in the schema, and it is deliberate: removing an educator's account must not silently destroy the courses other people are enrolled in.

3. **A learner enrols in a course at most once.** A uniqueness rule over the learner and course pair enforces this. Re-enrolment reactivates the existing enrolment rather than creating a second one, which is what keeps progress history continuous.

4. **All learning activity hangs off the enrolment, not the learner.** Progress, quiz attempts, checkpoints, recommendations and certificates all reference the enrolment rather than the learner directly. This makes the enrolment the single unit of participation, so that withdrawing a learner from a course removes exactly that course's record and nothing else.

5. **A lesson belongs to exactly one course, and optionally to one chapter.** The course link is mandatory. The chapter link is optional, because chapter organisation is a per-course feature an educator may switch on or leave off.

6. **A lesson may declare one prerequisite lesson.** If that prerequisite is later deleted, the requirement is simply cleared rather than the dependent lesson breaking.

7. **Assessment is strictly hierarchical.** A lesson has at most one quiz. A quiz has many questions. A question has many options, and exactly one option per question is flagged correct. An attempt records one answer per question, and an option that a submitted attempt still refers to cannot be deleted.

8. **One certificate per enrolment.** A revoked certificate keeps its row and its reference code rather than being deleted, with its status changed to revoked. A

check constraint enforces that a revoked record carries a revocation timestamp and an issued one does not, which is why public verification can report “revoked” rather than “not found”.

9. **An achievement is earned once per learner per course.** The record links the learner, the achievement definition and the course, and rows are inserted only by the platform’s own derivation logic, never directly by a client.

10. **Comments form a thread.** A comment may reference another comment as its parent, giving one level of reply nesting.

11. **Accessibility state is per learner, not per enrolment.** A learner’s presentation preferences follow them across every course, so they are stored on the learner’s profile rather than on any individual enrolment.

Domain 1: Identity, Access and Platform Figure 4.15 shows the identity domain. It holds the account record, the extended profile that carries accessibility and notification preferences as JSON documents, the educator-application workflow, referral codes, platform notifications, the reusable accessibility content templates, and the adaptation telemetry that records which accessibility features are actually exercised. Every entity in this domain reaches `users` directly, which is why its policies are the simplest in the schema.

Figure 4.15: Identity, Access and Platform ERD

Domain 2: Curriculum and Content Figure 4.16 shows the curriculum domain. It holds the course hierarchy (course, chapter, lesson), the authored artefacts attached to a

lesson (interactive activities, in-video questions, H5P embeds, checkpoints, cached AI summaries and discussion threads), and the supporting content infrastructure of templates, version snapshots and uploaded media. Everything in this domain is authored by an educator, and nothing in it is generated by a learner.

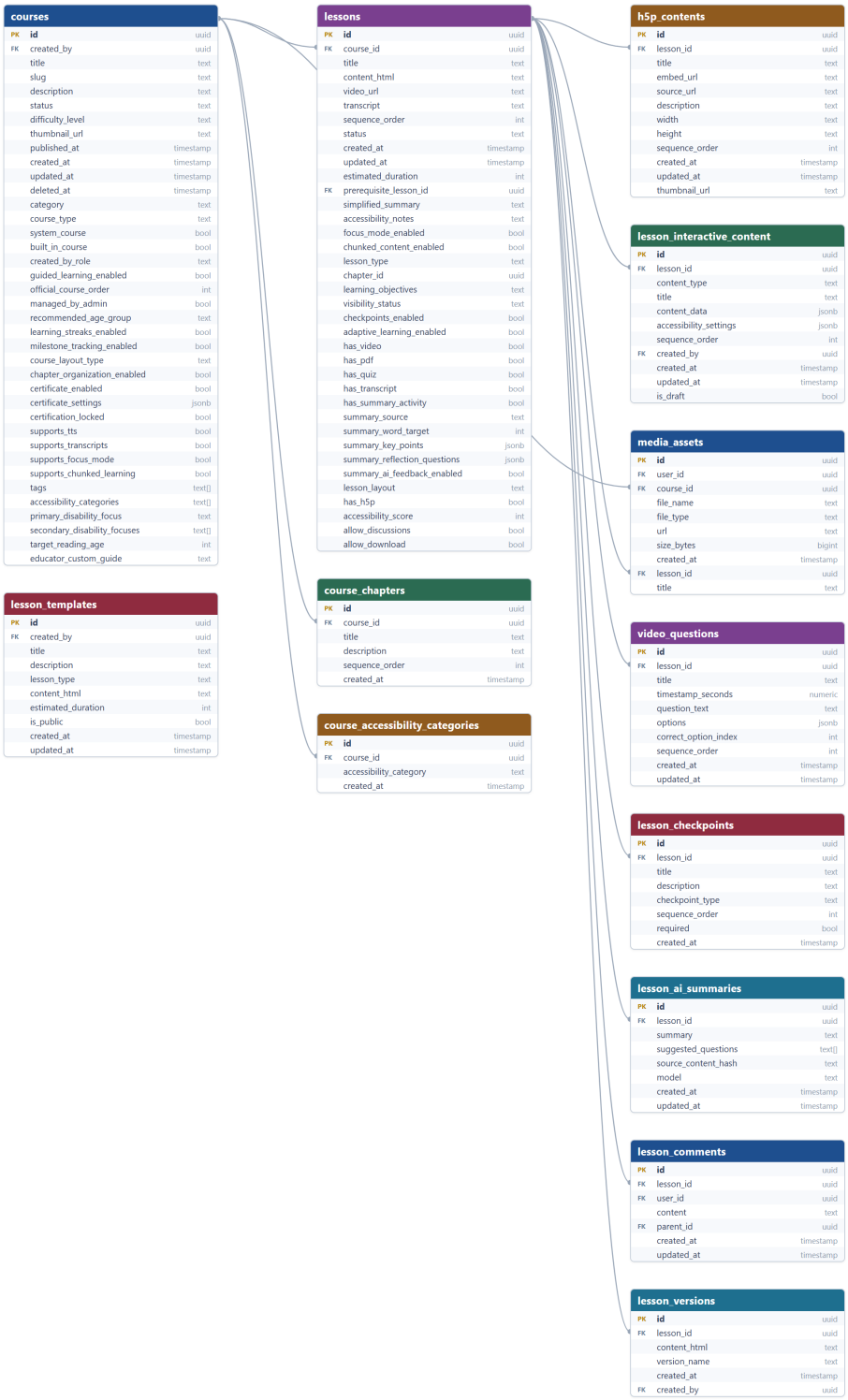


Figure 4.16: Curriculum and Content ERD

Domain 3: Assessment, Learning Record and Recognition Figure 4.17 shows the assessment and learning-record domain, which holds everything generated by a learner

rather than authored by an educator: the enrolment itself, per-lesson progress, quiz attempts and answers, checkpoint responses, H5P responses, generated recommendations, favourites, and the recognition artefacts of achievements, milestones and certificates. The convergence of so many entities on `enrollments` is what business rule 4 above describes.

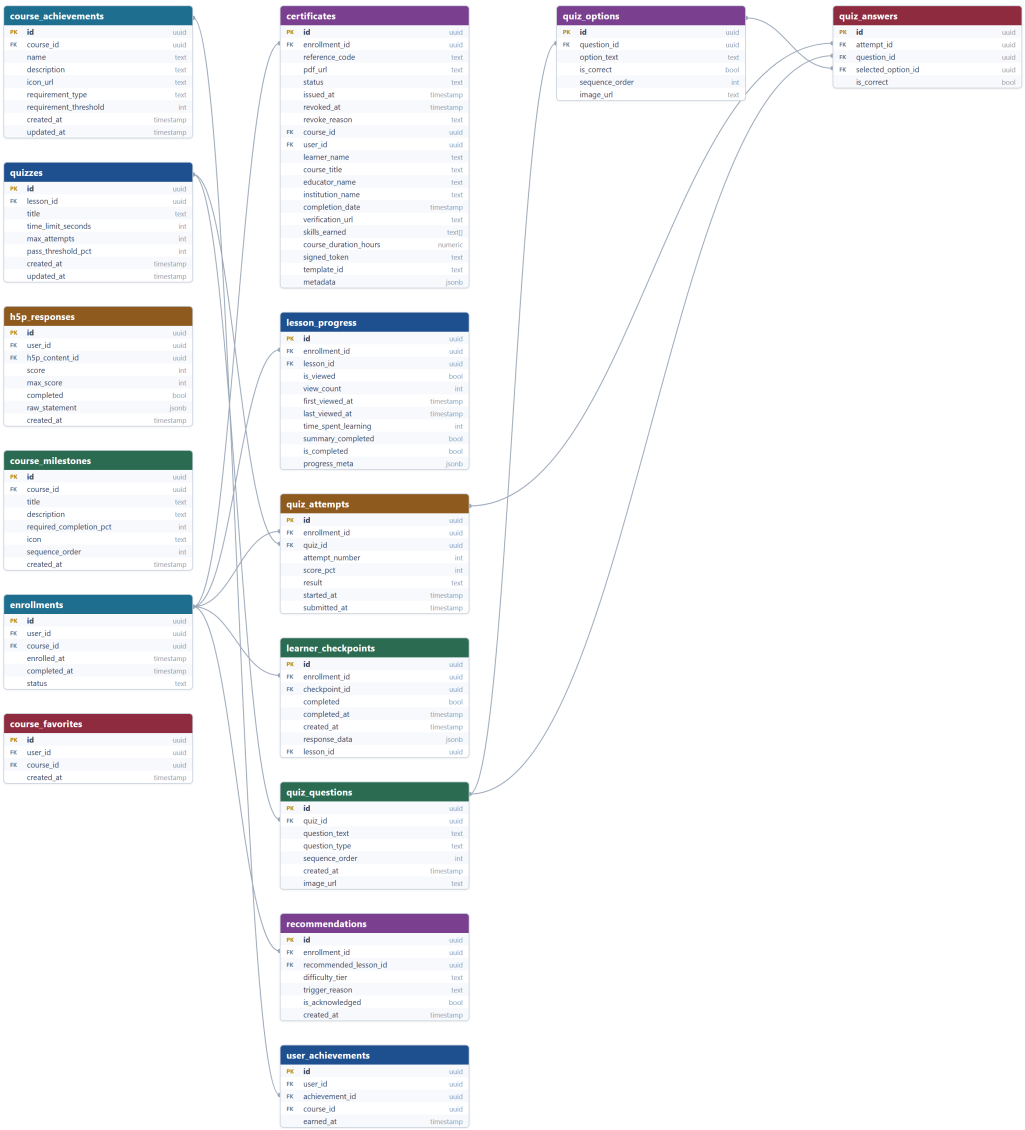


Figure 4.17: Assessment, Learning Record and Recognition ERD

Figure 4.18: Complete ACESS Entity Relationship Diagram

Figure 4.18 presents the full model, generated from all 36 tables and 57 foreign keys in the live schema. The `users` table acts as the identity hub and `enrollments` as the participation hub: nearly every learner-generated row reaches `users` through an enrolment rather than directly, which is what makes the row-level security policies expressible as a single join in most cases.

4.3 Accessibility Design

4.3.1 Design approach and standards baseline

Accessibility in ACESS is not a settings panel bolted onto a finished product. It is the design constraint the rest of the system is arranged around. Three commitments follow from that, and they explain most of the design decisions in this section.

The first is that **a control must do what its label says**. During development a number of settings were found to be inert: a word-spacing slider that a stylesheet rule silently overrode, a line-spacing control that changed the settings preview but not the lesson, and an "Easy Read" mode that could not be reached from anywhere in the interface. Each of these was a control that appeared to work and did not. They were repaired, and the platform now maintains an explicit catalogue of every learner-facing setting, checked automatically, under an invariant stated in the project's own design documentation: if a setting is not in the catalogue, it does not exist. Two settings that still do not fully honour their labels are recorded as known gaps in Section 4.3.7 rather than presented as working.

The second is that **the standards baseline has to be wider than WCAG**. WCAG 2.2 is necessary but was not written primarily for cognitive difference: it can tell an author that contrast is insufficient, but not that a paragraph is too long to re-enter after looking away. ACESS therefore works from three sources together: WCAG 2.2 (W3C Web Accessibility Initiative, 2023) for testable technical criteria, the W3C's *Making Content Usable for People with Cognitive and Learning Disabilities* (W3C Web Accessibility Initiative, 2021) for cognitive design objectives, and the British Dyslexia

Association Style Guide (British Dyslexia Association, 2023) for typographic and prose thresholds.

The third is that **the claim must not exceed the evidence**. ACESS *targets* WCAG 2.2 Level AA plus four Level AAA criteria. It has not been audited, has not been tested with a screen reader or other assistive technology, and has not been tested with disabled learners. That statement is published on the platform itself at `/accessibility-statement`, where a prospective learner can read it before investing time in the product, and it is repeated in Section 4.3.7 of this report.

4.3.2 Mapping of standards to implementation

Table 4.3 maps each addressed criterion to the ACESS feature that addresses it, the learner profile it primarily serves, and the module in which it is implemented. The final column names the figure in this report that shows it, where one exists.

Table 4.3: Accessibility Criteria Mapping

Criterion	ACESS feature addressing it	Profile served	Implemented in	Evidence
1.1.1 Non-text Content (A)	Lesson auditor requires alternative text on every image, and offers an inline field to supply it	All	`accessibility-audit.ts`	Fig. 4.24
1.2.1 Video-only (A)	Auditor treats a lesson with video and no transcript as a required failure	All	`accessibility-audit.ts`, `lessons.transcript`	Fig. 4.24
1.2.2 Captions (A)	Captions shown on video by default when the learner enables the setting	Hearing, slower processing	`AccessibilityProvider`	Fig. 4.10
1.3.1 Info and Relationships (A)	Auditor rejects empty paragraphs used as visual spacing and requires real headings	All	`accessibility-audit.ts`	Fig. 4.24
1.4.4 Resize Text (AA)	Font size adjustable 12–24 px, applied as a CSS custom property with no reload	Dyslexia	`AccessibilityProvider`	Fig. 4.10

continued on the next page

Table 4.3 continued from the previous page

Criterion	ACESS feature addressing it	Profile served	Implemented in	Evidence
1.4.5 Images of Text (AA)	Auditor requires body content to be real text so that read-aloud can reach it	Dyslexia	`accessibility-audit.ts`	Fig. 4.24
1.4.8 Visual Presentation (AAA)	Reading measure bounded in character units (72 default, 62 Dyslexia, 66 ADHD) and retained when navigation chrome is hidden	Dyslexia, ADHD	`globals.css` (`--content-measure`)	Fig. 4.20
1.4.12 Text Spacing (AA)	Line spacing 1.0–3.0 and word spacing reaching 0.2 em, both learner-set, with the Dyslexia preset at the 0.16 em floor	Dyslexia	`AccessibilityProvider`	Fig. 4.20
2.2.1 Timing Adjustable (A)	An educator's quiz time limit is displayed but never triggers automatic submission	ADHD, Autism	`QuizPage.tsx`	Fig. 4.11
2.2.4 Interruptions (AAA)	Distraction-free mode removes navigation and notification chrome during a lesson	ADHD	`globals.css`, `LessonViewPage.tsx`	Fig. 4.21
2.2.6 Timeouts (AAA)	15-minute inactivity timeout announced by a warning 30 seconds before it acts	All	`SessionTimeout.tsx`	N/A
2.3.3 Animation (AAA)	One animation-level control (normal / low / none) reduces or removes motion site-wide	Autism, ADHD	`AccessibilityProvider`	Fig. 4.10
2.4.4 Link Purpose (A)	Auditor flags non-descriptive link text such as "click here"	All	`accessibility-audit.ts`	Fig. 4.24
2.4.6 Headings and Labels (AA)	Auditor requires headings once a lesson passes 200 words	All	`accessibility-audit.ts`	Fig. 4.24
3.1.5 Reading Level (AAA)	Auditor computes a Flesch–Kincaid grade level and compares it with the course's declared target reading age (Kincaid et al., 1975)	Dyslexia	`accessibility-audit.ts`	Fig. 4.24
3.2.3 Consistent Navigation (AA)	Identical shell and navigation order on every page of a role, and the auditor checks structural consistency between lessons of one course	Autism	`Sidebar.tsx`, `accessibility-audit.ts`	Fig. 4.8
3.2.5 Change on Request (AAA)	A preset shows exactly what it will change before it changes it, read-aloud never starts by itself, and no media autoplays	All	`PresetDetailsDialog.tsx`	Fig. 4.19

continued on the next page

Table 4.3 continued from the previous page

Criterion	**ACESS feature addressing it**	**Profile served**	**Implemented in**	**Evidence**
4.1.2 Name, Role, Value (A)	Navigation built from real links carrying current-page state, and icon-only controls given accessible names and states	All	`Sidebar.tsx` and lesson header	Fig. 4.8
COGA Obj. 3–5	Chunked layout, per-section time estimates, visual schedule and step-by-step guidance	ADHD, Autism	`VisualSchedule.tsx`, `StepByStepGuidance.tsx`	Fig. 4.22

Table 4.3 makes one structural point about the design worth drawing out. The criteria divide cleanly into two groups by *who* can satisfy them. Presentation criteria (1.4.4, 1.4.8, 1.4.12, 2.3.3, 2.2.6, 3.2.5) are satisfied by the platform, once, for everyone. Content criteria (1.1.1, 1.2.1, 1.3.1, 2.4.4, 2.4.6, 3.1.5) cannot be satisfied by the platform at all, because they are properties of what an educator writes. A platform can only make the requirement visible at the moment of writing, which is precisely what the compliance auditor described in Section 4.3.6 exists to do.

4.3.3 The three learner profiles

Table 4.4 states, for each of the three profiles the platform is designed around, the accessibility needs it addresses and the concrete support ACESS provides. Every entry in the third column corresponds to code that runs. Nothing in it is aspirational.

Table 4.4: Learner Profiles and ACESS Support

Profile	Main accessibility needs	ACESS support
Dyslexia	• Reduced decoding effort • Reduced visual crowding between letters, words and lines • A reliable way not to lose one's place in a long passage • An alternative to reading • Reduced glare over long sessions	• Atkinson Hyperlegible or OpenDyslexic typeface, 19 px base size, 1.7 line spacing • Word spacing at the 0.16 em WCAG floor • Reading measure narrowed to 62 characters (the BDA recommends 50–70) • Cream background, and a reading spotlight that dims all but the current paragraph • Text-to-speech with adjustable rate and a click-to-read-from-here control • A dedicated reading toolbar carrying size, spacing, tint and spotlight controls without leaving the lesson • Italic rendered as bold rather than slanted, text never justified • Content presented one section at a time, on a single-column dashboard rather than a card grid
ADHD	• Sustained attention is costly • Task initiation is hard • An interruption destroys one's place • Passive reading does not retain • Competing on-screen elements all pull equally	• Distraction-free mode hiding sidebar and notification chrome • Reading measure held at 66 characters, *including* while chrome is hidden • A persistent "Now" bar carrying the single current task and its progress • A task checklist and a progress timeline • An auto-save indicator so that leaving the page carries no risk • Reading spotlight • Content chunked into sections with a per-section reading-time estimate
		On the authoring side, the auditor caps a lesson at 20 minutes, an unbroken block at 150 words, expects a section break roughly every 400 words, expects video under six minutes or in-video questions, and requires at least one thing to *do* in every lesson.

continued on the next page

Table 4.4 continued from the previous page

Profile	Main accessibility needs	ACESS support
Autism	• Uncertainty about what is coming is itself a barrier • Sensory input can overload • Figurative language must be decoded before content can be • Inconsistent structure has to be relearned every time	• A visual schedule listing every phase of the lesson, with progress, *before* the learner starts • Step-by-step guidance that always states why a "Next" control is unavailable rather than merely disabling it • Checklist structure mode, which states expectations before each task • Motion removed entirely, colour saturation reduced, pale blue background • The reading measure fixed at the same value for every course, so the page does not change shape between lessons • No autoplay anywhere
		On the authoring side, the auditor checks for figurative language, requires at least two stated learning objectives and a plain-language summary of at least 15 words, and checks structural consistency against the other lessons of the same course.

4.3.4 Preset composition

A preset is not a theme. It is a single decision by the learner that sets nineteen individual values at once, spanning typography, colour, layout, pacing and executive-function scaffolding. Table 4.5 gives the exact values, taken from the platform's own preset definitions in `src/lib/adaptive-engine.ts`. Every one of these settings remains individually adjustable afterwards. Choosing a preset is a starting point, not a mode the learner is locked into.

Table 4.5: Accessibility Preset Values

Setting	Default	Dyslexia	ADHD	Autism
Typeface	Arial	Atkinson Hyperlegible	Arial	Arial
Base font size	16 px	**19 px**	**18 px**	**18 px**
Line-spacing multiplier	1.5	**1.7**	**1.6**	**1.6**
Word spacing	0 % (0 em)	**40 % (0.16 em)**	**20 % (0.08 em)**	**20 % (0.08 em)**
Reading measure	72 ch	**62 ch**	**66 ch**	72 ch
Background tint	White	**Cream**	**Grey**	**Pale blue**

continued on the next page

Table 4.5 continued from the previous page

Setting	Default	Dyslexia	ADHD	Autism
Reading spotlight	Off	**On**	**On**	Off
Distraction-free mode	Off	Off	**On**	**On**
Layout mode	Slide	**One section at a time**	**One section at a time**	**Scroll**
Structure mode	Full schedule	Full schedule	**Minimal**	**Checklist**
Animation level	Normal	**Low**	**Low**	**None**
Muted colours	Off	Off	Off	**On**
Text-to-speech	Off	**On**	Off	Off
Task checklist	Off	Off	**On**	Off
Visual schedule	Off	Off	Off	**On**
Step-by-step guidance	Off	Off	Off	**On**
Progress timeline	Off	Off	**On**	**On**
Auto-save indicator	Off	**On**	**On**	**On**
Lesson chunking	Off	**On**	**On**	**On**

Applying a preset is itself an accessibility decision, and it is handled as one. Figure 4.19 shows the dialog that appears *before* a preset takes effect: it lists every value that will change, from what to what, with an explanation available for each, and it states that any of it can be changed afterwards one setting at a time. This is the platform's implementation of WCAG 3.2.5, and it exists because an interface that silently rearranges itself is precisely the kind of unpredictability the Autism profile is built to remove.

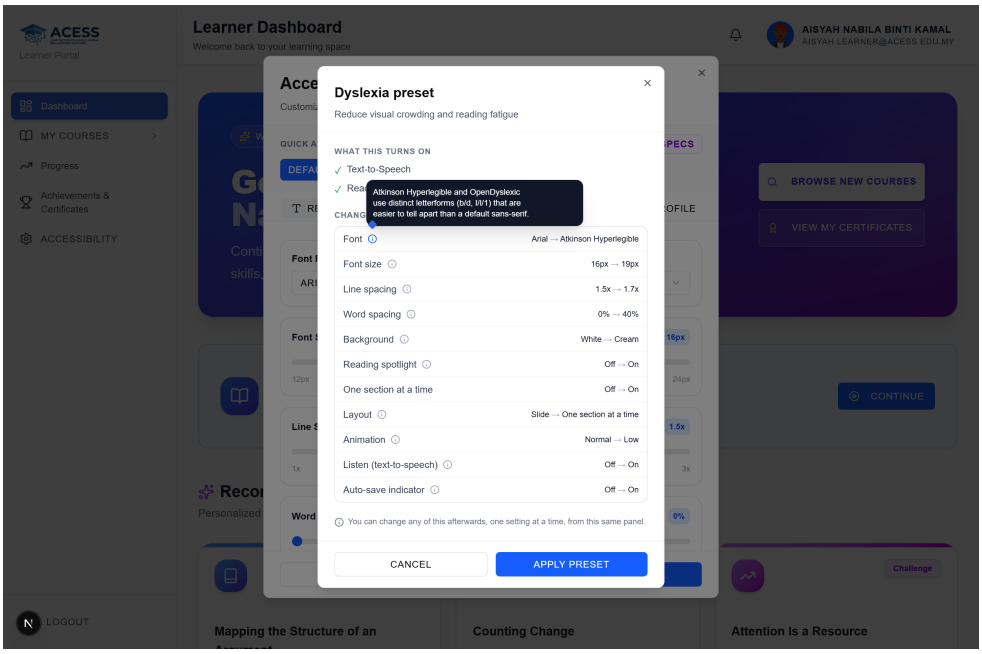
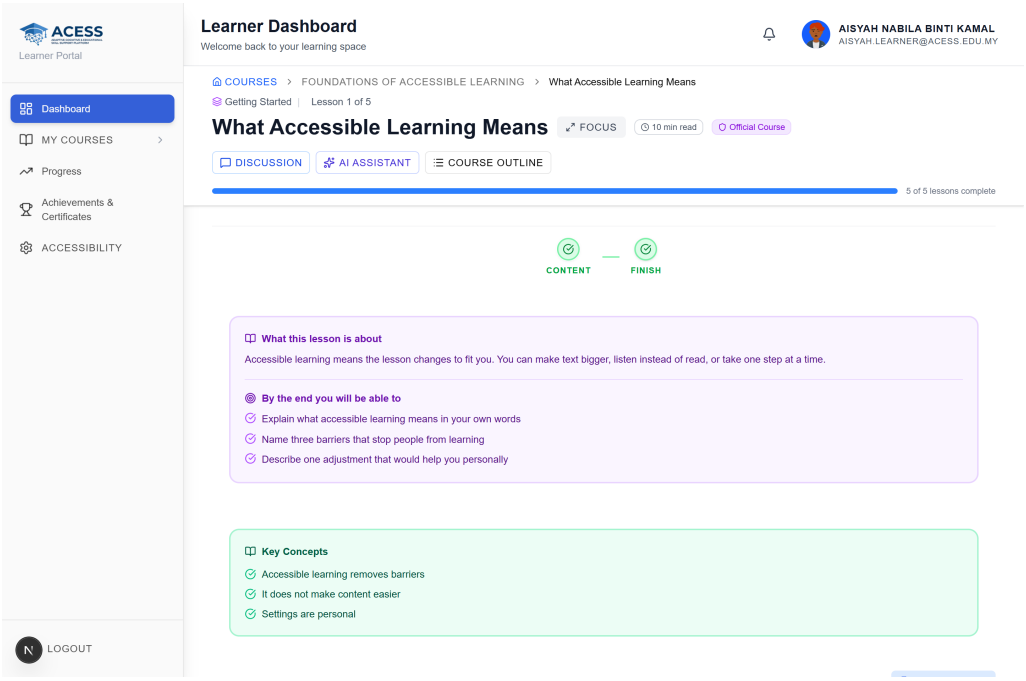


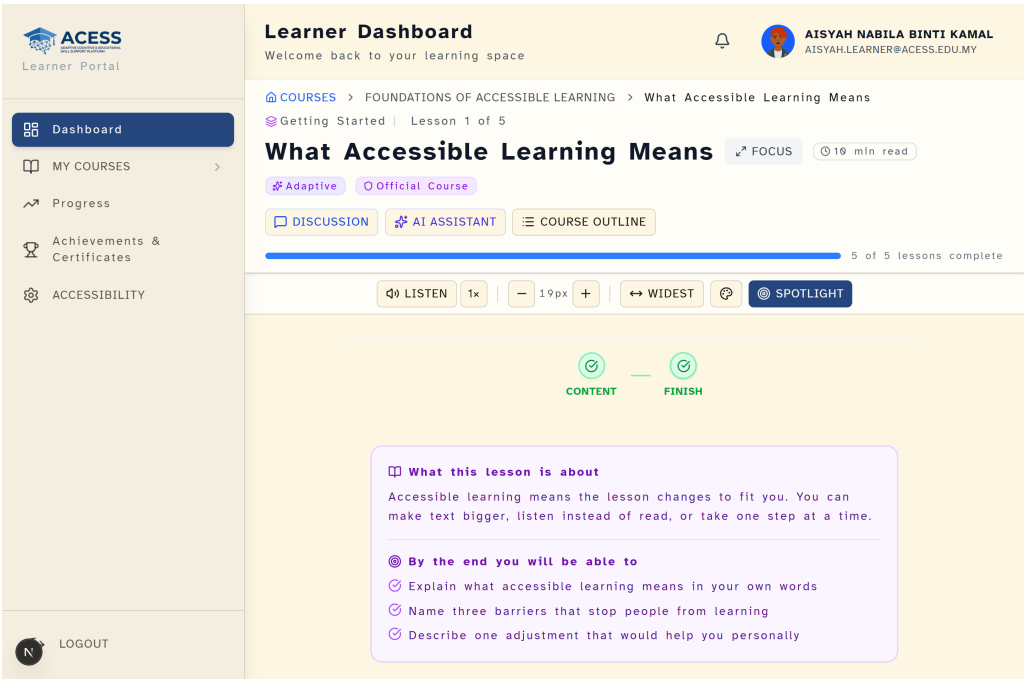
Figure 4.19: Preset Preview Dialog

4.3.5 Effect of the presets on the lesson interface

Describing an adaptive interface is less convincing than showing it. Figures 4.20 to 4.22 show the *same lesson*, viewed by the *same learner account*, at the same viewport size, with only the accessibility preset changed between captures. The preset values applied in each case were written by the platform's own preset function rather than set by hand, matching Table 4.5 exactly, so the comparison shows what a learner selecting that preset actually receives.



(a) Default



(b) Dyslexia Preset

Figure 4.20: Default vs. Dyslexia Preset

Figure 4.20 shows the change most people expect from an accessibility feature, typography and colour, but also two that are easy to miss. The reading column has narrowed rather than widened, because the constraint is expressed in characters and the characters got bigger. A persistent toolbar has also appeared below the lesson header,

carrying Listen, size, spacing, tint and spotlight controls, so that the learner can adjust size, spacing, tint and read-aloud *without leaving the lesson to find a settings panel*. Requiring a learner to navigate away in order to make reading easier is itself a barrier.

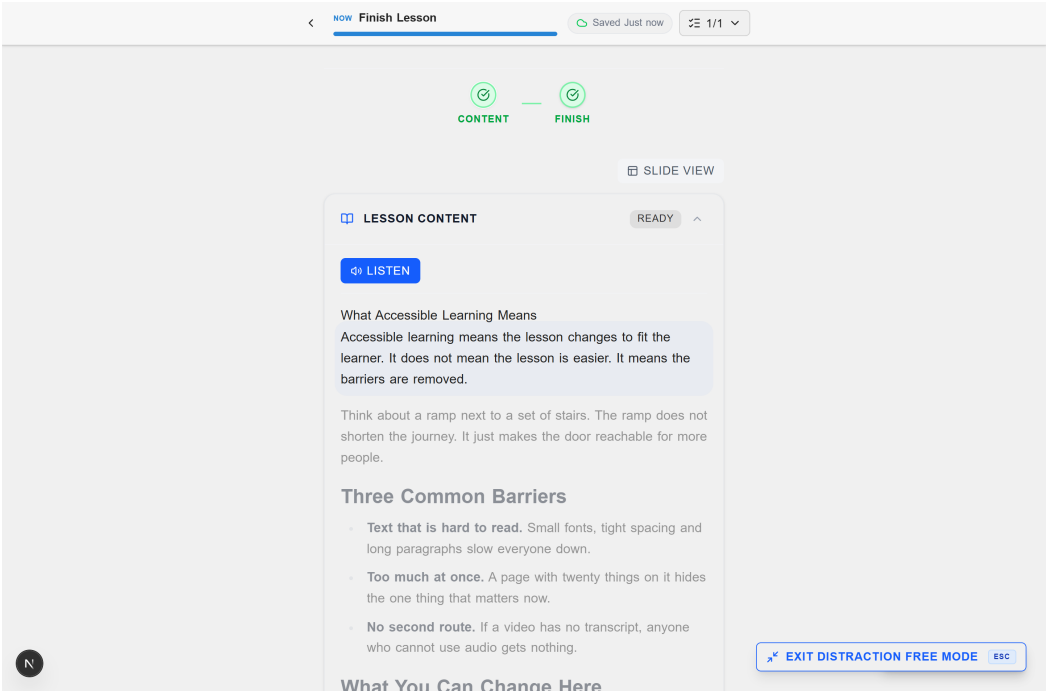


Figure 4.21: Lesson Under the ADHD Preset

Figure 4.21 is the clearest illustration of the difference between hiding things and supporting attention. The sidebar and notification chrome are removed, a persistent "Now" bar carries the current task together with an auto-save indicator, and the reading spotlight dims every paragraph except the one in focus. Everything that competes for attention has gone, but the reading column has *not* expanded to fill the space, and it is still bounded at 66 characters. This was a defect at one point in development: distraction-free mode released the measure and produced the widest possible line length, which is the opposite of what a learner with attention difficulties needs. The exit control in the lower right, and the Escape key it advertises, exist so that the mode is never a trap.

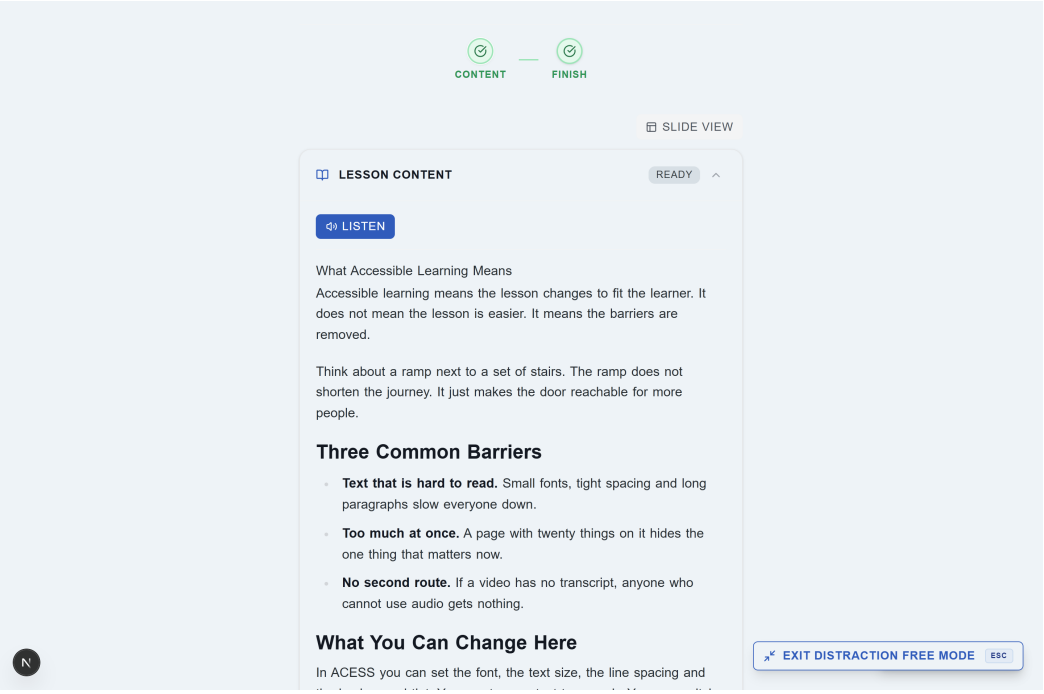


Figure 4.22: Lesson Under the Autism Preset

Figure 4.22 shows the least visually dramatic and most structurally significant of the three, with its pale blue background, reduced colour saturation and motion removed. The palette softens, but the substantive change is the lesson's phases shown as an explicit sequence above the content, which tells the learner what this lesson consists of and how far through it they are before they begin. The reading measure is deliberately *not* narrowed here: unlike dyslexia, where a narrow measure serves decoding, the Autism profile's need is predictability, and holding the same width as the default means the page does not change shape when the learner moves between a course that uses this preset and one that does not.

Figure 4.23 shows the effect at the dashboard rather than the lesson level, where the Dyslexia preset replaces the multi-column card grid with a single-column list and widens and enlarges the navigation, in place of the default card grid of Figure 4.8.

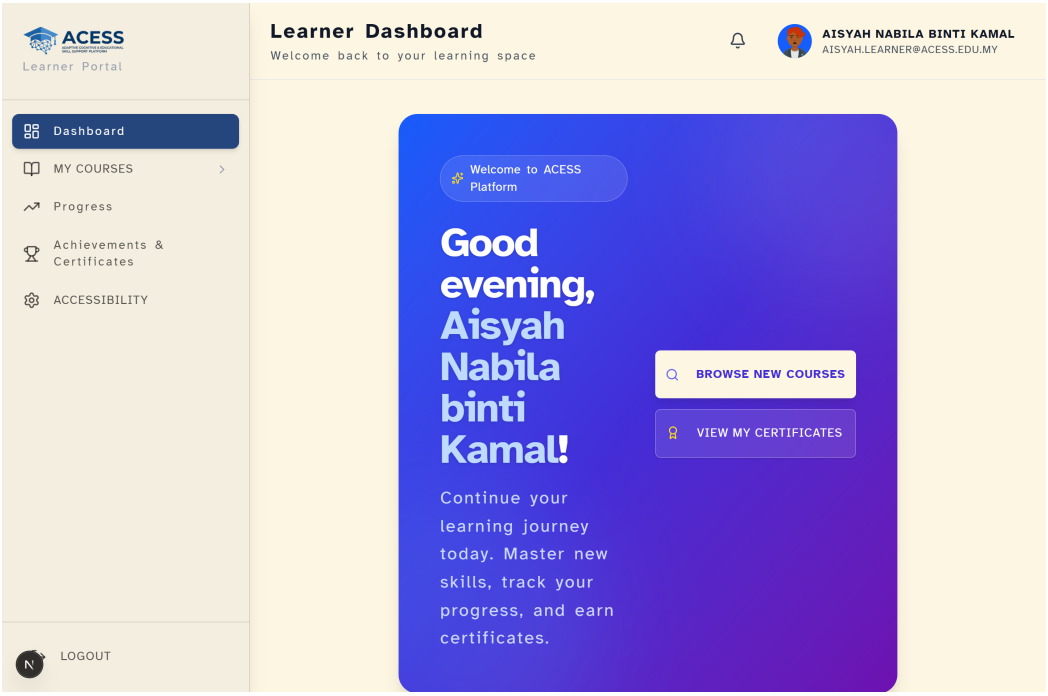


Figure 4.23: Learner Dashboard Under the Dyslexia Preset

4.3.6 Authoring-side compliance auditing

The content criteria identified in Section 4.3 cannot be met by the platform on the educator's behalf. ACESS therefore implements a deterministic auditor that evaluates a lesson while it is being written.

The engine has three properties that make its output defensible. It is **deterministic**: every rule is a fixed threshold with a published source, so the same lesson always produces the same score, since there is no model and no randomness. It is **pure and synchronous**, which is why the same engine can run against unsaved editor state on every keystroke and against rows read from the database for the course-level report, and the two necessarily agree. And it is **profile-aware**: a baseline of seven rules applies to every lesson, and a further set is added according to the course's declared focus profile (six for ADHD, five for Autism, seven for Dyslexia), so an educator writing for dyslexic readers is checked on sentence length and readability, while one writing for autistic learners is checked on figurative language and stated objectives.

Rules that cannot apply are marked *not applicable* rather than failed, and are excluded from the score: a lesson with no video is not penalised for lacking a transcript. The score is the weighted proportion of applicable rules passed, banded as critical below 50, warning below 80, and good at 80 or above.

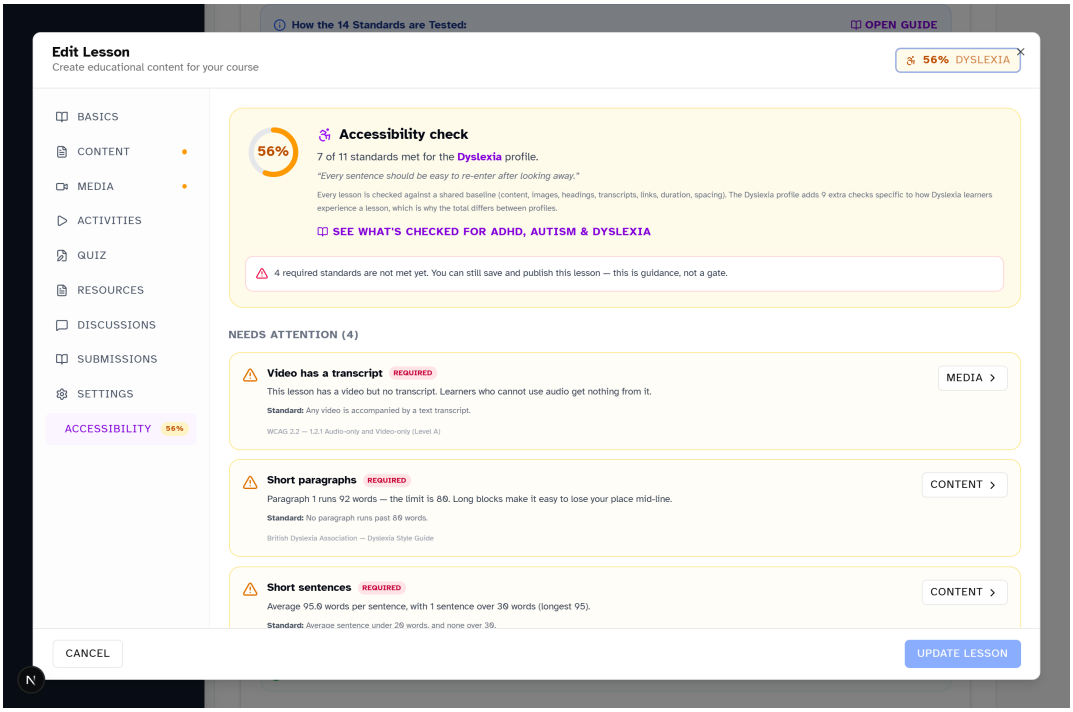


Figure 4.24: Lesson Accessibility Check in the Educator Editor

Figure 4.24 shows the auditor reporting on a real seeded lesson, with the score for the course's declared profile shown alongside the finding list. Three further properties of that output are worth noting, because they are what distinguish it from a generic accessibility linter. Each finding states *the measured value*, not merely that a rule failed: rather than "paragraph too long", a finding reads "Paragraph 1 runs 92 words, and the limit is 80". Each finding names *its source*, so an educator who disagrees can consult the standard rather than the tool. And each finding carries *a route to the fix*, jumping to the editor tab that owns the problem.

The panel also states, in the same view, that four required standards are unmet and that the lesson can still be saved and published. That is a deliberate policy choice. A publication gate would have produced compliance theatre (educators padding summaries to clear a word count) and would have positioned accessibility as an obstacle to teaching.

The auditor advises. The educator decides. What the platform guarantees is that the decision is an informed one.

The same engine runs at course level, aggregating the fourteen standards that apply across a whole course and identifying which lessons are responsible for each unmet one, so that an educator can see whether a problem is a single weak lesson or a course-wide setting.

4.3.7 Known accessibility gaps

This section exists because an accessibility statement with no gaps is not credible. Each of the following is a real limitation of the current build, and each is disclosed on the platform's own public accessibility statement as well as here.

1. **No assistive technology has been tested against this build.** No screen reader (NVDA, VoiceOver or TalkBack) and no other assistive technology has been used against ACESS. Every claim in Table 4.3 is verified by reading and tracing the code that implements it and by exercising it in a browser, not by using it as a screen-reader user would. This is the most significant limitation in this chapter.

2. **No disabled learner has tested the platform.** No learner with lived experience of dyslexia, ADHD or autism took part in building or evaluating it. The design draws on published guidance rather than on the people it is for, which is a real methodological weakness and is discussed further in Section 4.6.

3. **The "Muted Colors" setting is over-broad.** It is implemented as a global saturation filter, so it desaturates error, warning and success colours along with everything else, making status information harder to distinguish, which is the opposite of the intent. A correct fix requires reworking colour tokens throughout the stylesheet and was judged too large to attempt without visual verification. The setting's own description in the platform catalogue states this current behaviour rather than the intended one.

4. **The "Keyboard navigation" setting does not do what its label promises.** It sets a document attribute that nothing currently reads. Keyboard operability should not be optional in any case. The entry is retained in the catalogue so that the gap is visible rather than silently absent, pending either implementation or redefinition.

5. **Only one preset can be active at a time.** A learner who would benefit from Dyslexia typography together with ADHD pacing supports must currently choose one and adjust the rest manually. Composable profiles are the most frequently identified extension to this design.

6. **One drag-and-drop mode has no non-drag alternative.** The "categories" mode and every other activity type have a keyboard path and, where relevant, a non-drag alternative. The "diagram" mode, which places labels at arbitrary coordinates on an image, does not.

7. **Read-from-here is pointer-only.** Clicking a paragraph during read-aloud restarts speech from that paragraph, but this is not reachable from the keyboard. The Listen control remains the keyboard baseline.

8. **No quantitative accessibility baseline was captured.** No before-and-after measurement of learner outcomes exists, because no baseline was taken before the accessibility work began. The comparisons in this chapter are therefore demonstrations that the interface adapts as designed, not evidence that the adaptation improves learning outcomes. Establishing that would require the study described in Section 4.6.

4.4 Data Dictionary and Normalisation

4.4.1 Normalisation

The schema is normalised to Third Normal Form, with two documented and deliberate departures. The reasoning is set out here rather than left implicit, because both departures would otherwise read as oversights.

Every table has a single-column surrogate primary key, so first normal form holds trivially and no partial dependency on part of a composite key is possible, which gives second normal form. Third normal form (no transitive dependency of a non-key attribute on another non-key attribute) holds throughout the operational tables: a lesson stores its `course_id` but not its course's title, a quiz attempt stores its `enrollment_id` but not the learner's name, and progress rows carry no copy of the lesson's duration.

The two departures are:

1. **The certificate record is deliberately denormalised.** `certificates` stores `learner_name`, `course_title`, `educator_name`, `institution_name` and `course_duration_hours` as literal values, even though every one of them could be joined for. This is intentional and is the correct design for this entity: a certificate is a statement about a moment in time. If the course were later renamed, or the learner changed their display name, a joined certificate would silently rewrite history and no longer match the PDF the learner already holds. Freezing the values at issue is what makes the document verifiable years later.

2. **Accessibility and notification preferences are stored as JSON documents** on `user_profiles` rather than decomposed into columns. Strictly, this is a repeating group and is not first normal form. It was chosen because the setting vocabulary changes considerably faster than the rest of the schema (the platform's own catalogue currently defines twenty-four learner-facing settings, and that list has been revised repeatedly during development), and because every consumer reads the entire object at once rather than querying individual settings. The cost is that the database cannot constrain an individual setting's value. The platform compensates by defining the permitted vocabulary in a single application-level catalogue against which an automated check runs, so that a setting cannot exist in the data without also existing in that catalogue.

Two further points are worth recording. First, the schema contains no many-to-many relationship implemented without a junction table:

`user_achievements` and `course_favorites` both exist precisely to resolve one. Second, one denormalised column pair was removed during the schema consolidation described in Section 4.5.2: `lessons.chapter_title` duplicated `course_chapters.title` and could disagree with it, so the current schema is closer to 3NF than the design documented in the previous revision of this report.

Reading the tables

The following tables constitute the physical schema of the ACESS PostgreSQL database as it currently stands: 36 base tables and 364 columns (370 including the six columns of the `quiz_options_scoped` view, which is described in Section 4.5.2). They were generated directly from the live database catalogue rather than transcribed, so the attribute names, types, defaults and constraints in them are the ones the database actually holds. The **Constraint** column reports the key role, referential target, uniqueness and nullability of each attribute. `CHECK` constraints, which apply to a value rather than to a column's structure, are discussed with the physical design in Section 4.5.2.

4.4.2 Core Identity and Configuration

Tables 4.6 to 4.13 define the 8 entities of the core identity and configuration domain.

1. Table: users

Table 4.6: Users Data Dictionary

Attribute	Type	Constraint	Description
id	uuid	PRIMARY KEY	Unique identifier (UUID v4).
email	character varying	UNIQUE, NOT NULL	User email address, used for login and notifications.
role	character varying	NOT NULL	User role: learner, educator, or admin.
is_active	boolean DEFAULT true	NOT NULL	Soft-ban flag, inactive users cannot log in.
email_verified_at	timestamp	-	Timestamp when user confirmed email via verification link, NULL if unverified.
last_login_at	timestamp	-	Timestamp of last successful login.
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp DEFAULT now()	NOT NULL	Timestamp of the last modification.
deleted_at	timestamp	-	Soft-delete timestamp, null if not deleted.
full_name	text	-	User's display name shown across the platform.
instructor_application_status	text	-	Status of educator application.

2. Table: user_profiles

Table 4.7: User Profiles Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, UNIQUE, NOT NULL	Foreign key referencing users.id.
username	text	UNIQUE	Public-facing username handle.
avatar_url	text	-	URL to the user's profile image.
phone_number	text	-	Contact phone number for account recovery and notifications.
birth_date	date	-	User's date of birth for age verification and age-group analytics.
bio	text	-	Short biographical text for educator profiles.
country	text	-	ISO 3166-1 country code for regional content localisation.
preferred_language	text DEFAULT 'en'	-	UI language code (en, ms).
created_at	timestamptz DEFAULT now()	-	Timestamp when the record was created.
updated_at	timestamptz DEFAULT now()	-	Timestamp of the last modification.
disability_type	character varying	-	Disability classification for adaptive engine.
accessibility_prefs	jsonb DEFAULT ''	-	JSON object storing active accessibility settings (font size, colour scheme, TTS rate, dyslexia font toggle).
notification_prefs	jsonb DEFAULT ''	-	JSON object storing notification delivery preferences (email, in-app, frequency).

3. Table: referral_codes

Table 4.8: Referral Codes Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
code	text	UNIQUE, NOT NULL	Unique alphanumeric code assigned to a user for referral tracking.
user_id	uuid	FK → users	Foreign key referencing users.id.
usage_count	integer DEFAULT 0	-	Number of times used.
max_uses	integer DEFAULT 50	-	Maximum number of times this referral code can be redeemed.
is_active	boolean DEFAULT true	-	Soft-ban flag, inactive users cannot log in.
created_at	timestamptz DEFAULT now()	-	Timestamp when the record was created.

4. Table: instructor_applications

Table 4.9: Instructor Applications Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users	Foreign key referencing users.id.
full_name	text	NOT NULL	User's display name shown across the platform.
email	text	NOT NULL	User email address, used for login and notifications.
experience	text	-	Free-text description of applicant's teaching or subject-matter experience.
reason	text	-	Applicant's motivation statement for wanting to become an educator on the platform.
portfolio_links	text	-	Comma-separated URLs to the applicant's teaching portfolio or work samples.
referral_code	text	-	Unique referral token string.
status	text DEFAULT 'pending'	NOT NULL	Record status (e.g., active, completed, dropped).
admin_notes	text	-	Internal reviewer notes regarding the application decision.
reviewed_by	uuid	FK → users	UUID of the administrator who reviewed the application.
reviewed_at	timestamptz	-	Timestamp of the review decision.
created_at	timestamptz DEFAULT now()	-	Timestamp when the record was created.
updated_at	timestamptz DEFAULT now()	-	Timestamp of the last modification.

5. Table: contact_messages

Table 4.10: Contact Messages Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
name	text	NOT NULL	Display name of the record.
email	text	NOT NULL	User email address, used for login and notifications.
category	text	NOT NULL	Content category classification.
subject	text	NOT NULL	Message subject line.
message	text	NOT NULL	Full body text of the contact form submission.
status	text DEFAULT 'unread'	NOT NULL	Record status (e.g., active, completed, dropped).
created_at	timestamptz DEFAULT now()	-	Timestamp when the record was created.

6. Table: notifications

Table 4.11: Notifications Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, NOT NULL	Foreign key referencing users.id.
type	text	NOT NULL	Classification type of the record.
title	text	NOT NULL	Display title of the record.
body	text	-	Message body text content.
metadata	jsonb DEFAULT ''	-	JSONB field for extensible metadata.
is_read	boolean DEFAULT false	-	Whether the message has been read.
created_at	timestamptz DEFAULT now()	-	Timestamp when the record was created.

7. Table: accessibility_templates

Table 4.12: Accessibility Templates Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
name	text	NOT NULL	Display name of the record.
description	text	-	Free-text description or summary.
target_disability	text	NOT NULL	Disability category the template targets (e.g. 'dyslexia', 'adhd', 'asd').
content_structure	jsonb	NOT NULL	JSON definition of template layout rules and component arrangement.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.

8. Table: adaptive_interactions

Table 4.13: Adaptive Interactions Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, NOT NULL	Foreign key referencing users.id.
lesson_id	uuid	FK → lessons	Foreign key referencing lessons.id.
course_id	uuid	FK → courses	Foreign key referencing courses.id.
adaptation_used	text	NOT NULL	Identifier of the adaptive preset applied (e.g. 'dyslexia_standard', 'adhd_focus').
session_id	text	-	Unique identifier linking interactions within a single browsing session.
duration_seconds	integer	-	Duration in seconds.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.

4.4.3 Curriculum Structure

Tables 4.14 to 4.22 define the 9 entities of the curriculum structure domain.

9. Table: courses

Table 4.14: Courses Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
created_by	uuid	FK → users, NOT NULL	Foreign key referencing the creating user.
title	character varying	NOT NULL	Display title of the record.
slug	character varying	UNIQUE, NOT NULL	URL-safe identifier string.
description	text	-	Free-text description or summary.
status	character varying DEFAULT 'draft'	NOT NULL	Record status (e.g., active, completed, dropped).
difficulty_level	character varying	-	Course difficulty classification ('beginner', 'intermediate', 'advanced') for learner filtering.
thumbnail_url	character varying	-	URL to the thumbnail image.
published_at	timestamp	-	Timestamp when published.
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp DEFAULT now()	NOT NULL	Timestamp of the last modification.

continued on the next page

Table 4.14 continued from the previous page

Attribute	Type	Constraint	Description
deleted_at	timestamp	-	Soft-delete timestamp, null if not deleted.
category	text	-	Content category classification.
course_type	text DEFAULT 'educator'	NOT NULL	Origin of course creation ('educator' or 'system') determining edit permissions.
system_course	boolean DEFAULT false	NOT NULL	Boolean flag indicating the course was created by administrators, not educators.
built_in_course	boolean DEFAULT false	NOT NULL	Boolean flag marking pre-seeded platform courses that cannot be deleted.
created_by_role	text DEFAULT 'educator'	NOT NULL	Role of the user who created the course ('educator' or 'admin') for permission checks.
guided_learning_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling step-by-step guided lesson progression.
official_course_order	integer	-	Integer display priority for system-created courses in the catalog.
managed_by_admin	boolean DEFAULT false	NOT NULL	Boolean flag indicating administrative oversight of educator-created course.
recommended_age_group	text	-	Suggested learner age range for age-appropriate content filtering.
learning_streaks_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle activating daily learning streak tracking and rewards.
milestone_tracking_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling course milestone progress indicators.
course_layout_type	text DEFAULT 'standard'	NOT NULL	Visual layout template ('standard', 'simplified') for course pages.
chapter_organization_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling hierarchical chapter-based lesson grouping.
certificate_enabled	boolean DEFAULT false	-	Boolean toggle allowing certificate generation upon course completion.
certificate_settings	jsonb DEFAULT ''	-	JSON object storing certificate configuration (template, signature, expiry).
certification_locked	boolean DEFAULT false	-	Boolean flag preventing certificate issuance until all criteria are met.
supports_tts	boolean DEFAULT false	-	Boolean flag indicating whether course lessons support text-to-speech playback.
supports_transcripts	boolean DEFAULT false	-	Boolean flag indicating whether video lessons include text transcripts.

continued on the next page

Table 4.14 continued from the previous page

Attribute	Type	Constraint	Description
supports_focus_mode	boolean DEFAULT false	-	Boolean flag indicating whether Focus Mode is available for this course.
supports_chunked_learning	boolean DEFAULT false	-	Boolean flag enabling content to be broken into smaller learning chunks.
tags	text[] DEFAULT ''	-	JSONB array of tag strings.
accessibility_categories	text[] DEFAULT ''	-	Array of disability categories this course is designed to support.
primary_disability_focus	text	-	Main disability category the course content is optimised for.
secondary_disability_focuses	text[] DEFAULT ''	-	Array of additional disability categories the course partially supports.
target_reading_age	integer DEFAULT 13	-	Minimum reading level the course content is written at for age-appropriate filtering.
educator_custom_guide	text	-	Free-text educator instructions for how to use or navigate this course.

10. Table: course_chapters

Table 4.15: Course Chapters Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
course_id	uuid	FK → courses, UNIQUE, NOT NULL	Foreign key referencing courses.id.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
sequence_order	integer DEFAULT 0	UNIQUE, NOT NULL	Integer determining the display order of this chapter within the course.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.

11. Table: lessons

Table 4.16: Lessons Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
course_id	uuid	FK → courses, UNIQUE, NOT NULL	Foreign key referencing courses.id.
title	character varying	NOT NULL	Display title of the record.

continued on the next page

Table 4.16 continued from the previous page

Attribute	Type	Constraint	Description
content_html	text	-	Rich-text HTML body of the lesson rendered via Tiptap editor.
video_url	character varying	-	URL of the embedded video asset for this lesson.
transcript	text	-	Full text transcript of the lesson content for accessibility compliance.
sequence_order	integer	UNIQUE, NOT NULL	Integer determining the display order of this lesson within its course.
status	character varying DEFAULT 'draft'	NOT NULL	Record status (e.g., active, completed, dropped).
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp DEFAULT now()	NOT NULL	Timestamp of the last modification.
estimated_duration	integer	-	Approximate lesson completion time in minutes for progress estimation.
prerequisite_lesson_id	uuid	FK → lessons	UUID of the lesson that must be completed before this one unlocks.
simplified_summary	text	-	Plain-text summary of lesson content for simplified reading mode.
accessibility_notes	text	-	Accessibility improvement notes.
focus_mode_enabled	boolean DEFAULT false	NOT NULL	Boolean flag allowing Focus Mode override for this specific lesson.
chunked_content_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling lesson content to be split into digestible segments.
lesson_type	text DEFAULT 'standard'	NOT NULL	Lesson classification ('standard', 'video', 'interactive') determining content rendering.
chapter_id	uuid	-	Foreign key referencing course_chapters.id.
learning_objectives	text	-	Semicolon-separated list of learning outcomes displayed before lesson content.
visibility_status	text DEFAULT 'visible'	NOT NULL	Content visibility state ('visible', 'hidden', 'draft') controlling catalog display.
checkpoints_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle requiring milestone checkmarks before lesson progression.
adaptive_learning_enabled	boolean DEFAULT false	NOT NULL	Boolean flag activating adaptive content delivery for this lesson.
has_video	boolean DEFAULT true	-	Boolean flag indicating whether this lesson contains an embedded video.

continued on the next page

Table 4.16 continued from the previous page

Attribute	Type	Constraint	Description
has_pdf	boolean DEFAULT true	-	Boolean flag indicating whether this lesson has a downloadable PDF resource.
has_quiz	boolean DEFAULT true	-	Boolean flag indicating whether this lesson has an associated quiz.
has_transcript	boolean DEFAULT true	-	Boolean flag indicating whether this lesson includes a text transcript.
has_summary_activity	boolean DEFAULT false	-	Boolean flag enabling the learner summary submission activity.
summary_source	text DEFAULT 'entire_lesson'	-	Content scope for summary generation ('entire_lesson', 'current_section').
summary_word_target	integer DEFAULT 100	-	Target word count for learner summary submissions.
summary_key_points	jsonb DEFAULT '[]'	-	JSON array of expected key points for summary evaluation.
summary_reflection_questions	jsonb DEFAULT '[]'	-	JSON array of reflection prompts shown after summary submission.
summary_ai_feedback_enabled	boolean DEFAULT false	-	Boolean toggle enabling AI-generated feedback on summaries.
lesson_layout	text DEFAULT 'standard'	-	Visual template ('standard', 'focus', 'minimal') for lesson page rendering.
has_h5p	boolean DEFAULT false	-	Flag indicating that the lesson carries at least one embedded H5P interactive object.
accessibility_score	integer DEFAULT 100	-	Accessibility compliance score.
allow_discussions	boolean DEFAULT false	-	Boolean toggle enabling threaded comments on this lesson.
allow_download	boolean DEFAULT false	-	Boolean toggle enabling resource file downloads for this lesson.

12. Table: course_accessibility_categories

Table 4.17: Course Accessibility Categories Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
course_id	uuid	FK → courses, NOT NULL	Foreign key referencing courses.id.
accessibility_category	text	NOT NULL	Disability category label (e.g. 'dyslexia', 'adhd', 'asd') linked to this course.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.

13. Table: course_milestones

Table 4.18: Course Milestones Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
course_id	uuid	FK → courses, NOT NULL	Foreign key referencing courses.id.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
required_completion_pct	integer DEFAULT 100	NOT NULL	Minimum percentage of lessons that must be completed to achieve this milestone.
icon	text	-	Emoji or icon identifier displayed alongside the milestone label.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this milestone in the progress tracker.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.

14. Table: course_achievements

Table 4.19: Course Achievements Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
course_id	uuid	FK → courses, NOT NULL	Foreign key referencing courses.id.
name	character varying	NOT NULL	Display name of the record.
description	text	NOT NULL	Free-text description or summary.
icon_url	character varying	-	URL of the achievement badge image displayed in the learner's profile.
requirement_type	character varying	NOT NULL	Achievement criterion type (e.g. 'quiz_score', 'course_complete', 'streak').
requirement_threshold	integer DEFAULT 1	NOT NULL	Numeric threshold value that must be met to unlock this achievement.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamptz DEFAULT now()	NOT NULL	Timestamp of the last modification.

15. Table: lesson_templates

Table 4.20: Lesson Templates Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
created_by	uuid	FK → users, NOT NULL	Foreign key referencing the creating user.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
lesson_type	text DEFAULT 'standard'	NOT NULL	Template classification ('standard', 'video', 'interactive') for the template.
content_html	text	-	Default HTML content body pre-filled in the template for educator customisation.
estimated_duration	integer	-	Default approximate completion time in minutes suggested by the template.
is_public	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether this template is available to all educators.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamptz DEFAULT now()	NOT NULL	Timestamp of the last modification.

16. Table: lesson_versions

Table 4.21: Lesson Versions Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing lessons.id.
content_html	text	NOT NULL	Snapshot of the lesson HTML content at the time this version was saved.
version_name	text	NOT NULL	Human-readable label identifying this version (e.g. 'v1', 'draft-2024-03').
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.
created_by	uuid	FK → users, NOT NULL	Foreign key referencing the creating user.

17. Table: media_assets

Table 4.22: Media Assets Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, NOT NULL	Foreign key referencing users.id.
course_id	uuid	FK → courses	Foreign key referencing courses.id.
file_name	character varying	NOT NULL	Original uploaded file name including extension (e.g. 'lecture_slides.pdf').
file_type	character varying	NOT NULL	MIME type of the file.
url	character varying	NOT NULL	External resource URL.
size_bytes	bigint	-	File size in bytes for storage quota enforcement.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.
lesson_id	uuid	FK → lessons	Foreign key referencing lessons.id.
title	character varying	-	Display title of the record.

4.4.4 Lesson Content and Interaction

Tables 4.23 to 4.28 define the 6 entities of the lesson content and interaction domain.

18. Table: lesson_interactive_content

Table 4.23: Lesson Interactive Content Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing lessons.id.
content_type	text	NOT NULL	MIME type of the content.
title	text	NOT NULL	Display title of the record.
content_data	jsonb DEFAULT ''	NOT NULL	JSONB payload containing the H5P interactive activity configuration and state.
accessibility_settings	jsonb DEFAULT ''	-	JSON object storing accessibility overrides specific to this interactive activity.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this activity within the lesson.
created_by	uuid	FK → users	Foreign key referencing the creating user.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamptz DEFAULT now()	NOT NULL	Timestamp of the last modification.
is_draft	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether this interactive content is unpublished.

19. Table: video_questions

Table 4.24: Video Questions Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing lessons.id.
title	text	NOT NULL	Display title of the record.
timestamp_seconds	numeric	NOT NULL	Video playback position in seconds where the question is displayed.
question_text	text	NOT NULL	Display text of the question.
options	jsonb DEFAULT '[]'	NOT NULL	JSONB array of answer options.
correct_option_index	integer DEFAULT 0	NOT NULL	Zero-based index of the correct answer in the options array.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this question within the video.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamptz DEFAULT now()	NOT NULL	Timestamp of the last modification.

20. Table: h5p_contents

Table 4.25: H5P Contents Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing the lesson the interactive object is embedded in.
title	text	NOT NULL	Display title shown above the embedded object.
embed_url	text	NOT NULL	URL of the externally hosted H5P object, rendered inside a sandboxed frame.
source_url	text	-	Canonical page for the object on its host, retained for attribution and re-editing.
description	text	-	Optional explanatory text shown to the learner before the object loads.
width	text DEFAULT '100%'	-	CSS width applied to the embed frame, defaults to full container width.
height	text DEFAULT '500px'	-	CSS height applied to the embed frame.
sequence_order	integer DEFAULT 0	NOT NULL	Position of this object among the lesson's embedded objects.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamptz DEFAULT now()	NOT NULL	Timestamp of the last modification.
thumbnail_url	text	-	Optional preview image shown before the object is loaded.

21. Table: lesson_checkpoints

Table 4.26: Lesson Checkpoints Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing lessons.id.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
checkpoint_type	text DEFAULT 'reflection'	NOT NULL	Checkpoint category ('reflection', 'quiz', 'activity') determining interaction format.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this checkpoint within the lesson.
required	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether this checkpoint must be passed to proceed.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.

22. Table: lesson_ai_summaries

Table 4.27: Lesson AI Summaries Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, UNIQUE, NOT NULL	Foreign key referencing the lesson the summary describes.
summary	text	NOT NULL	Cached plain-language summary of the lesson, generated on first request and reused thereafter.
suggested_questions	text[] DEFAULT ''	-	Generated prompts offered to the learner as starting points for the lesson assistant.
source_content_hash	text	NOT NULL	Digest of the lesson content the summary was generated from, so a change to the lesson invalidates the cache rather than serving a stale summary.
model	text	NOT NULL	Identifier of the generative model that produced the summary, recorded so that output can be attributed and regenerated deliberately.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the summary was first generated.
updated_at	timestamptz DEFAULT now()	NOT NULL	Timestamp of the last regeneration.

23. Table: lesson_comments

Table 4.28: Lesson Comments Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, NOT NULL	Foreign key referencing lessons.id.
user_id	uuid	FK → users, NOT NULL	Foreign key referencing users.id.
content	text	NOT NULL	Rich-text HTML or JSON content body.
parent_id	uuid	FK → lesson_comments	UUID of the parent comment for threaded reply nesting, NULL for top-level comments.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamptz DEFAULT now()	NOT NULL	Timestamp of the last modification.

4.4.5 Assessment

Tables 4.29 to 4.33 define the 5 entities of the assessment domain.

24. Table: quizzes

Table 4.29: Quizzes Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
lesson_id	uuid	FK → lessons, UNIQUE, NOT NULL	Foreign key referencing lessons.id.
title	character varying	NOT NULL	Display title of the record.
time_limit_seconds	integer DEFAULT 0	NOT NULL	Maximum allowed quiz completion time in seconds, 0 meaning unlimited.
max_attempts	integer DEFAULT 0	NOT NULL	Maximum number of allowed attempts.
pass_threshold_pct	integer DEFAULT 60	NOT NULL	Minimum percentage required to pass.
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp DEFAULT now()	NOT NULL	Timestamp of the last modification.

25. Table: quiz_questions

Table 4.30: Quiz Questions Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
quiz_id	uuid	FK → quizzes, UNIQUE, NOT NULL	Foreign key referencing quizzes.id.
question_text	text	NOT NULL	Display text of the question.
question_type	character varying	NOT NULL	Question format (multiple-choice, etc.).
sequence_order	integer	UNIQUE, NOT NULL	Integer determining the display order of this question within the quiz.
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.
image_url	text	-	URL of an optional image asset displayed alongside the question text.

26. Table: quiz_options

Table 4.31: Quiz Options Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
question_id	uuid	FK → quiz_questions, UNIQUE, NOT NULL	Foreign key referencing quiz_questions.id.
option_text	text	NOT NULL	Display text of the quiz option.
is_correct	boolean DEFAULT false	NOT NULL	Whether this is the correct answer.
sequence_order	integer	UNIQUE, NOT NULL	Integer determining the display order of this answer option.
image_url	text	-	URL of an optional image asset displayed as this answer option.

27. Table: quiz_attempts

Table 4.32: Quiz Attempts Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
enrollment_id	uuid	FK → enrollments, UNIQUE, NOT NULL	Foreign key referencing enrollments.id.
quiz_id	uuid	FK → quizzes, UNIQUE, NOT NULL	Foreign key referencing quizzes.id.
attempt_number	integer	UNIQUE, NOT NULL	Sequential counter for quiz attempts.
score_pct	integer	-	Quiz score as a percentage (0-100).
result	character varying DEFAULT 'in_progress'	NOT NULL	Outcome of the quiz attempt ('in_progress', 'passed', 'failed').
started_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the quiz attempt was initiated.
submitted_at	timestamp	-	Timestamp of response submission.

28. Table: quiz_answers

Table 4.33: Quiz Answers Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
attempt_id	uuid	FK → quiz_attempts, UNIQUE, NOT NULL	Foreign key referencing quiz_attempts.id.
question_id	uuid	FK → quiz_questions, UNIQUE, NOT NULL	Foreign key referencing quiz_questions.id.
selected_option_id	uuid	FK → quiz_options	Foreign key referencing the chosen quiz option.
is_correct	boolean	-	Whether this is the correct answer.

4.4.6 Learner Progress, Recognition and Recommendation

Tables 4.34 to 4.41 define the 8 entities of the learner progress, recognition and recommendation domain.

29. Table: enrollments

Table 4.34: Enrollments Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, UNIQUE, NOT NULL	Foreign key referencing users.id.
course_id	uuid	FK → courses, UNIQUE, NOT NULL	Foreign key referencing courses.id.
enrolled_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the enrollment was created.
completed_at	timestamp	-	Timestamp when the record was completed.
status	character varying DEFAULT 'active'	NOT NULL	Record status (e.g., active, completed, dropped).

30. Table: lesson_progress

Table 4.35: Lesson Progress Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
enrollment_id	uuid	FK → enrollments, UNIQUE, NOT NULL	Foreign key referencing enrollments.id.
lesson_id	uuid	FK → lessons, UNIQUE, NOT NULL	Foreign key referencing lessons.id.
is_viewed	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether the learner has opened this lesson.
view_count	integer DEFAULT 0	NOT NULL	Total number of times the learner has viewed this lesson.
first_viewed_at	timestamp	-	Timestamp of the learner's first access to this lesson.
last_viewed_at	timestamp	-	Timestamp of the learner's most recent access to this lesson.
time_spent_learning	integer DEFAULT 0	NOT NULL	Cumulative time in seconds the learner has spent on this lesson.
summary_completed	boolean DEFAULT false	-	Boolean flag indicating whether the learner submitted a lesson summary.
is_completed	boolean DEFAULT false	-	Boolean flag indicating whether the learner has fully completed this lesson.
progress_meta	jsonb DEFAULT ''	NOT NULL	JSONB object storing supplementary progress metrics and adaptive engine data.

31. Table: learner_checkpoints

Table 4.36: Learner Checkpoints Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
enrollment_id	uuid	FK → enrollments, UNIQUE, NOT NULL	Foreign key referencing enrollments.id.
checkpoint_id	uuid	FK → lesson_checkpoints, UNIQUE	Foreign key referencing lesson_checkpoints.id.
completed	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether the learner has completed this checkpoint.
completed_at	timestamptz	-	Timestamp when the record was completed.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the record was created.
response_data	jsonb	-	JSONB payload of the learner's response.
lesson_id	uuid	FK → lessons	Foreign key referencing lessons.id.

32. Table: h5p_responses

Table 4.37: H5P Responses Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, NOT NULL	Foreign key referencing the learner who produced the response.
h5p_content_id	uuid	FK → h5p_contents, NOT NULL	Foreign key referencing the embedded object the response belongs to.
score	integer	-	Score reported by the object, where the object reports one.
max_score	integer	-	Maximum attainable score reported by the object.
completed	boolean DEFAULT false	NOT NULL	Whether the object reported the interaction as completed.
raw_statement	jsonb	-	The unmodified statement emitted by the object, retained so that the record can be re-interpreted without replaying the interaction.
created_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the response was recorded.

33. Table: recommendations

Table 4.38: Recommendations Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
enrollment_id	uuid	FK → enrollments, NOT NULL	Foreign key referencing enrollments.id.
recommended_lesson_id	uuid	FK → lessons, NOT NULL	UUID of the lesson recommended to the learner by the adaptive engine.
difficulty_tier	character varying	NOT NULL	Difficulty classification of the recommended lesson relative to learner performance.
trigger_reason	text	-	Explanation of why the adaptive engine generated this recommendation.
is_acknowledged	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether the learner has viewed this recommendation.
created_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the record was created.

34. Table: course_favorites

Table 4.39: Course Favorites Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, UNIQUE, NOT NULL	Foreign key referencing users.id.
course_id	uuid	FK → courses, UNIQUE, NOT NULL	Foreign key referencing courses.id.
created_at	timestamptz DEFAULT now()	-	Timestamp when the record was created.

35. Table: user_achievements

Table 4.40: User Achievements Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
user_id	uuid	FK → users, UNIQUE, NOT NULL	Foreign key referencing users.id.
achievement_id	uuid	FK → course_achievements, UNIQUE, NOT NULL	Foreign key referencing course_achievements.id.
course_id	uuid	FK → courses, NOT NULL	Foreign key referencing courses.id.
earned_at	timestamptz DEFAULT now()	NOT NULL	Timestamp when the achievement was earned.

36. Table: certificates

Table 4.41: Certificates Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	PRIMARY KEY	Unique identifier (UUID v4).
enrollment_id	uuid	FK → enrollments, UNIQUE, NOT NULL	Foreign key referencing enrollments.id.
reference_code	character varying	UNIQUE, NOT NULL	Unique alphanumeric identifier for external certificate verification.
pdf_url	character varying	-	URL to the generated PDF certificate file stored in cloud storage.
status	character varying DEFAULT 'issued'	NOT NULL	Record status (e.g., active, completed, dropped).
issued_at	timestamp DEFAULT now()	NOT NULL	Timestamp when the certificate was issued.
revoked_at	timestamp	-	Timestamp when the certificate was revoked, NULL if still valid.
revoke_reason	text	-	Free-text explanation for why the certificate was revoked.
course_id	uuid	FK → courses	Foreign key referencing courses.id.
user_id	uuid	FK → users	Foreign key referencing users.id.
learner_name	text	-	Full name of the learner as displayed on the certificate.
course_title	text	-	Title of the completed course as displayed on the certificate.
educator_name	text	-	Name of the course educator as displayed on the certificate.

continued on the next page

Table 4.41 continued from the previous page

Attribute	Type	Constraint	Description
institution_name	text DEFAULT 'ACESS Platform'	-	Name of the issuing institution displayed on the certificate.
completion_date	timestamptz	-	Timestamp when the learner completed all course requirements.
verification_url	text	-	Public URL for third-party verification of certificate authenticity.
skills_earned	text[] DEFAULT ''	-	Array of skill labels or competencies acquired through course completion.
course_duration_hours	numeric DEFAULT 0	-	Total duration of the course in hours as displayed on the certificate.
signed_token	text	-	Cryptographic signature token for tamper-proof certificate verification.
template_id	text DEFAULT 'default'	-	Identifier of the certificate template used for this certificate.
metadata	jsonb DEFAULT ''	-	JSONB field for extensible metadata.

4.5 Detailed Design

4.5.1 Software Design

ACESS is a component-based React application organised by feature domain, with three global context providers, three role-scoped data-access modules, a set of pure computation modules, and a small number of server-side API routes reserved for operations that must not run in the browser.

Provider components (global state)

- **AuthProvider.** Wraps the application root, initialises the Supabase browser client, and exposes the session, the user object and the sign-in, sign-up and sign-out operations through React context. It also drives the `SessionTimeout` component, which watches for pointer and keyboard activity, warns 30 seconds before a 15-minute inactivity limit is reached, and signs the user out if the warning is not answered.

- **AccessibilityProvider.** Loads the learner's stored preferences from `user_profiles.accessibility_prefs`, passes them through `resolveSettings()` to normalise contradictory or legacy values, and writes the resolved result onto the document root. Continuous values are written as CSS custom properties (`--user-font-size`, `--user-line-spacing`, `--user-word-spacing`). Categorical values are written as `data-*` attributes (`data-preset`, `data-bg-tint`, `data-layout-mode`, `data-structure-mode`, `data-animation-level`, `data-distraction-free`, and others). The stylesheet consumes these, so a settings change repaints without any component re-render, which is what makes the response feel immediate.

- **LanguageProvider.** Loads `preferred_language` and exposes a `t()` lookup over the locale dictionaries in `src/locales`, switching between English and Bahasa Melayu at runtime.

An implementation detail of the AccessibilityProvider is worth recording, because it caused a real defect. The provider distinguishes `base_preset` from `active_preset`: the latter changes to `custom` the moment a learner adjusts any single value, while the former records which preset the configuration descends from. Two parts of the system previously disagreed about which of these meant "the current preset", with the consequence that adjusting one unrelated setting silently discarded the preset's chunking and pacing behaviour while retaining its colours. Both now read `base_preset`.

Pure computation modules Four modules are deliberately free of input/output, which makes them independently testable and lets the same logic serve more than one surface:

- `accessibility-audit.ts`: the lesson auditor described in Section 4.3.6. Takes a plain object describing a lesson and a focus profile, and returns a score, the rule list with pass, fail or not-applicable status, and the failures grouped by the editor tab that owns them.

- `accessibility-resolver.ts`: normalises a settings object and reports conflicts between settings.
- `adaptive-engine.ts`: holds the preset definitions of Table 4.5, computes the difference between the current settings and a candidate preset (which is what Figure 4.19 renders), and derives lesson modes from the active preset.
- `accessibility-profiles.ts`: the educator-facing explanation of each profile, with its numeric thresholds read from the auditor's own constants so that the guidance can never state a different number from the one the engine enforces.

Data access layer Three role-scoped modules wrap all database access from the browser, each following the same pattern of resolving the current user, issuing a query under the anonymous key (and therefore under Row-Level Security), and throwing on error:

- `learner-api.ts`: 56 exported functions covering profile, catalogue, enrolment, lesson content and progress, quiz attempts, checkpoints, certificates, recommendations, favourites and accessibility settings.
- `educator-api.ts`: 73 exported functions covering course and lesson CRUD with versioning, quiz and activity authoring, asset upload, student progress, analytics and certificate issue.
- `admin-api.ts`: 34 exported functions covering user management, system courses, chapters, milestones and templates, application review and platform metrics.

Server-side routes reserved for privileged operations Most mutations are issued directly from the browser under RLS. Four categories of operation cannot be, and are implemented as server routes holding the service-role key:

- `POST /api/admin/users/[id]`: role and status changes. The route updates both `public.users` and the account's authentication metadata, because routing

reads the role from the token while policies read it from the table. Updating only one of the two was a real defect that locked newly promoted educators out of their own dashboard.

- `POST /api/admin/approve-instructor`: promotes an approved applicant, with the same dual update.
- `POST /api/recommendations/generate`: runs the recommendation engine, which must read across courses the requesting learner is not enrolled in and therefore cannot run under that learner's own policies.
- `POST /api/certificates/claim`: issues a certificate after the eligibility check, generates the reference code and verification URL, and guarantees code uniqueness.

The recommendation engine Algorithm 1 sets out the rule-based logic of the recommendation engine. The engine produces four kinds of recommendation, each with a human-readable reason recorded alongside it, so that a learner is told *why* something is being suggested rather than being handed an unexplained list.

Algorithm 1 generateRecommendations(*learner*)

Require: A learner identifier
Ensure: Rows in `recommendations`, replacing any previous set for this learner

$E \leftarrow$ enrolments of *learner* where status $\neq$ dropped
if $E = \emptyset$ **then**
    return
end if
$C \leftarrow$ courses of E with their lessons; $P \leftarrow$ completed lesson ids per enrolment
$F \leftarrow$ lowest failing quiz score per lesson, per enrolment
– Per-enrolment rules
for all $e \in E$ **do**
    for all lesson l with a failing score in $F[e]$ **do**
        emit (REVISION, l, "scored $F[e][l]$%, review and try again")
    end for
    $n \leftarrow$ first lesson of e in sequence order not in $P[e]$
    if n exists and $n \notin F[e]$ **then**
        emit (STANDARD, n, "continue: n is next")
    end if
    if $80 \leq progress(e) < 100$ **then**
        emit (ADVANCED, last lesson of e, "almost done, reinforce")
    end if
end for
– Interest profile: completion counts more than enrolment, favourites count too
$W \leftarrow \emptyset$ ▷ weighted tag, category and difficulty counts
for all $c \in C$ **do**
    add c to W with weight 3 if completed, else 1
end for
for all favourited course c **do**
    add c to W with weight 2
end for
– Cross-course rule
for all published course c the learner is not enrolled in **do**
    $s \leftarrow$ tag overlap + category overlap +5 if favourited +1 if difficulty matches
end for
emit (ADVANCED, first lesson of the two highest-scoring c, most specific reason)
– Replace atomically
delete previous recommendations for E; insert all emitted rows

Two design decisions in Algorithm 1 are worth noting. Completion is weighted three times as heavily as enrolment, and a favourited course five points above tag overlap, because finishing a course or deliberately saving one is a stronger statement of interest than merely starting one. And the near-completion rule is bounded strictly below 100%: without that bound it fired on finished courses, telling learners to review the last lesson of something they had already completed.

Server-side assessment Quiz grading is the one operation in the system where a client-side implementation would be indefensible, and it is implemented accordingly. The client calls a database function that runs with definer privileges and does the following:

- verifies the caller is authenticated
- verifies an active enrolment in the course owning the quiz
- computes the next attempt number and refuses it if the quiz's attempt limit is exhausted
- grades each answer against the stored correctness flag
- writes the attempt and its answers
- returns the score, the pass decision and the threshold used

The answer key is protected on the way in as well as on the way out. Learner code never reads `quiz_options` directly. It reads the `quiz_options_scoped` view, which returns the correctness flag as `NULL` unless a database function confirms that the reader is staff or has already submitted an attempt for that quiz. The client therefore cannot obtain the answers before submitting, even by querying the API directly.

Course completion is protected the same way. A database function derives whether a course is complete and awards whatever achievements the learner has genuinely earned. The guard trigger on `enrollments` rejects a client attempting to mark a course complete directly. Both grading and completion set a transaction-local flag that the guards recognise, which is how a definer function is permitted to write rows a normal session may not.

4.5.2 Physical Database Design

The database is PostgreSQL 17 hosted on Supabase, with an identical stack run locally under Docker for development. The complete schema (tables, view, functions,

triggers, policies, indexes and grants) is defined by four version-controlled migrations, so a working database can be rebuilt from the repository alone with a single command. That property is what makes the schema figures in this chapter verifiable rather than merely asserted.

Schema consolidation The physical schema in this report is not the one in the previous revision. The project's migration history had reached 44 files that could *not* rebuild the database: none of them contained a `CREATE TABLE` for `users`, `courses`, `lessons`, `enrollments`, `quizzes` or `certificates`, every one assuming those tables already existed. The core schema existed only in an out-of-band dump, so a clean rebuild produced a broken database, and the history had drifted from reality: one migration dropped a table that was still present in the live project.

The history was consolidated into a single verified baseline, and six tables that no code path referenced were removed. Table 4.42 records them, because the previous revision of this report documented all six as part of the design and they must not simply disappear without explanation.

Table 4.42: Tables Removed During Schema Consolidation

Table removed	Reason
`user_accessibility_preferences`	Superseded by `user_profiles.accessibility_prefs`, which is what the application actually reads and writes. The table had no reader and no writer.
`password_reset_tokens`	Recovery is handled by Supabase Auth's own token generation, and no custom token table participates.
`certificate_verifications`	Verification resolves a certificate by its reference code and records no event.
`learner_milestones`	No reader or writer. Learner-facing milestones are computed from `course_milestones` plus live progress.
`lesson_summaries`	Superseded design. The lesson summary activity persists into `learner_checkpoints.response_data`.
`certificate_templates`	Its only query filtered on a column the table did not have and could never match. Certificate layout is generated in code.

Eleven further columns were removed from surviving tables on the same basis, including `lessons.chapter_title` (a denormalised duplicate of the chapter's own title, able to disagree with it) and `lessons.template_id` (whose foreign key

pointed at the wrong table entirely). Nothing was dropped merely for lacking a front-end reference: every object was first verified unreferenced across application code, API routes, function bodies, policy expressions, index definitions, inbound foreign keys and views.

Representative table definitions The definitions below are taken verbatim from the baseline migration. Two details differ from what a reader might expect and are deliberate. `users.id` has *no* default: the row is provisioned by a trigger on the authentication schema and inherits that identifier, so the two can never diverge. `users.role` has no default either, so that a row can never be created without an explicit role decision.

```
CREATE TABLE public.users (
  id uuid NOT NULL,
  email character varying NOT NULL,
  role character varying NOT NULL,
  is_active boolean DEFAULT true NOT NULL,
  email_verified_at timestamp,
  last_login_at timestamp,
  created_at timestamp DEFAULT now() NOT NULL,
  updated_at timestamp DEFAULT now() NOT NULL,
  deleted_at timestamp,
  full_name text,
  instructor_application_status text,
  CONSTRAINT users_role_check
    CHECK (role IN ('learner','educator','admin'))
);
```

```
CREATE TABLE public.enrollments (
  id uuid DEFAULT gen_random_uuid() NOT NULL,
  user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  course_id uuid NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
  enrolled_at timestamp DEFAULT now() NOT NULL,
  completed_at timestamp,
  status character varying DEFAULT 'active' NOT NULL,
  CONSTRAINT enrollments_status_check
    CHECK (status IN ('active','completed','dropped')),
```

```
  UNIQUE (user_id, course_id)
);
```

Constraints The schema carries 33 `CHECK` constraints and 20 unique constraints. They fall into three groups. *Enumerations* bound a text column to a permitted vocabulary: course status, lesson layout, lesson type, checkpoint type, difficulty level, contact-message category, quiz question type. *Range constraints* bound a numeric column: a pass threshold and a milestone completion percentage to 0–100, an attempt number and a sequence order above zero, a view count at or above zero. *Consistency constraints* bind two columns together, and are the most interesting group. `lesson_progress` enforces that a lesson cannot be complete without also being viewed, and `certificates` enforces that a revoked row has a revocation timestamp while an issued row does not, so an inconsistent certificate state cannot be written at all, by any client.

Indexes 115 indexes are defined. Beyond the primary keys, they follow three patterns: every foreign key used in a read path is indexed (`idx_enrollments_user`, `idx_lesson_progress_enrollment`, `idx_lessons_course` and their peers), status and category columns used as list filters are indexed (`idx_courses_status`, `idx_certificates_status`, `idx_contact_messages_status`), and the catalogue's hottest query is served by a partial index:

```
CREATE INDEX idx_courses_published ON public.courses (id)
  WHERE deleted_at IS NULL AND status = 'published';
```

A partial index is used here because the public catalogue only ever asks for published, undeleted courses, and indexing only those rows keeps the index small as the number of drafts and archived courses grows.

Row-Level Security RLS is enabled on all 36 tables, carrying 88 policies, and the arrangement is verified by an automated probe rather than asserted. The policies express

four rules. A learner reaches their own data through the enrolment: the policy on `lesson_progress`, for example, tests whether the progress row's enrolment belongs to the caller. An educator reaches their own courses and the progress recorded inside them, but not another educator's unpublished work. Anonymous visitors reach published courses and nothing else. And the answer key is readable only after submission.

```
-- A learner reaches progress only through their own enrolment
CREATE POLICY "Users can view their own progress"
ON public.lesson_progress FOR SELECT USING (
  EXISTS (SELECT 1 FROM enrollments e
      WHERE e.id = lesson_progress.enrollment_id
       AND e.user_id = auth.uid()));

-- An educator may create a course only as its own owner
CREATE POLICY "Educators can insert courses"
ON public.courses FOR INSERT WITH CHECK (
  auth.uid() = created_by
  AND current_user_role() IN ('educator','admin'));

-- The answer key is gated on having already submitted
CREATE POLICY "Staff and post-attempt learners can view quiz options"
ON public.quiz_options FOR SELECT
  USING (may_see_quiz_answer_key(question_id));
```

Note that this check reads the stored role from `public.users.role` rather than the session token. This is the point made in Chapter 3: the route interceptor trusts the token for speed, and the database does not trust it at all.

Triggers Fifteen triggers are defined in the application schema, plus one on the authentication schema. They serve three purposes. *Provisioning*: a trigger creates the matching `users` and `user_profiles` rows when an account is created, pinning self-registered accounts to the learner role. Previously the role was taken from the signup request itself, which meant a caller could mint an administrator by asking for one. *Guards*: the three guard triggers described above. *Derivation and notification*: five notification triggers fire on enrolment, lesson progress, quiz attempt, lesson creation and

course publication, and the remaining triggers maintain `updated_at`. Placing notification generation in the database rather than in application code means a notification is raised whichever path caused the change.

Storage Three buckets are defined: `course-assets` for lesson media and thumbnails, `certificates` for educator-uploaded custom certificates, and `avatars` for profile pictures. All three are public-read because the application distributes public URLs rendered directly in the page. Write access is policy-controlled, and the avatar policy restricts each user to a folder named for their own identifier, so one learner cannot overwrite another's picture.

4.5.3 Systemic Implementation Details

In-video questions Video is embedded from an external host and controlled through its player API (Google LLC, 2025). A polling loop compares the player's current time against the timestamps stored for the lesson's video questions. On crossing one, playback is paused and the question is presented over the player. Because an embed can fail for reasons outside the platform's control (a blocked host, an unavailable video), an alternative completion path exists. Relying solely on the player's own "ended" event previously made a lesson permanently uncompletable when the embed would not play, which is a total barrier rather than an inconvenience.

Notification generation by trigger Notifications are generated inside the database. A trigger fires after a quiz attempt is written, for example, resolving the owning course's educator and inserting a notification row. The application never assembles these payloads, so the behaviour cannot diverge between the paths that write an attempt.

Session timeout `SessionTimeout` wraps authenticated routes and tracks pointer and keyboard activity. At 14 minutes 30 seconds of inactivity it presents a warning with a 30-second countdown. If unanswered, it signs the user out and returns to the login

page. The warning is not decoration: an unannounced timeout is a data-loss event, and WCAG 2.2.6 exists for exactly this case.

Text-to-speech Read-aloud uses the browser's own speech synthesis rather than a network service (Mozilla Developer Network, 2025), so that lesson text is not transmitted in order to be spoken and the feature carries no per-request cost. It never starts on its own. It stops when the learner navigates away, which was a defect found only by running the product rather than by reading it. And while it is speaking, clicking a later paragraph restarts speech from that paragraph.

4.6 Limitations of Design

The boundaries of this design are set out below. The accessibility-specific gaps are enumerated separately in Section 4.3.7 and are not repeated here.

- **No evaluation with the intended users.** The platform has not been tested by any learner with lived experience of dyslexia, ADHD or autism, and no assistive technology has been exercised against it. Everything in this chapter is verified against published guidance, against the code, and by operating the interface, none of it against the people it is designed for. This is the most consequential limitation of the work, and it bounds what can be claimed: the design demonstrably adapts, and it has not been shown to help.
- **No quantitative baseline exists.** Because no measurement was taken before the accessibility work began, no before-and-after comparison of learner outcomes is possible even retrospectively. The comparisons in Section 4.3 are demonstrations of behaviour, not evidence of effect.
- **The recommendation engine reads outcomes, not behaviour.** It operates on quiz results, completion state, favourites and course metadata. It does not use time-on-task, scroll behaviour, revisit patterns or any of the other telemetry the platform already collects, although `lesson_progress` and

`adaptive_interactions` hold enough to support it. It is rule-based by design (the reason for each recommendation is stated to the learner, which a learned model could not do as directly), but the rules are coarse.

- **Only one accessibility preset can be active at a time.** A learner needing dyslexia typography together with ADHD pacing must choose one and adjust the remainder by hand. Composable profiles are the clearest single extension to this design.
- **Video is not hosted by the platform.** Instructional video is embedded from an external host, so its availability, captioning quality and playback behaviour are outside the platform's control, and a lesson's video can become unavailable without the platform knowing.
- **Validation is not centralised.** Field rules are expressed in the form components and again as database constraints. The database layer is authoritative and cannot be bypassed, so this is not a security weakness, but the duplication means a rule can be tightened in one place and not the other. A single shared schema definition would remove the duplication.
- **Route gating trusts the session token.** The request interceptor reads the role from the token rather than the database, to avoid a database round trip on every request. Every authorisation decision that matters is made again by Row-Level Security against the authoritative table, so a forged token yields an interface shell and no data. The arrangement is a deliberate trade-off rather than a maximally strict design, and it is documented here rather than left for a reader to discover.
- **No load testing.** No concurrency or throughput figure is claimed anywhere in this report, because none has been measured.
- **Offline use is not supported.** All features require a network connection, and no offline caching or progressive-web-app behaviour is implemented. This is a genuine accessibility limitation as well as a functional one, since intermittent connectivity is itself a barrier for some learners.
- **Localisation is partial.** The interface is available in English and Bahasa Melayu, but the coverage is incomplete: the accessibility settings panel's own field labels

are still English-only, which is a poor outcome for precisely the panel a learner reading in a second language is most likely to need. Course *content* is authored in one language and is not translated by the platform.

4.7 Summary

This chapter translated the requirements of Chapter 3 into design. The high-level design presented the behavioural and interaction models of the three roles, a user interface design covering navigation, input validation and output classification, and the conceptual and logical data model as three domain Entity Relationship Diagrams generated from the live schema.

Section 4.3 set out the accessibility design that is this project's substantive contribution:

- a mapping of eighteen accessibility criteria to the specific feature, learner profile and module that addresses each
- the three learner profiles and the concrete support provided for each
- the exact values written by each of the three presets
- screenshot evidence of the same lesson rendered under each preset
- the deterministic authoring-time auditor that carries the content criteria a platform cannot satisfy on an educator's behalf

It also stated, in the same detail, eight known gaps, foremost among them that no assistive technology and no disabled learner has yet tested the platform.

The data dictionary documented 36 tables and 364 columns as they currently exist, together with the normalisation position and the two deliberate departures from it. The detailed design covered the component architecture, the pure computation modules, the division between browser-issued and privileged server-issued operations, the rule-based

recommendation algorithm, and the physical database implementation (constraints, indexes, 88 row-level security policies, triggers and storage), including the schema consolidation that removed six tables the previous revision of this report documented as part of the design.

The next chapter describes how this design was implemented: the development and deployment environment, the configuration and version-control procedure, and the implementation status of each module.

CHAPTER 5: IMPLEMENTATION

5.1 Introduction

This chapter describes how the design of Chapter 4 was built. It covers the hardware and software environment the system was developed and deployed in, the configuration management and version control procedure used to keep that environment reproducible, and the implementation status of every module.

The expected output of the implementation phase is a running system whose behaviour can be verified independently of the developer's own machine. That framing shaped two decisions recorded in this chapter. First, the database is defined entirely by version-controlled migrations and populated by a deterministic seed script, so a complete working instance (schema, policies, functions, triggers and a realistic dataset) can be reconstructed from the repository by a single command. Second, the properties that matter most and are easiest to break silently, namely the row-level security rules and the learner-facing data paths, are checked by executable probes rather than by inspection.

Section 5.2 describes the environment. Section 5.3 describes configuration management and version control. Section 5.4 reports what is implemented, what is implemented with known limitations, and what is not implemented at all.

5.2 Development Environment Setup

5.2.1 Hardware Environment

Development was carried out on a single workstation. The specification is stated in Table 5.1 because one figure in it is load-bearing rather than incidental: the machine has 8 GB installed, of which 7.6 GB is usable, and the local Supabase stack runs eleven containers alongside the Next.js development server. This constrained how the project was worked on: the database stack and the application server compete for memory. That is the reason the local stack is started and stopped deliberately rather than left running.

Table 5.1: Development Workstation Specification

Component	Specification
Processor	AMD Ryzen 5 9600X, 6 cores / 12 threads
Memory	8 GB installed (7.6 GB usable)
Storage	1 TB NVMe solid-state drive
Operating system	Windows 11 Pro (build 10.0.26200)
Display	1920×1080, used for responsive and preset verification
Network	Broadband connection, required for the hosted Supabase project, Vercel deployment and the external video and model services

No dedicated server hardware was provisioned. Production runs on Vercel's serverless platform for the application and on Supabase's managed infrastructure for the database, authentication and object storage. Development runs the same backend stack locally in Docker containers.

5.2.2 Software Environment

Table 5.2 lists the software environment with the versions actually in use, rather than a generic list of technologies. Versions are given because several decisions in Chapter 4 depend on them: the request-interception entry point is named `proxy` rather than `middleware` because of the Next.js major version, and the schema figures depend on PostgreSQL 17.

Table 5.2: Software Environment and Versions

Software	Version	Role in the project
Node.js	24.15.0	JavaScript runtime for the development server, build and tooling
npm	11.12.1	Dependency and script management
TypeScript	5.9.3	Static typing across the whole codebase
Next.js	16.2.3	React framework: routing, server rendering, request interception
React	18.2	User interface library
Tailwind CSS	4.x	Styling, and the CSS custom properties the accessibility engine writes are consumed here
Supabase CLI	2.116.0	Local stack, migrations, database reset, type generation
Docker	29.5.3	Container runtime hosting the local Supabase stack
PostgreSQL	17.6	Database (local container and hosted project)
Tiptap	3.24	Rich-text lesson editor
Recharts	3.8	Analytics visualisations
jsPDF	4.2	Certificate and administrator report generation
dnd kit	6.3	Drag-and-drop with keyboard sensor for interactive activities
DOMPurify	3.18	Sanitisation of authored lesson HTML
Visual Studio Code	N/A	Primary editor, with ESLint and Tailwind extensions
Git	N/A	Version control
Vercel	N/A	Production deployment, built from the GitHub repository

The codebase comprises 335 TypeScript and TSX source files totalling approximately 80,000 lines, organised as described in Section 5.3.

5.2.3 Local backend environment

A design decision worth recording is that development does *not* run against the hosted database. The Supabase CLI starts a complete local stack in Docker (PostgreSQL, the authentication service, PostgREST, object storage, the realtime service and the Studio console), and the application is pointed at it through environment variables, with the hosted configuration retained in a separate file so the two can be swapped.

This matters for one specific reason. Verifying row-level security honestly requires attempting operations that *should* fail: signing in as one learner and trying to read another's progress, attempting to promote an account, attempting to write a quiz score directly. Those probes are destructive in intent and cannot responsibly be run against a database holding anyone's real data. Running them against a disposable local instance that can be rebuilt in a minute is what made the security verification in Section 5.4 possible at all.

5.3 Software Configuration Management

5.3.1 Configuration environment setup

Configuration in this project falls into three categories, each handled differently.

Environment configuration. Runtime configuration (the database URL, the anonymous and service-role keys, the mail transport credentials, the model API key and the public application URL) is supplied through environment variables and never committed. Two files exist: the active configuration and a saved copy of the hosted-project configuration, both excluded from version control, so switching between the local and hosted backend is a file swap rather than an edit. The distinction between the two Supabase keys is enforced structurally rather than by discipline: the anonymous key is exposed to the browser and is subject to row-level security, while the service-role key is read only inside server-side API routes and would bypass those policies, which is

precisely why the four operations listed in Section 4.5.2 are the only ones permitted to use it.

Database configuration. The schema is not configured by hand at any point. It is defined by four ordered migrations under `supabase/migrations/`:

- a consolidated baseline creating every table, view, function, trigger, policy, index and grant
- a storage migration creating the three buckets and their policies
- a migration enabling row-level security on two tables that had been left open
- a migration repairing account provisioning and pinning self-registration to the learner role

Applying them to an empty database produces the exact schema documented in Chapter 4.

This replaced a migration history that had become unusable. The project had accumulated 44 migration files, none of which contained a `CREATE TABLE` for any core entity: every one assumed the tables already existed and only altered them. As a result, a clean rebuild produced a broken database, and the files had drifted from the live schema. The consolidation was performed by replaying the newest schema dump plus the eleven migrations that post-dated it and then verifying the result column for column against the live project before any cleanup was applied. All 44 original files are retained unmodified in an archive directory rather than deleted, so the project's history remains inspectable.

Demonstration data. A seed script populates a rebuilt database with a deterministic dataset: twelve accounts across the three roles, eleven courses in four lifecycle states, thirty lessons, five quizzes, ten interactive activities, twenty-one enrolments, nineteen quiz attempts, seven certificates and the supporting progress and notification history. It

uses a fixed pseudorandom seed, so repeated runs reproduce the same data, and it enforces twenty-one referential-integrity checks before reporting success.

Two properties of the seed are worth stating because they affect the credibility of the figures in this report. First, chronology is generated forwards (enrolment, then first lesson view, then completion, then quiz attempt, then course completion, then achievement, then certificate), and the checks reject any record that violates that order. Second, lesson accessibility scores are *not* invented: every seeded lesson is passed through the same auditor the educator's compliance checker uses, and the score it returns is what is stored. The weakest lessons in the dataset fail for real, diagnosable reasons, which is why Figure 4.24 can show genuine findings rather than a constructed example.

The commands are:

```
supabase start                                  # start the local stack
npm run db:rebuild                  # reset from migrations, then seed
npm run verify:data          # run the RLS and learner-scenario probes
npm run dev                                     # start the application
```

Code quality configuration. TypeScript runs in strict mode and ESLint uses the framework's recommended configuration together with accessibility-specific rules. A pull-request template in the repository requires an author to state, for any change touching an accessibility setting, what the setting does, which standard justifies it, and whether the claim in its description matches its behaviour. This is a procedural control rather than an automated one, and its limitation is recorded in Section 5.4: nothing currently blocks a pull request that ignores it.

5.3.2 Version control procedure

The project is held in a Git repository hosted on GitHub (`aliff1024/ACESS`), with 56 commits between 14 April and 26 August 2026. Vercel is connected to the repository and builds the production deployment from the `main` branch, so a merge to `main` is a release.

The working procedure was as follows.

1. **Commit granularity follows the change, not the clock.** A commit corresponds to one coherent change (a feature, a fix, or an audit pass), and its message names the change rather than the date. Early in the project this discipline was not observed, and the history contains a run of commits titled only by date. Those messages are of no use in locating when a behaviour changed, and the practice was abandoned.

2. **A destructive change is preceded by a checkpoint commit.** Before the database consolidation, a commit was made specifically to capture the pre-consolidation state, so that the 44 replaced migrations and the schema they produced remain recoverable independently of the archive directory.

3. **Generated artefacts are committed, their inputs are authoritative.** The Entity Relationship Diagrams and the data dictionary in Chapter 4 are produced by scripts in the repository from the live schema. The generated files are committed so the report compiles without a database, but the generator is the source of truth, and regenerating is the correct response to a schema change. Editing the generated figure is not.

4. **Secrets are never committed.** Environment files are excluded by `.gitignore`. Keys are held in the local environment and in the deployment platform's own configuration.

Two limitations of this procedure should be recorded rather than glossed over. The project was developed by a single author, so branching was minimal and most work landed directly on `main`. A multi-author project would require a branch-per-change and review procedure that this one did not exercise. And there is no continuous integration pipeline: the type checker, linter and verification probes are run manually and are not enforced by the repository on push. This is the single most significant gap in the configuration management of the project, and it is listed as such in Table 5.3.

5.4 Implementation Status

Table 5.3 reports the status of every module. Three status values are used, and the distinction between them is deliberate. **Completed** means the module performs its specified function and has been exercised in a running system. **Completed with known limitations** means the module performs its specified function, with a specific, documented shortfall stated in the notes, not a general caveat. **Not implemented** means the item was planned and does not exist.

Table 5.3: Implementation Status of Each Module

Module	Duration	Completed	Status	Notes
Project scaffolding and deployment pipeline	2 weeks	14/04/2026	Completed	Next.js project, Vercel connection, initial Supabase project.
Authentication and role-based access	3 weeks	08/05/2026	Completed	Provisioning trigger, route gating, session timeout. Self-registration pinned to the learner role after an escalation path was found and closed.
Course and lesson authoring	3 weeks	11/05/2026	Completed	Course, chapter and lesson CRUD, rich-text editor, media upload, version snapshots.
Learner course experience	3 weeks	11/05/2026	Completed	Catalogue, enrolment, lesson viewer, roadmap, favourites.
Assessment and grading	2 weeks	15/06/2026	Completed	Server-side grading, answer key withheld until submission, attempt limits enforced in the database.
Progress and telemetry	2 weeks	15/06/2026	Completed	View, completion, time-on-lesson and checkpoint state, consistency enforced by constraint.
Notifications	1 week	10/05/2026	Completed	Five database triggers, generated regardless of the client path that caused the event.
Certificates and achievements	2 weeks	11/05/2026	Completed	Eligibility check, reference code, public verification page, PDF generation, educator issue and revoke.
Recommendation engine	2 weeks	17/06/2026	Completed	Four rule tiers with a stated reason per recommendation, reads outcomes not behaviour (Section 4.6).
Educator analytics	2 weeks	17/06/2026	Completed	Course and student views, at-risk flagging.
Administrator console and reports	3 weeks	18/06/2026	Completed	User and course management, application review, PDF reports, platform analytics including accessibility adoption.
Email delivery	1 week	25/06/2026	Completed	Password recovery and notification mail.

continued on the next page

Table 5.3 continued from the previous page

Module	Duration	Completed	Status	Notes
Interactive activities	2 weeks	26/06/2026	Completed with known limitations	Five activity types with builders and viewers. Keyboard operability added for all five. The drag-and-drop "diagram" mode still has no non-drag alternative.
H5P embedding	1 week	26/06/2026	Completed	Embedded objects and response capture.
AI lesson assistant	1 week	24/08/2026	Completed	Cached plain-language summaries and lesson-scoped question answering. Optional, and the platform degrades gracefully without an API key. No assessment is model-graded.
Accessibility personalisation engine	3 weeks	25/08/2026	Completed with known limitations	Three presets and 24 catalogued settings applied without reload. Two settings do not fully match their labels (Section 4.3.7), and only one preset can be active at a time.
Accessibility compliance auditor	2 weeks	25/08/2026	Completed	27 profile-aware rules with cited sources, live in the editor and at course level.
Localisation (English / Bahasa Melayu)	1 week	25/08/2026	Completed with known limitations	Runtime switching, persisted to the profile. The accessibility settings panel's own field labels are not yet translated.
Security hardening and audit	2 weeks	25/08/2026	Completed	Privilege-escalation paths closed, completion derived server-side, guard triggers added, eight accessibility defects found by live testing and fixed.
Database consolidation and seed	1 week	26/08/2026	Completed	Rebuildable schema in four migrations, six unused tables removed, deterministic seed with 21 integrity checks.
Published accessibility statement	N/A	25/08/2026	Completed	Public page stating supported features and known gaps honestly.
Continuous integration pipeline	N/A	N/A	**Not implemented**	Type checking, linting and the verification probes are run manually, and nothing enforces them on push.
Automated accessibility testing	N/A	N/A	**Not implemented**	No automated accessibility scanner and no assistive-technology test suite exists (Section 4.3.7).
Composable accessibility profiles	N/A	N/A	**Not implemented**	Combining supports across presets is not possible, identified as the clearest extension.
Offline support	N/A	N/A	**Not implemented**	No offline caching or progressive-web-app behaviour.

5.4.1 Verification performed during implementation

Five executable checks were built alongside the system and are run against a rebuilt local database. They are reported here as part of implementation rather than testing, because their purpose is to confirm that what was built matches what was designed, not to judge whether the design itself is right. Systematic testing, including a reference to these same checks by name for the security test class, is the subject of Chapter 6. Table 5.4 lists them.

Table 5.4: Executable Verification Scripts

Check	Assertions	What it confirms
`npm run verify:rls`	20	That a learner cannot read another learner's progress, promote themselves, enrol someone else, forge a quiz score, or read an answer key before submitting, and that anonymous visitors see published courses only.
`npm run verify: scenarios`	28	That the learner-facing data paths return what the interface expects for each seeded learner scenario.
`npm run seed`	21	Referential integrity and chronological consistency of the demonstration dataset before it is accepted.
`npm run test: a11y-resolver`	18	That the settings resolver normalises contradictory and legacy accessibility settings correctly.
`npm run test: a11y-catalog`	234	That every field in the accessibility settings type is either catalogued with a description and a justification, or explicitly listed as excluded with a reason.

The last of these enforces a design rule rather than a behaviour: no accessibility setting may exist outside the catalogue, so adding a field to the settings type without describing it or explicitly excluding it fails the check. This exists because the most damaging accessibility defects found during this project were not crashes but *inert controls*, settings that appeared to work and did nothing. A control that cannot be added without being described is harder to leave inert.

5.5 Conclusion

This chapter described the environment in which ACESS was built and deployed, the configuration and version control procedure that keeps that environment

reproducible, and the implementation status of each module. Twenty-one modules are implemented, three of them with specific documented limitations, and four planned items are not implemented at all, the most consequential being the absence of a continuous integration pipeline and of any automated accessibility testing.

The substantive outcome is that the system is reconstructible rather than merely running: the schema is defined by migrations, and the demonstration data by a deterministic seed, so the claims made in Chapter 4 can be re-verified by a third party rather than taken on trust. Whether the system behaves correctly once running is the subject of Chapter 6.

CHAPTER 6: TESTING

6.1 Introduction

This chapter describes how ACESS was tested once implemented. Testing is organised into three classes, kept clearly separate because they carry different kinds of evidence and were carried out by different actors:

- **Functional testing**, split into an automated end-to-end suite that is the primary evidence for functional correctness, driving a real browser against the real running application, plus a small manual spot-check of tasks that automation cannot judge, executed by the project author.
- **WCAG 2.2 expert review**, a findings table covering every criterion in Table 4.3, with the objective, code- and browser-verifiable findings filled in and the subjective judgement calls left for the project author to complete as the signing reviewer.
- **User acceptance testing**, using six people role-playing personas in place of the target users this project could not recruit (Section 4.6).

Security testing is not repeated as a fourth class here: it was already covered by the `npm run verify:rls` and `npm run verify:scenarios` scripts described in Section 5.4.1, and this chapter references those results rather than re-running them.

Where a result in this chapter has not yet been supplied by the project author, it is marked **TODO** rather than filled in with a plausible-looking placeholder.

6.2 Test Plan

6.2.1 Test Organization

Table 6.1: Test Organization

Role	Responsibility
Muhd Aliff (project author)	Wrote and executed the automated end-to-end suite, wrote the objective, code- and browser-verifiable rows of the WCAG findings table, and is tester for the manual spot-check matrix, expert reviewer signing off the WCAG findings table's subjective rows, and UAT coordinator: briefed and distributed the persona tasks and form, and collects and interprets the responses.
Project Supervisor	Reviewed test evidence and scope during supervision meetings, as for the rest of the project.

No dedicated external test users were recruited for functional or accessibility testing. The seeded demonstration accounts described in Section 5.3 stand in for real learners, educators and administrators there. User acceptance testing (Class 3, below) is the one place real people were involved, and they are proxies role-playing a persona rather than the neurodivergent learners this platform targets, which is recorded as a limitation in Section 7.5 rather than presented as equivalent to it.

6.2.2 Test Environment

The automated suite drives a real Chromium browser against the deployed application, connected to the project's live hosted Supabase project (the same one the seed data and demo accounts in Section 5.3 describe), so every test exercises the actual rendered interface rather than only the underlying data layer. Ten seeded personas across the four roles served as test accounts, each already carrying a realistic history (part-finished courses, quiz attempts, issued certificates) so that tests exercise real, varied data rather than an empty account. The manual spot-check matrix and the WCAG expert review are likewise carried out by the project author in a real browser against the deployed application at `https://acess-tau.vercel.app`.

Every write the automated suite makes to this shared seed data is undone at the end of the run (an enrolment dropped, a lesson-progress or accessibility-preference row restored to its prior state, a quiz attempt left unsubmitted, a test course deleted, an approved instructor application and the account it provisions both reverted), so the suite can be re-run repeatedly without accumulating side effects. One of these reversals needed a second attempt to get right: approving the instructor application was found, only by reading the database afterwards, to provision a real educator account linked back to the application by a foreign key that cascades on delete, so the first teardown attempt deleted the application record along with the account it was undoing. The corrected teardown clears that link before removing the account.

6.2.3 Test Schedule

Table 6.2: Test Schedule

Cycle	Timing	Method
Unit-level verification	Continuous, Sprints 1–6 (14 April – 26 August 2026)	The five executable scripts in Table 5.4 (Section 5.4.1), run by hand after changes to the affected area.
Automated end-to-end suite	2 September 2026	The nine browser-driven scenarios in Table 6.3, run against the deployed application.
Manual spot-check	TODO	Ten task scenarios in Table 6.4, under an hour of the project author's time.
WCAG 2.2 expert review	TODO	The project author completes the subjective rows of Table 6.5.
User acceptance testing	TODO	Six persona-based sessions distributed and collected by the project author.

6.3 Test Strategy

6.3.1 Test classes

- **Functional correctness, automated.** Does a real account, exercising a real feature by clicking through the real rendered interface, produce the correct on-screen and database result. This is the primary evidence for functional correctness, and its results in this chapter are real and complete.

- **Functional correctness, manual spot-check.** A small set of tasks where the correct outcome is a matter of human judgement (does a reading experience actually feel more comfortable, does a calmer view actually feel calmer) rather than a checkable assertion, and which automation therefore cannot score.
- **Security and data isolation.** Whether one account can read or write another account's data. Not repeated here: see Section 5.4.1 for the `verify:rls` and `verify:scenarios` results.
- **Accessibility conformance, expert review.** Whether each WCAG 2.2 criterion ACESS targets is actually met, reviewed criterion by criterion against Table 4.3's claims rather than accepted on the strength of that table alone.
- **User acceptance.** Whether a person, briefed as a specific persona and working through that persona's real tasks, can complete them, find their way around, and feel the accessibility support was adequate for that persona.
- **Stress and load.** Not conducted. ACESS was never subjected to concurrent-user or volume testing, which remains a limitation (Section 7.5).

6.4 Test Design

6.4.1 Automated end-to-end suite

The suite lives in `tests/e2e/`, one specification file per scenario, configured through `playwright.config.ts` and run with `npx playwright test tests/e2e/` (aliased as `npm run test:e2e`). Each test signs in as a real seeded persona through the real login form, then drives the same screens, clicks and form fields a person would use, asserting on what actually renders and, where a database check is the only way to confirm a write truly happened (for example, that an approval changed a stored status rather than only the on-screen label), cross-checking the result against the database directly. Table 6.3 sets out the test description for each of the nine scenarios: what is being tested, the concrete steps the

automated script performs, and the result that would confirm it passed. Table 6.9 in Section 6.5 reports what actually happened when it ran.

Table 6.3: Automated Suite Test Descriptions

ID	Role/Profile	Task Description	Steps	Expected Result
PUBLIC-01	Anonymous	Confirm the landing page is reachable with no session, and confirm what unauthenticated Public navigation actually reaches.	1) Open the landing page with no session. 2) Confirm it renders. 3) Navigate directly to `/courses`.	Landing page renders. `/courses` either renders a public catalogue or redirects to login, confirming which.
LEARNER-01	Learner (baseline, no preset)	Enrol in a course not already enrolled in, through the real catalogue UI.	1) Sign in as haziq. 2) Open the detail page of a course confirmed unenrolled beforehand. 3) Click Enroll in Course.	The Enroll in Course button disappears and an active enrolment exists for that learner and course in the database.
LEARNER-02	Learner (ADHD preset)	Apply the ADHD preset through the real Accessibility settings modal, then complete a lesson confirmed incomplete beforehand.	1) Sign in as amir. 2) Open Accessibility Settings, select ADHD, confirm the preset-diff dialog, Apply, Save. 3) Open the target lesson. 4) Look for a clickable lesson-completion control among every button, role=button and link on the page.	A lesson-completion control equivalent to the baseline lesson page's "Complete Lesson" button is present and clickable.
LEARNER-03	Learner (Dyslexia preset)	Take a quiz attempt through the real quiz UI.	1) Sign in as mei (whose profile already carries the Dyslexia preset and `preferred_reading_level='simplified'`). 2) Open the target lesson's quiz and start an attempt. 3) Answer question 1.	The quiz UI continues to show quiz content (the next question or feedback on this one) after answering, rather than going blank.
LEARNER-04	Learner	View an already-issued certificate.	1) Sign in as amir. 2) Navigate to the Certificates page.	At least one certificate renders with a reference code, issuing instructor, completion date and a working verification link.

continued on the next page

Table 6.3 continued from the previous page

ID	Role/Profile	Task Description	Steps	Expected Result
EDUCATOR-01 & 02	Educator	Create a course through the real four-step course wizard, then edit it through the real course-workspace Settings tab.	1) Sign in as farah. 2) Fill in title and description, Continue. 3) Add a tag, Continue. 4) Skip the Lessons step, Continue. 5) Create Course. 6) Confirm the course appears in the real course list. 7) Open its Edit action. 8) On the Settings tab, set Primary Accessibility Focus to ADHD and Save Focus.	The course exists in the course list after creation, and `primary_disability_focus` is `adhd` in the database after saving.
EDUCATOR-03	Educator	View the real course analytics dashboard.	1) Sign in as siti (five published courses). 2) Open the Analytics page.	Per-course analytics (enrolled, active, at-risk, average progress, completion rate, average quiz score) render with real figures.
ADMIN-01	Administrator	Approve a real pending instructor application through the real Educator Applications UI.	1) Sign in as an administrator. 2) Open Educator Applications and select the pending application from “Tan Chee Meng”. 3) Click Approve. 4) Confirm the resulting dialog (“Confirm Approval . . . this will activate the educator account”).	The application’s status is `approved` in the database after confirming.

6.4.2 Manual spot-check matrix

Table 6.4 lists ten tasks that need a person’s judgement rather than an assertion, tagged to the learner profile each exercises. Actual Result and Status are left blank for the project author to complete. The tasks were scaffolded from real, already-implemented features described in Chapter 4, not performed or scored here.

Table 6.4: Manual Spot-Check Matrix

ID	Profile	Task & judgement question	Actual Result	Status
MSC-01	Dyslexia	Open a lesson with the Dyslexia preset active and read a full paragraph. Does the OpenDyslexic typeface, cream background and 1.6 line spacing actually read more comfortably than the same paragraph at default settings, or does it just look different?		
MSC-02	Dyslexia	Enable the reading spotlight on a lesson and read through it. Does the spotlight genuinely help you keep your place, or is it distracting?		
MSC-03	Dyslexia	Use the Listen (text-to-speech) control on a lesson with the Dyslexia preset's 0.9x rate. Is the pacing and voice actually easier to follow than the default rate, or no different?		
MSC-04	ADHD	Enter a lesson with the ADHD preset active (distraction-free mode on). Does the reduced-chrome view actually feel calmer than the standard lesson view, or does removing the sidebar and navigation feel disorienting instead?		
MSC-05	ADHD	Open a chunked lesson with the ADHD preset's task checklist enabled. At every point, is it clear what to do next?		
MSC-06	Autism	Before starting a lesson with the Autism preset active, look at the visual schedule. Is it clear what to expect before you begin, or does it raise more questions than it answers?		
MSC-07	Autism	Navigate between two different lessons with the Autism preset active. Does the layout and structure feel consistent and predictable from one lesson to the next?		
MSC-08	Autism	With the Autism preset active (`animation_level=none`), move through several pages of the platform. Is the experience genuinely free of distracting motion, or do any animations or transitions still slip through?		
MSC-09	Baseline (no preset)	Start a timed quiz with no accessibility preset active and let the timer run out without submitting. Table 6.5 flags that this auto-submits with no warning for a baseline learner. Confirm this live, and judge whether that behaviour is reasonable or feels punitive.		
MSC-10	Baseline (no preset)	Open the preset preview dialog for any preset without applying it. Does the "before you apply" preview accurately describe what will actually change on screen once applied?		

6.4.3 WCAG 2.2 expert review

Table 6.5 walks every criterion in Table 4.3 once more, this time as a review rather than an implementation claim. Findings and notes that could be verified objectively, from the live codebase, the live published content, or the automated suite's own browser session, are filled in below, each stating exactly what was checked. Rows marked TODO need the project author's own judgement, generally because they need a screen reader or a subjective read that neither static inspection nor a scripted browser session can supply. Those are cross-referenced to the relevant item in Table 6.4 where one exists. This table does not restate the gaps already recorded in Section 4.6 or the defects already recorded in Section 5.4: it cross-references them instead.

Table 6.5: WCAG 2.2 Expert Review Findings

Criterion	Finding	Note
1.1.1 Non-text Content (A)	Partial	Rule confirmed in `accessibility-audit.ts`. Live scan of all 29 published lessons: 3 images total, 0 missing the `alt` attribute, 1 with `alt=""` (empty). Confirm whether that image is genuinely decorative: if not, this is a live content gap rather than a code defect. TODO: reviewer judgement.
1.2.1 Video-only (A)	Fail	The auditor's own rule requires a transcript on a lesson with video (confirmed in code). Live scan: 2 of 7 published lessons with video have no transcript, contradicting that rule. Check whether the auditor runs only at save time and these predate it, or whether publishing can bypass it. TODO: reviewer judgement.
1.2.2 Captions (A)	TODO	Requires viewing a captioned video live, not verifiable from source alone.
1.3.1 Info and Relationships (A)	Pass	Rule confirmed in `accessibility-audit.ts`. Live scan: 0 of the published lessons over 200 words lack a heading.
1.4.4 Resize Text (AA)	TODO	Range is documented as 12–24 px. Confirm live that the control is actually keyboard-operable and the resulting text remains legible at both ends of the range.
1.4.5 Images of Text (AA)	TODO	Distinguishing genuine text from an image of text needs visual inspection of authored content.
1.4.8 Visual Presentation (AAA)	TODO	Reading-measure values are documented in Table 4.5. Confirm the bound is actually respected on a live rendered lesson.
1.4.12 Text Spacing (AA)	TODO	Confirm live that spacing remains legible and no content is clipped at the extremes of the learner-set range.

continued on the next page

Table 6.5 continued from the previous page

Criterion	Finding	Note
2.2.1 Timing Adjustable (A)	**Fail**	**Contradicts Table 4.3's own claim.** `QuizPage.tsx` shows the timer reaching zero calls `handleAutoSubmit()` directly whenever no accessibility preset is active, submitting whatever is answered with no warning and no extension. Only when a preset *is* active does the code show an Extend/Submit-now prompt instead. A baseline (no-preset) learner's quiz is auto-submitted on timeout. This needs your decision on whether Table 4.3's claim should be corrected.
2.2.4 Interruptions (AAA)	Partial	Confirmed live in a real browser (automated suite, LEARNER-02): distraction-free mode does remove navigation chrome during a lesson as intended, but it also removes the sidebar across the *entire* application once the ADHD preset is active, not only inside a lesson as Table 4.3 states, which left no way back to Accessibility Settings without resetting the account directly. TODO: reviewer judgement on whether this is acceptable.
2.2.6 Timeouts (AAA)	Pass	`SessionTimeout.tsx` confirmed: 15-minute inactivity timeout (`INACTIVITY_TIMEOUT_MS = 15 * 60 * 1000`), 30-second warning (`WARNING_BEFORE_MS = 30 * 1000`), matching the documented behaviour exactly.
2.3.3 Animation (AAA)	TODO	Confirming every animation/transition in the codebase is actually gated by the animation-level setting would need a full component-by-component sweep, not done here. Spot-check live under `animation_level=none`.
2.4.4 Link Purpose (A)	Partial	Auditor rule confirmed for authored lesson content. One hardcoded, non-descriptive link exists in the platform's own interface, outside anything the auditor scores: `TimelineViewer.tsx` renders a bare "Read more" control. Confirm live whether it has an accessible name from surrounding context.
2.4.6 Headings and Labels (AA)	Pass	Same scan as 1.3.1: 0 published lessons over 200 words lack a heading.
3.1.5 Reading Level (AAA)	Partial	Flesch–Kincaid computation confirmed present in `accessibility-audit.ts`. Separately, the automated suite found that the quiz component a learner with `preferred_reading_level='simplified'` is routed into (mei's Dyslexia persona) shows raw, untranslated interface strings (`quiz.tryAgain`, `quiz.reviewLesson`, `quiz.questionTooltip`) instead of plain-language text on a wrong answer, the opposite of what a simplified-reading learner needs. TODO: reviewer judgement on whether a specific lesson's reported grade level matches your own read of its difficulty.
3.2.3 Consistent Navigation (AA)	TODO	Requires comparing the rendered shell across multiple pages and roles live.
3.2.5 Change on Request (AAA)	Partial	`PresetDetailsDialog.tsx` confirmed to render a before/after diff prior to applying a preset. Several embedded video/H5P `iframe` elements (`H5PViewer.tsx`, `LessonRenderer.tsx`, `AdminCourseDetail.tsx`, `CourseAssets.tsx`, `LessonEditor.tsx`) include `autoplay` in their permissions-policy `allow` attribute. None of the checked embed URLs pass an `autoplay` query parameter, so nothing appears to auto-start from the platform's side, but confirm this live since the permission is granted.

continued on the next page

Table 6.5 continued from the previous page

Criterion	Finding	Note
4.1.2 Name, Role, Value (A)	Partial	`Sidebar.tsx` confirmed to use real `<Link>` elements with `aria-current`, and icon-only controls carry `aria-label`/`aria-pressed` (this was found broken and fixed during earlier testing). Modal dialogs use Radix UI's `Dialog` primitive, which provides a maintained, independently tested focus-trap and Escape-to-close implementation rather than a custom one. Separately, the automated suite found that a lesson has no lesson-completion control of any role under the ADHD preset at all (Table 6.9, LEARNER-02), not a naming or role defect on an existing control but the control's total absence, which is a functional gap this criterion cannot fully capture on its own.
COGA Obj. 3–5	TODO	`VisualSchedule.tsx` and `StepByStepGuidance.tsx` confirmed to exist and render. Whether the result is actually clear before starting a lesson is a judgement call, carried into the manual spot-check matrix (Table 6.4, items MSC-06 and MSC-07) rather than answered here.

6.4.4 User acceptance testing: personas and form

Six people, each briefed to role-play one persona rather than to test the whole platform, stand in for the target users this project was not able to recruit directly (Section 4.6). The form below is a single Google Form: one role-select question routes each respondent to their persona's own section, every section carries the same four Likert statements so results are comparable across personas, and one shared open-ended question closes every path.

Table 6.6: UAT Form Structure

Section	Content
0. Role select	Single-choice question: "Which persona were you asked to role-play?" with the six options in Table 6.8. Routes to the matching section 1–6 below.
1–6. Per-persona (one per row of Table 6.8)	The persona's task brief, shown as a reminder, followed by the four Likert statements in Table 6.7, each answered on a 5-point Strongly Disagree – Strongly Agree scale.
7. Closing (shared, every section routes here)	One open-text question: "What felt missing or could be improved for this role?"

Table 6.7: UAT Likert Statements (asked once per persona)

#	Statement (5-point Strongly Disagree – Strongly Agree)
L1	I was able to complete the tasks in this brief.
L2	Finding what I needed in the interface was easy.
L3	The accessibility settings or tools available met my needs for these tasks.
L4	I felt confident using the features specific to this role.

Table 6.8: UAT Persona Task Briefs

Persona	Task brief
Learner – Dyslexia	(1) Apply the Dyslexia accessibility preset from your profile settings. (2) Open any lesson and read through it under the applied preset. (3) Use the Listen (text-to-speech) control to hear part of the lesson read aloud. (4) Adjust one individual setting (for example font size or word spacing) after the preset is applied, and confirm it takes effect immediately.
Learner – ADHD	(1) Apply the ADHD accessibility preset. (2) Enrol in a course and open a lesson, noting the distraction-free, chunked view. (3) Work through a lesson's task checklist step by step. (4) Take that lesson's quiz and note the visible timer.
Learner – Autism	(1) Apply the Autism accessibility preset. (2) Before opening a lesson, review its visual schedule. (3) Navigate between two lessons in the same course and compare their layout and structure. (4) Complete a lesson and check your progress on the Progress page.
Learner – Baseline (no preset)	(1) Browse the course catalogue and enrol in a course with no accessibility preset active. (2) Complete a full lesson. (3) Take that lesson's quiz. (4) View your certificate or a newly earned achievement.
Educator	(1) Create a new course (title, description, difficulty level). (2) Author a lesson that includes an image, then run the accessibility compliance auditor on it before publishing and address at least one flagged issue. (3) Open your course analytics dashboard. (4) Issue or review a certificate for a learner in one of your courses.
Administrator	(1) Review a pending instructor application and approve or reject it. (2) Moderate a course awaiting review (approve or reject its publication). (3) Open the platform-wide analytics view, including accessibility adoption figures. (4) Review and respond to a contact-form enquiry.

6.5 Test Results and Analysis

6.5.1 Automated end-to-end suite: results

The suite was run on 2 September 2026 against the deployed application, driving a real Chromium browser. Eight of nine scenarios pass. Table 6.9 reports the actual outcome of that run, including the one that did not pass and why. Two further, unplanned findings surfaced only because the suite drove a real browser rather than calling the database directly: a lesson genuinely cannot be marked complete under the ADHD preset (below), and the quiz variant a Dyslexia-profile learner is routed into leaks raw, untranslated interface text on a wrong answer. Neither would have been visible to a database-level check, because both are failures of what actually renders.

Table 6.9: Automated Suite Results

ID	Status	Actual Result
PUBLIC-01	**Pass**	Landing page rendered with real content (course and learner counts drawn from live figures). `/courses` redirected to `/login?redirect=%2Fcourses`: there is no public catalogue route at all, only a login wall behind the "Explore Courses" call to action. This contradicts Section 4.2.3's claim that unauthenticated visitors reach "the course catalogue preview", and that claim should be corrected or the route should be built.
LEARNER-01	**Pass**	Enrolled in "Reading Fluency with Dyslexia Support" through the real UI. The Enroll in Course button disappeared and an active enrolment was confirmed in the database, then removed in teardown.
LEARNER-02	**Fail**	The ADHD preset applied correctly and was confirmed active on the profile. On the lesson page, every clickable element was enumerated (step tabs, Slide View, Listen, Exit Distraction Free Mode) and none of them completes the lesson: "Complete Lesson" appears only as inert checklist text, not a button, link or role=button element of any kind. A learner using the ADHD preset cannot finish this lesson through the interface at all. The equivalent baseline (no-preset) page for the same lesson was confirmed separately to have a real, working Complete Lesson button.

continued on the next page

Table 6.9 continued from the previous page

ID	Status	Actual Result
LEARNER-03	**Pass**	Started a real attempt on "Foundations Review Quiz" under the Dyslexia preset. Answering question 1 correctly, the quiz advanced and continued to show quiz content. Exploring the same flow with a deliberately wrong answer (a separate, non-scored run) reproduced a genuine defect: the wrong-answer retry state shows raw, untranslated strings (`quiz.tryAgain`, `quiz.reviewLesson`, four instances of `quiz.questionTooltip`) as literal on-screen text, in the one quiz variant built specifically for a simplified-reading-level learner.
LEARNER-04	**Pass**	Certificates page listed 2 certificates: a system certificate (reference `ZULU-OYGX-GZET`, "Foundations of Accessible Learning", instructor named, completion date, downloadable) and an educator-issued certificate, both rendered correctly.
EDUCATOR-01 & 02	**Pass**	Created a course through all four real wizard steps, and it appeared in the real course list immediately after. Opened its Edit view, set Primary Accessibility Focus to ADHD on the Settings tab and clicked Save Focus. The database confirmed `primary_disability_focus = 'adhd'`. Test course deleted in teardown.
EDUCATOR-03	**Pass**	Analytics page rendered real per-course figures for all of siti's published courses, for example "Focus and Study Skills for ADHD Learners": 4 enrolled, 1 active, 0 at risk, 60% average progress, 25% completion rate, 88% average quiz score.
ADMIN-01	**Pass**	Approved the pending application from "Tan Chee Meng" through the real UI, including its confirmation dialog. The database confirmed `status = 'approved'` afterwards. One finding along the way: `reviewed_by` was left `NULL` by this approval path, unlike the `updateInstructorApplication()` function in `admin-api.ts`, which explicitly sets it, so the audit trail for who approved an application is incomplete on at least one code path. A second finding, found only by reading the database rather than the UI: approval provisions a real educator account for the applicant. Both the account and the application's approval were reverted in teardown.

6.5.2 Manual spot-check: results

TODO. Table 6.4's Actual Result and Status columns are not yet filled in.

6.5.3 WCAG 2.2 expert review: results

The objective findings in Table 6.5 are final as reported, and two of them are now backed by a real rendered browser rather than code reading alone. **1.2.1 Video-only** is a live Fail (2 of 7 published lessons with video have no transcript, against the auditor's own rule). **2.2.1 Timing Adjustable** is a Fail that contradicts Table 4.3's existing claim (confirmed in code: a baseline, no-preset learner's quiz auto-submits with no warning when the timer reaches zero, and only a learner with a preset active gets the Extend/Submit-now prompt the table describes). **2.2.4 Interruptions** is now Partial rather than TODO: the automated suite found, while applying the ADHD preset, that distraction-free mode removes the sidebar across the whole application once active, not only inside a lesson as Table 4.3 states, which needed the project author's own account to be reset by hand mid-session to continue testing. **TODO:** the remaining rows marked TODO in Table 6.5 need the project author's sign-off, since they call for a screen reader, an assistive-technology session, or a subjective read of an actual sensory experience that a scripted browser session cannot supply.

6.5.4 User acceptance testing: results

TODO. No form responses have been collected yet.

6.6 Conclusion

This chapter is not yet complete, and its overall analysis is deliberately not written until the remaining evidence exists: writing a summary conclusion now would mean characterising manual, expert-review and user-acceptance results that do not exist yet.

What can be said now, because it is the one class of evidence that is complete, is that the automated end-to-end suite ran against a real browser and the real deployed application, not a substitute for one, and it found two genuine functional defects that a database-only check would have missed entirely: a lesson that cannot be completed at all under the ADHD preset, and raw interface text leaking through the Dyslexia-profile quiz view on a wrong answer. Assembling the WCAG review also surfaced a Fail that contradicts an existing claim elsewhere in this report (Table 4.3, criterion 2.2.1) and needs a decision on whether that claim should be corrected. Once the manual spot-check, the WCAG review's subjective rows, and the UAT responses are supplied, this section will be rewritten to state what the three classes together show, including where they disagree with each other or with earlier chapters.

CHAPTER 7: CONCLUSION

7.1 Introduction

This chapter closes the report. It restates the objectives set out in Chapter 1 and reports how design, implementation and testing addressed each one, ties the project's outcomes back to the contributions it set out to make, states the limitations that remain after implementation and testing rather than only after design, sets out realistic future work, and closes with a short statement on the project as a whole.

7.2 Project Summarization

Chapter 1 set four objectives. The first, to analyse the accessibility requirements of dyslexic, ADHD and autistic learners against existing platforms and standards, was carried out in Chapter 2 through the comparison of four established LMS platforms and a mapping of WCAG 2.2, W3C COGA guidance and the British Dyslexia Association Style Guide onto specific interface interventions, and carried through into Chapter 4 as the standards baseline the accessibility design and the compliance auditor both cite by criterion number.

The second, to build a functional platform with rule-based adaptive delivery, text-to-speech, adjustable accessibility settings, lesson and quiz management, and role-based access for three user types, was achieved and is the bulk of Chapters 4 and 5: the accessibility personalisation engine, the recommendation engine, the quiz and grading system, and the learner, educator and administrator roles are all implemented and, per Table 5.3, complete or complete with a stated limitation, with only continuous integration, automated accessibility testing, composable presets and offline support unimplemented.

The third, to implement certificate generation and educator analytics for verifiable completion and progress monitoring, was achieved: certificates carry a public reference code and verification page, and educator analytics aggregates progress and flags at-risk

students per course.

The fourth, to evaluate functional correctness, usability and accessibility through structured walkthroughs and expert review, was achieved in the form actually available to a solo developer, and Chapter 6 says so plainly rather than overstating it: walkthroughs, executable probes and self-assessment against WCAG 2.2 criteria were carried out, but no structured session with an independent expert or a real neurodivergent learner took place.

The project's central strength is that its claims are checkable rather than asserted: the schema is rebuildable from migrations, the demonstration data from a deterministic seed, and the security and behavioural properties from executable scripts and a documented manual audit that together found and fixed sixteen real defects, two of them privilege-escalation holes that would have been serious in a deployed system. Its central weakness is the one Chapter 4 and Chapter 6 both already name: nothing in this report is evidence that the system helps the learners it was built for, only that it behaves as designed for someone without their specific needs testing it.

7.3 Project Contribution

For the university and faculty, the project is a submitted final-year project artefact together with its own report, and it demonstrates a build-and-verify method, generating diagrams and data-dictionary tables directly from the live schema and backing implementation claims with executable scripts, that could reasonably inform how other accessibility-focused or data-heavy student projects are supervised and assessed.

For the research community, the project's most transferable contribution is empirical rather than theoretical: several of the sixteen fixed defects in Chapter 6, among them a CSS cascade that silently discarded a learner's own spacing settings and an accessibility mode that had never been reachable from the interface, were runtime, lifecycle or reachability problems invisible to code review, in features already marked complete. That is a concrete illustration of why static verification of accessibility claims is insufficient, consistent with the caution already raised against unverified accessibility claims in Section 2.2.2.

For industry, the compliance auditor and the settings-catalogue invariant enforced by `npm run test:a11y-catalog` are patterns, not just features: scoring authored content against a fixed, cited rule set at authoring time, and refusing to let a setting exist without either a description or an explicit exclusion, are both applicable to any content-authoring platform that wants to prevent accessibility controls from silently doing nothing.

For individual users, the immediate contribution is the platform itself: a learner receives three profile-appropriate presets that measurably change typography, layout, pacing and motion; an educator receives authoring tools with a compliance auditor built in; and an administrator receives the oversight and moderation tools to run the platform responsibly. The user-facing accessibility statement, published at `/accessibility-statement`, is where a learner or educator can read what the platform supports and what it does not, and the source code, migrations and the scripts referenced throughout Chapters 4 to 6 are the project's user manual in the sense that matters for a system of this kind: the definitive description of its behaviour is the behaviour itself, kept reproducible rather than left to drift from a separate document.

7.4 Project Limitation

Section 4.6 already sets out the limitations visible at the design stage, and they are not repeated here. Implementation and testing surfaced further limitations that design alone could not have shown:

- **Verification depends entirely on the developer.** Every executable script, every manual test case and every defect fix in Chapter 6 was written, run and judged by the same person who built the feature being tested, with no independent reviewer and no automated gate preventing a regression from reaching the deployed system.
- **No assistive technology was exercised against the system.** Screen-reader behaviour, in particular, is asserted from semantic markup and ARIA attributes rather than observed with a real screen reader, because none was available in the testing environment.

- **Defects were found in features already believed complete.** Three of the sixteen fixed defects, and the two Easy Read defects among them, were in accessibility controls that had already been marked done before this project's own testing found them inert. This is a limitation of process, not just of the current build: nothing in the project's workflow catches this class of defect before a live pass happens to find it.

- **A known performance cost was left unfixed.** Eighteen redundant authentication round-trips on a single dashboard load were measured and are recorded as correct but wasteful, left unaddressed because fixing the shared pattern behind all of the learner-facing data functions that trigger it carried more regression risk than the remaining project time could absorb safely.

7.5 Future Works

- **Independent evaluation with neurodivergent learners.** The single most valuable next step is a structured usability study with real dyslexic, ADHD and autistic participants, using the existing presets as the intervention and a validated instrument such as the System Usability Scale (Brooke, 1996) to produce the first outcome measurement this project does not have.

- **Independent accessibility audit.** An audit against WCAG 2.2 AA by a party other than the developer, ideally including screen-reader testing with NVDA or VoiceOver, would convert the self-assessment in Table 4.3 into a verified conformance claim.

- **Composable accessibility profiles.** Allowing a learner to combine settings from more than one preset, rather than choosing exactly one, was identified as the clearest design extension in Section 4.6 and remains unimplemented.

- **A continuous integration pipeline.** The five executable scripts in Table 5.4 already exist and already catch real regressions; wiring them into a pipeline that runs automatically on every push, rather than by hand, would close a gap

Chapter 6 identifies: every test in this project, automated or manual, was still run by hand by a single person.

- **Reducing the authentication round-trip cost.** Consolidating the learner-facing data functions in `learner-api.ts` onto a single cached session read, rather than one `auth.getUser()` call each, is a scoped, well-understood fix that this project's remaining time could not absorb safely.
- **Completing localisation.** The accessibility settings panel's own field labels remain English-only. Translating them, and extending translation to authored course content where an educator chooses to provide it, would close the one part of the platform where a second-language learner is least served.

7.6 Conclusion

ACESS set out to show that an e-learning platform can adapt to a neurodivergent learner's cognitive profile rather than asking the learner to adapt to it, and to make that adaptation something implemented and checkable rather than promised. The objectives set in Chapter 1 were achieved to the extent a single-developer final-year project can achieve them: the platform is built, its behaviour is reproducible from source, and sixteen real defects that would have undermined its accessibility or security claims were found and fixed before submission. What remains, honestly stated throughout this report rather than only here, is evidence from the people the platform was built for. That gap does not diminish what was built; it defines what should happen to it next.

References

Anthology Inc. (2023). Blackboard learn. Accessed 2 September 2026, <https://www.blackboard.com/>.

Anthology Inc. (2024a). Anthology ally: Accessibility reporting and alternative formats. Accessed 2 September 2026, <https://help.blackboard.com/Ally>.

Anthology Inc. (2024b). Blackboard learn: Achievements. Accessed 2 September 2026, <https://help.blackboard.com/Learn/Instructor/Ultra/Performance/Achievements>.

Barkley, R. A. (1997). Implications of executive function deficits for treatment of adhd children: Empirical and practical issues. *Journal of Clinical Child Psychology*, 26(2):129–140.

Batanero, C., Boticario, J. G., et al. (2014). Accessibility of learning management systems: A review. *Journal of Universal Computer Science*, 20(9):1253–1272.

Bernier, Claudéric (2025). dnd kit — a modern drag and drop toolkit for react. Accessed 2 September 2026, <https://dndkit.com/>.

Braille Institute of America (2021). Atkinson hyperlegible font. Accessed 2 September 2026, <https://brailleinstitute.org/freefont>.

Brainhub (2024). Tiptap: The headless rich text editor framework. Accessed 2 September 2026, <https://tiptap.dev/>.

Brickfield Education Labs (2024). Accessibility toolkit for moodle (brickfield) — plugin directory entry. Accessed 2 September 2026, <https://moodle.org/plugins/tool_brickfield>.

British Dyslexia Association (2023). Dyslexia style guide 2023: Creating dyslexia friendly content. Accessed 2 September 2026, <https://www.bdadyslexia.org.uk/advice/employers/creating-a-dyslexia-friendly-workplace/dyslexia-friendly-style-guide>.

Brooke, J. (1996). Sus: A “quick and dirty” usability scale. In Jordan, P. W., Thomas, B., Weerdmeester, B. A., and McClelland, I. L., editors, *Usability evaluation in industry*, pages 189–194. Taylor & Francis.

Burl, M. and O’Brien, K. (2024). Dropout rates and accessibility barriers in e-learning for neurodivergent students. *Computers & Education*, 198:104–118.

Docker, Inc. (2025). Docker desktop documentation. Accessed 2 September 2026, <https://docs.docker.com/desktop/>.

Figma (2023). Figma: The collaborative interface design tool. Accessed 2 September 2026, <https://www.figma.com/>.

Framer (2024). Framer motion: Production-ready motion library for react. Accessed 2 September 2026, <https://www.framer.com/motion/>.

Franceschini, S., Gori, S., Ruffino, M., Viola, S., Molteni, M., and Facoetti, A. (2013). Action video games make dyslexic children read better. *Current Biology*, 23(6):462–466.

Gonzalez, Abelardo (2024). Opendyslexic: A typeface for dyslexia. Accessed 2 September 2026, <https://opendyslexic.org/>.

Google DeepMind (2025). Gemini api documentation. Accessed 2 September 2026, <https://ai.google.dev/gemini-api/docs>.

Google LLC (2023). Google classroom. Accessed 2 September 2026, <https://classroom.google.com/>.

Google LLC (2024). Use google classroom with a screen reader and accessibility tools. Accessed 2 September 2026, <https://support.google.com/edu/classroom/answer/6272747>.

Google LLC (2025). Youtube iframe player api reference. Accessed 2 September 2026, <https://developers.google.com/youtube/iframe_api_reference>.

Heiderich, Mario and Cure53 (2025). Dompurify: A dom-only xss sanitizer for html. Accessed 2 September 2026, <https://github.com/cure53/DOMPurify>.

Instructure, Inc. (2023). Canvas learning management system. Accessed 2 September 2026, <https://www.instructure.com/canvas>.

Instructure, Inc. (2024a). Canvas accessibility standards and voluntary product accessibility template. Accessed 2 September 2026, <https://www.instructure.com/canvas/accessibility>.

Instructure, Inc. (2024b). How do i use microsoft immersive reader in a canvas course? Accessed 2 September 2026, <https://community.canvaslms.com/t5/Instructor-Guide/tkb-p/Instructor>.

Instructure, Inc. (2024c). How do i use the accessibility checker in the rich content editor? Accessed 2 September 2026, <https://community.canvaslms.com/t5/Instructor-Guide/tkb-p/Instructor>.

Kincaid, J. P., Fishburne, R. P., Rogers, R. L., and Chissom, B. S. (1975). Derivation of new readability formulas (automated readability index, fog count and flesch reading ease formula) for navy enlisted personnel. Technical Report Research Branch Report 8-75, Naval Technical Training Command, Millington TN Research Branch.

Lawson, W. (2011). *The Passionate Mind: How People with Autism Learn*. London: Jessica Kingsley Publishers.

Meta Platforms, Inc. (2023). React: The library for web and native user interfaces. Accessed 2 September 2026, <https://react.dev/>.

Meyer, A., Rose, D. H., and Gordon, D. (2014). *Universal Design for Learning: Theory and Practice*. Wakefield, MA: CAST Professional Publishing.

Microsoft (2023). Typescript: Javascript with syntax for types. Accessed 2 September 2026, <https://www.typescriptlang.org/docs/>.

MoodleHQ (2023a). Moodle architecture: Technical overview. Accessed 2 September 2026, <https://docs.moodle.org/dev/Architecture>.

MoodleHQ (2023b). Moodle: Open-source learning platform. Accessed 2 September 2026, <https://moodle.org/>.

MoodleHQ (2024a). Moodle accessibility checker (atto and tinymce editor plugins). Accessed 2 September 2026, <https://docs.moodle.org/en/Accessibility_checker>.

MoodleHQ (2024b). Moodle accessibility documentation. Accessed 2 September 2026, <https://docs.moodle.org/en/Accessibility>.

Mozilla Developer Network (2025). Web speech api — speech synthesis. Accessed 2 September 2026, <https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API>.

Munoz, D. P., Armstrong, I. T., Hampton, K. A., and Moore, K. D. (2003). Saccadic eye movements in attention deficit hyperactivity disorder. *Clinical Neurophysiology*, 114(9):1667–1674.

Parallax Agency Ltd. (2025). jspdf: Client-side javascript pdf generation. Accessed 2 September 2026, <https://github.com/parallax/jsPDF>.

PostgreSQL Global Development Group (2025). Postgresql 17 documentation. Accessed 2 September 2026, <https://www.postgresql.org/docs/17/>.

PostgREST Contributors (2025). Postgrest: Rest api for any postgresql database. Accessed 2 September 2026, <https://postgrest.org/>.

Prensky, M. (2001). Digital natives, digital immigrants part 1. *On the Horizon*, 9(5):1–6.

Recharts Team (2024). Recharts: A composable charting library built on react components. Accessed 2 September 2026, <https://recharts.org/>.

Rello, L. and Baeza-Yates, R. (2013). Good fonts for dyslexia. In *Proceedings of the 15th International ACM SIGACCESS Conference on Computers and Accessibility*, pages 1–8.

Schwaber, K. (1997). Scrum development process. In *Business Object Design and Implementation: OOPSLA'95 Workshop Proceedings*, pages 117–134. Springer.

Spencer, M. and Podesta, F. (2022). Neurodiversity in higher education: A comprehensive review of progress and challenges. *Journal of Applied Higher Education*, 13(2):45–62.

Supabase (2023). Supabase documentation: The open source firebase alternative. Accessed 2 September 2026, <https://supabase.com/docs>.

Supabase (2024). Supabase row level security documentation. Accessed 2 September 2026, <https://supabase.com/docs/guides/auth/row-level-security>.

Supabase Inc. (2025a). Supabase auth documentation. Accessed 2 September 2026, <https://supabase.com/docs/guides/auth>.

Supabase Inc. (2025b). Supabase cli and local development. Accessed 2 September 2026, <https://supabase.com/docs/guides/local-development>.

Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive science*, 12(2):257–285.

Tailwind Labs (2023). Tailwind css documentation. Accessed 2 September 2026, <https://tailwindcss.com/docs>.

Vercel Inc. (2023a). Next.js documentation: Server-side rendering (ssr). Accessed 2 September 2026, <https://nextjs.org/docs>.

Vercel Inc. (2023b). Vercel documentation: The platform for frontend developers. Accessed 2 September 2026, <https://vercel.com/docs>.

Vercel Inc. (2024). Next.js middleware documentation. Accessed 2 September 2026, <https://nextjs.org/docs/app/building-your-application/routing/middleware>.

W3C Web Accessibility Initiative (2021). Making content usable for people with cognitive and learning disabilities. Accessed 2 September 2026, <https://www.w3.org/TR/coga-usable/>.

W3C Web Accessibility Initiative (2023). Web content accessibility guidelines (wcag) 2.2. Accessed 2 September 2026, <https://www.w3.org/TR/WCAG22/>.

Zelazo, P. D., Muller, U., Frye, D., and Marcovitch, S. (2003). The development of executive function in early childhood. *Monographs of the Society for Research in Child Development*, 68(3):vii–137.